

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng nền tảng quản lý dự án cộng tác thời gian
thực dựa trên mô hình Kanban tích hợp trợ lý AI**

TRỊNH HỮU AN

an.th225593@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Lê Đức Trung

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng nền tảng quản lý dự án cộng tác thời gian
thực dựa trên mô hình Kanban tích hợp trợ lý AI**

TRỊNH HỮU AN

an.th225593@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Lê Đức Trung

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Lời đầu tiên, em xin được bày tỏ lòng biết ơn sâu sắc nhất tới ThS Lê Đức Trung, người thầy đã luôn tận tình chỉ bảo, định hướng và đồng hành cùng em trong suốt quá trình nghiên cứu và hoàn thành đồ án tốt nghiệp này. Sự nghiêm túc trong khoa học cùng những lời khuyên quý báu của thầy không chỉ giúp em hoàn thành đề tài mà còn là hành trang quý giá cho em trên con đường sự nghiệp tương lai.

Em cũng xin gửi lời tri ân chân thành tới các thầy cô giáo tại Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội, những người đã truyền dạy cho em tri thức, khơi dậy niềm đam mê khoa học và tạo điều kiện học tập tốt nhất cho em trong những năm tháng thanh xuân dưới mái trường Bách khoa.

Con xin gửi lời cảm ơn sâu sắc nhất tới bố mẹ và gia đình thân yêu. Gia đình luôn là điểm tựa vững chắc nhất, là nơi tiếp thêm cho con niềm tin và sức mạnh để vượt qua những giai đoạn áp lực và khó khăn nhất trong cuộc sống cũng như trong chặng đường học tập này.

Đặc biệt, tôi xin gửi lời cảm ơn chân thành đến những người bạn đã luôn đồng hành cùng tôi trong suốt quá trình thực hiện đồ án: Khải, Huy, Long, Hiếu, Minh và Hương. Cảm ơn các bạn vì đã cùng tôi chia sẻ những ngày làm việc căng thẳng, những đêm thức khuya để sửa lỗi, những lúc bế tắc khi chương trình chưa hoạt động như mong muốn, cũng như niềm vui khi từng vấn đề khó dần được giải quyết. Sự động viên, góp ý và hiện diện của các bạn là nguồn động lực rất lớn, giúp tôi có thêm tinh thần để kiên trì hoàn thành đồ án này.

Cuối cùng, tôi muốn dành một lời cảm ơn chân thành đến chính bản thân mình, vì đã kiên trì, không từ bỏ và luôn nỗ lực hết mình để bước tiếp và hoàn thiện chặng đường ý nghĩa này.

Mặc dù đã cố gắng hoàn thiện đồ án với tất cả sự tâm huyết, nhưng do kiến thức và kinh nghiệm còn hạn chế nên bài viết không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp, chỉ bảo quý báu của các thầy cô giáo.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh làm việc nhóm từ xa và phát triển phần mềm theo phương pháp Agile ngày càng phổ biến, nhu cầu về một nền tảng quản lý công việc trực quan, cập nhật kịp thời và hỗ trợ cộng tác hiệu quả trở nên ngày càng cần thiết. Nhiều hệ thống quản lý dự án hiện nay đã hỗ trợ mô hình Kanban, phân công nhiệm vụ và theo dõi tiến độ, tuy nhiên vẫn còn một số hạn chế như việc đồng bộ thay đổi giữa các thành viên chưa thật sự tức thời, khó quan sát mối quan hệ phụ thuộc giữa các công việc, cũng như còn thiếu các công cụ hỗ trợ tự động hóa trong quá trình tạo, mô tả và tổng hợp thông tin nhiệm vụ.

Từ thực tế đó, đồ án lựa chọn hướng tiếp cận xây dựng nền tảng quản lý dự án cộng tác thời gian thực HustFlow dựa trên mô hình bảng Kanban, đồng thời tích hợp trí tuệ nhân tạo để hỗ trợ một số tác vụ quản lý thường gặp. Hướng tiếp cận này được lựa chọn vì vừa giữ được sự trực quan, dễ sử dụng của Kanban, vừa giúp cải thiện khả năng phối hợp giữa các thành viên và giảm bớt các thao tác thủ công trong quá trình quản lý dự án.

Hệ thống HustFlow được thiết kế với các chức năng chính như quản lý không gian làm việc, bảng, danh sách và thẻ công việc; phân quyền thành viên; bình luận; nhật ký hoạt động; quản lý nhãn, checklist, tệp đính kèm và quan hệ phụ thuộc giữa các thẻ. Bên cạnh đó, hệ thống tích hợp cơ chế đồng bộ thời gian thực nhằm cập nhật các thay đổi trên bảng tới các thành viên đang cùng làm việc mà không cần tải lại trang. Ngoài ra, trợ lý AI được sử dụng để hỗ trợ tạo thẻ từ văn bản tự nhiên, chuẩn hóa mô tả công việc và tổng hợp báo cáo dựa trên dữ liệu hoạt động của bảng.

Đóng góp chính của đồ án là thiết kế và triển khai một nền tảng quản lý công việc có tính ứng dụng thực tế, kết hợp giữa mô hình Kanban, cộng tác thời gian thực, quản lý phụ thuộc công việc và hỗ trợ thông minh bằng AI. Kết quả đạt được cho thấy hệ thống đáp ứng được các nghiệp vụ chính, hoạt động ổn định trong phạm vi thử nghiệm và có khả năng tiếp tục mở rộng để phục vụ các nhóm phát triển phần mềm, nhóm học tập hoặc các tổ chức vừa và nhỏ.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Trịnh Hữu An

ABSTRACT

In the context of growing remote teamwork and Agile software development, the demand for an intuitive, real-time, and collaborative task management platform has become increasingly essential. While many current project management systems support the Kanban model, task assignment, and progress tracking, they still present several limitations, such as non-instantaneous synchronization of changes among team members, difficulty in visualizing dependencies between tasks, and a lack of automated features in task creation, description, and information synthesis.

Based on these practical needs, this thesis presents the development of HustFlow, a real-time collaborative project management platform based on the Kanban board model, integrated with artificial intelligence to assist with common management tasks. This approach is chosen because it preserves the simplicity and intuitiveness of the Kanban model, while simultaneously enhancing team collaboration and reducing manual operations in project management.

The HustFlow system is designed with core functionalities including the management of workspaces, boards, lists, and cards; member authorization; comments; activity logs; and the management of labels, checklists, attachments, and card dependencies. In addition, the platform integrates a real-time synchronization mechanism to update board modifications for active members without requiring a page refresh. Furthermore, an AI assistant is incorporated to support card creation from natural language text, standardization of task descriptions, and report generation based on board activity data.

The primary contribution of this thesis is the design and implementation of a practical task management platform that integrates the Kanban model, real-time collaboration, task dependency management, and intelligent AI assistance. Experimental results demonstrate that the system satisfies the primary functional requirements, operates stably within the testing scope, and is highly extensible to serve software development teams, academic groups, or small-to-medium organizations.

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	vii
DANH MỤC THUẬT NGỮ	ix
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.2 Tổng quan chức năng	6
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã.....	9
2.2.3 Quy trình nghiệp vụ	16
2.3 Đặc tả chức năng	18
2.3.1 Đặc tả use case 1: Tạo bảng mới	18
2.3.2 Đặc tả use case 2: Xem và cập nhật chi tiết thẻ.....	19
2.3.3 Đặc tả use case 3: Kéo thả và di chuyển thẻ	20
2.3.4 Đặc tả use case 4: Thiết lập quan hệ phụ thuộc giữa các thẻ	21
2.3.5 Đặc tả use case 5: Bình luận trong thẻ.....	22
2.3.6 Đặc tả use case 6: Theo dõi tiến độ và báo cáo.....	23
2.3.7 Đặc tả use case 7: Tạo thẻ tự động từ văn bản thô.....	23
2.4 Yêu cầu phi chức năng	25
2.4.1 Tính hiệu năng.....	25
2.4.2 Bảo mật và phân quyền.....	26

2.4.3 Độ tin cậy và tính toàn vẹn dữ liệu	26
2.4.4 Tính dễ dùng	27
2.4.5 Tính bảo trì và mở rộng	27
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	29
3.1 Nền tảng lý thuyết.....	29
3.1.1 Đồ thị có hướng phi chu trình và phát hiện chu trình	29
3.1.2 Mô hình cơ sở dữ liệu quan hệ và toàn vẹn dữ liệu.....	30
3.2 Các công nghệ phát triển ứng dụng.....	31
3.2.1 Next.js và React	31
3.2.2 MySQL và Prisma ORM	32
3.2.3 Pusher cho đồng bộ thời gian thực	32
3.2.4 Clerk cho xác thực và phân quyền.....	33
3.2.5 Stripe cho thanh toán gói dịch vụ.....	33
3.2.6 OpenAI API cho trợ lý trí tuệ nhân tạo	34
3.2.7 @hello-pangea/dnd cho tương tác kéo thả	34
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	36
4.1 Thiết kế kiến trúc.....	36
4.1.1 Lựa chọn kiến trúc phần mềm	36
4.1.2 Thiết kế tổng quan.....	38
4.1.3 Thiết kế chi tiết gói quản lý thẻ và cộng tác	41
4.2 Thiết kế chi tiết.....	42
4.2.1 Thiết kế giao diện	42
4.2.2 Thiết kế lớp	44
4.2.3 Thiết kế cơ sở dữ liệu	52

4.3 Xây dựng ứng dụng.....	55
4.3.1 Thư viện và công cụ sử dụng.....	55
4.3.2 Kết quả đạt được	56
4.3.3 Minh họa các chức năng chính	58
4.4 Kiểm thử.....	61
4.4.1 Phương pháp và môi trường kiểm thử.....	61
4.4.2 Thiết kế ca kiểm thử chức năng	62
4.4.3 Tổng hợp kết quả.....	63
4.5 Triển khai	63
4.5.1 Môi trường chạy thử nghiệm	63
4.5.2 Mô hình chạy thử nghiệm cục bộ.....	64
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	67
5.1 Đồng bộ dữ liệu thời gian thực có kiểm soát.....	67
5.1.1 Bài toán đặt ra	67
5.1.2 Giải pháp đề xuất.....	68
5.1.3 Kết quả đạt được	70
5.2 Kiểm soát quan hệ phụ thuộc bằng đồ thị có hướng.....	70
5.2.1 Bài toán đặt ra	70
5.2.2 Giải pháp đề xuất.....	71
5.2.3 Kết quả đạt được	73
5.3 Tích hợp trí tuệ nhân tạo theo nguyên tắc sinh bản nháp có ràng buộc	73
5.3.1 Bài toán đặt ra	73
5.3.2 Giải pháp đề xuất.....	74
5.3.3 Kết quả đạt được	76
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	77
6.1 Kết luận.....	77

6.2 Hướng phát triển.....	78
TÀI LIỆU THAM KHẢO.....	80

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống HustFlow	7
Hình 2.2	Biểu đồ use case phân rã Quản lý tổ chức	9
Hình 2.3	Biểu đồ use case phân rã Quản lý bảng	10
Hình 2.4	Biểu đồ use case phân rã Quản lý thành viên và phân quyền bảng	11
Hình 2.5	Biểu đồ use case phân rã Quản lý danh sách và thẻ công việc .	12
Hình 2.6	Biểu đồ use case phân rã Cộng tác thời gian thực	13
Hình 2.7	Biểu đồ use case phân rã Theo dõi tiến độ và báo cáo	14
Hình 2.8	Biểu đồ use case phân rã Sử dụng trợ lý AI	15
Hình 2.9	Biểu đồ use case phân rã Quản lý gói dịch vụ và thanh toán . .	16
Hình 2.10	Biểu đồ hoạt động quy trình quản lý thẻ công việc, kiểm tra phụ thuộc và đồng bộ thời gian thực	17
Hình 4.1	Kiến trúc tổng thể của hệ thống HustFlow	37
Hình 4.2	Biểu đồ gói và chiều phụ thuộc giữa các mô-đun	39
Hình 4.3	Thiết kế chi tiết nhóm gói quản lý thẻ và cộng tác	41
Hình 4.4	Bản phác thảo bố cục không gian làm việc chính	43
Hình 4.5	Bản phác thảo giao diện cho các tình huống sử dụng quan trọng	44
Hình 4.6	Biểu đồ trình tự cập nhật thuộc tính thẻ và đồng bộ thời gian thực	49
Hình 4.7	Biểu đồ trình tự tạo và kiểm tra quan hệ phụ thuộc	50
Hình 4.8	Biểu đồ trình tự phân tích và tạo thẻ bằng Smart Capture . . .	52
Hình 4.9	Sơ đồ thực thể liên kết của cơ sở dữ liệu HustFlow	53
Hình 4.10	Giao diện Kanban tổng quan của HustFlow	59
Hình 4.11	Giao diện chi tiết thẻ công việc	59
Hình 4.12	Giao diện Lịch với khung thẻ chưa lên lịch	60
Hình 4.13	Giao diện lập kế hoạch theo dòng thời gian	60
Hình 4.14	Bản nháp thẻ do Smart Capture tạo ra	61
Hình 4.15	Giao diện thống kê và báo cáo của bảng	61
Hình 4.16	Mô hình chạy thử nghiệm cục bộ của HustFlow, không phải môi trường production	65
Hình 5.1	Chiến lược đồng bộ và phục hồi dữ liệu thời gian thực	69
Hình 5.2	Kiểm tra đường đi ngược trước khi thêm cạnh phụ thuộc . . .	72
Hình 5.3	Quy trình kiểm soát đầu ra của trợ lý AI	75

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả use case Tạo bảng mới	18
Bảng 2.2	Đặc tả use case Xem và cập nhật chi tiết thẻ	19
Bảng 2.3	Đặc tả use case Kéo thả và di chuyển thẻ	20
Bảng 2.4	Đặc tả use case Thiết lập quan hệ phụ thuộc giữa các thẻ	21
Bảng 2.5	Đặc tả use case Bình luận trong thẻ	22
Bảng 2.6	Đặc tả use case Theo dõi tiến độ và báo cáo	23
Bảng 2.7	Đặc tả use case Tạo thẻ tự động từ văn bản thô	24
Bảng 4.1	Thiết kế thuộc tính của các lớp thực thể chủ đạo	45
Bảng 4.2	Phương thức và trách nhiệm của bốn lớp chủ đạo	48
Bảng 4.3	Thiết kế vật lý của các bảng dữ liệu cốt lõi	54
Bảng 4.4	Thư viện và công cụ chính được sử dụng trong HustFlow	55
Bảng 4.5	Dung lượng các thành phần bàn giao của HustFlow	57
Bảng 4.6	Thống kê mã nguồn HustFlow	58
Bảng 4.7	Thống kê cấu trúc mô-đun của ứng dụng	58
Bảng 4.8	Các ca kiểm thử chức năng trọng tâm	62
Bảng 4.9	Cấu hình môi trường chạy thử nghiệm HustFlow	64

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
ACID	Atomicity, Consistency, Isolation, Durability	Tính nguyên tử, nhất quán, cô lập và bền vững
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
BFS	Breadth-First Search	Tìm kiếm theo chiều rộng
CLI	Command-Line Interface	Giao diện dòng lệnh
CSDL	Database	Cơ sở dữ liệu
CSS	Cascading Style Sheets	Ngôn ngữ định kiểu theo tầng
DAG	Directed Acyclic Graph	Đồ thị có hướng phi chu trình
DFS	Depth-First Search	Tìm kiếm theo chiều sâu
DOM	Document Object Model	Mô hình đối tượng tài liệu
ERD	Entity–Relationship Diagram	Sơ đồ thực thể–liên kết
FK	Foreign Key	Khóa ngoại
GB	Gigabyte	Đơn vị dung lượng gigabyte
HD	High Definition	Độ phân giải cao
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HyperText Transfer Protocol Secure	Giao thức truyền tải siêu văn bản bảo mật
IDE	Integrated Development Environment	Môi trường phát triển tích hợp
ISO	International Organization for Standardization	Tổ chức Tiêu chuẩn hóa Quốc tế
JSON	JavaScript Object Notation	Ký pháp đối tượng JavaScript
JSONB	Binary JSON representation	Dạng biểu diễn nhị phân của dữ liệu JSON
JS	JavaScript	Ngôn ngữ lập trình JavaScript
MiB	Mebibyte	Đơn vị dung lượng mebibyte

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
ORM	Object–Relational Mapping	Ánh xạ đối tượng–quan hệ
PK	Primary Key	Khóa chính
RAM	Random-Access Memory	Bộ nhớ truy cập ngẫu nhiên
REST	Representational State Transfer	Kiểu kiến trúc chuyển giao trạng thái biểu diễn
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
TSX	TypeScript with JSX syntax	Tập TypeScript sử dụng cú pháp JSX
UC	Use Case	Cả sử dụng
UI	User Interface	Giao diện người dùng
URL	Uniform Resource Locator	Địa chỉ định vị tài nguyên thống nhất
UTC	Coordinated Universal Time	Giờ Phối hợp Quốc tế
UUID	Universally Unique Identifier	Định danh duy nhất toàn cục

DANH MỤC THUẬT NGỮ

Thuật ngữ	Giải thích
Agile	Phương pháp phát triển phần mềm theo hướng lặp và thích ứng, ưu tiên phản hồi sớm, cộng tác nhóm và điều chỉnh theo thay đổi.
App Router	Cơ chế định tuyến của Next.js tổ chức trang, bố cục và điểm xử lý theo cấu trúc thư mục app.
Audit log	Nhật ký hoạt động ghi lại thao tác, đối tượng bị tác động, người thực hiện và thời điểm xảy ra thay đổi.
Backlog	Danh sách các công việc hoặc yêu cầu chưa được hoàn thành và đang chờ sắp xếp, ưu tiên hoặc đưa vào thực hiện.
Board	Bảng công việc thuộc một tổ chức, chứa thành viên, danh sách, thẻ, nhãn và các chế độ xem tiến độ.
Board member role	Vai trò thành viên ở cấp bảng: ADMIN quản trị bảng và thành viên; MEMBER được chỉnh sửa nội dung; VIEWER (thành viên quan sát) chỉ được xem dữ liệu.
Cache	Vùng lưu dữ liệu tạm thời ở phía máy khách nhằm giảm số lần truy vấn và giúp giao diện phản hồi nhanh hơn.
Card	Thẻ biểu diễn một công việc, lưu tiêu đề, mô tả, thời gian, trạng thái, người phụ trách và các dữ liệu cộng tác liên quan.
Checklist	Danh sách các mục công việc con nằm trong một thẻ, dùng để theo dõi mức độ hoàn thành chi tiết.
Client Component	Thành phần React chạy ở phía trình duyệt, được dùng cho giao diện cần trạng thái, sự kiện và tương tác trực tiếp với người dùng.
Dashboard	Màn hình tổng hợp các chỉ số và biểu đồ để người dùng theo dõi tình trạng, tiến độ và phân bổ công việc.
Dependency	Quan hệ phụ thuộc giữa hai thẻ, trong đó một thẻ đóng vai trò chặn và thẻ còn lại là công việc bị chặn.

Thuật ngữ	Giải thích
Drag and drop	Cơ chế kéo và thả dùng để sắp xếp hoặc di chuyển danh sách, thẻ và các phần tử công việc trên giao diện.
Full-stack	Cách phát triển kết hợp giao diện, xử lý phía máy chủ và truy cập dữ liệu trong cùng một hệ thống ứng dụng.
Gantt chart	Biểu đồ tiến độ thể hiện công việc theo trục thời gian, hỗ trợ quan sát thời lượng và quan hệ trước-sau.
Hook	Hàm của React hoặc hàm tùy biến đóng gói trạng thái và logic có thể tái sử dụng giữa các thành phần giao diện.
Kanban	Phương pháp quản lý công việc trực quan bằng các cột trạng thái và thẻ, cho phép theo dõi luồng công việc qua từng giai đoạn.
Label	Nhãn màu gắn với thẻ để phân loại, nhận diện hoặc lọc công việc theo nhóm ý nghĩa.
List	Danh sách hoặc cột trong bảng Kanban, chứa các thẻ và thường biểu diễn một trạng thái của quy trình công việc.
Markdown	Cú pháp đánh dấu văn bản nhẹ được HustFlow hỗ trợ trong phần mô tả thẻ.
Optimistic update	Kỹ thuật cập nhật giao diện trước khi máy chủ xác nhận, sau đó đồng bộ lại hoặc hoàn tác nếu thao tác thất bại.
Private channel	Kênh thời gian thực yêu cầu xác thực và kiểm tra quyền trước khi người dùng được phép đăng ký nhận sự kiện.
Prompt	Chỉ dẫn và dữ liệu ngữ cảnh được gửi tới mô hình trí tuệ nhân tạo để yêu cầu tạo đầu ra theo nhiệm vụ xác định.
Real-time synchro-nization	Cơ chế phát và nhận sự kiện để các phiên đang mở cùng dữ liệu có thể cập nhật thay đổi mà không cần tải lại trang.

Thuật ngữ	Giải thích
Route Handler	Hàm xử lý yêu cầu HTTP tại các tệp route.ts trong Next.js App Router, được dùng để xây dựng các điểm cuối phía máy chủ.
Server Action	Hàm phía máy chủ được giao diện gọi để thực hiện thao tác nghiệp vụ như xác thực, kiểm tra quyền và thay đổi dữ liệu.
Server Component	Thành phần React được kết xuất ở phía máy chủ, phù hợp để lấy dữ liệu ban đầu mà không chuyển logic truy cập dữ liệu xuống trình duyệt.
Smart Capture	Chức năng AI của HustFlow phân tích văn bản tự do để tạo bản nháp thể có cấu trúc trước khi người dùng xác nhận lưu.
Source of truth	Nguồn dữ liệu chuẩn được dùng để đối chiếu và phục hồi trạng thái; trong HustFlow, cơ sở dữ liệu giữ vai trò này.
Webhook	Yêu cầu HTTP do một dịch vụ bên ngoài chủ động gửi tới hệ thống khi sự kiện như thanh toán xảy ra.
WebSocket	Giao thức duy trì kết nối hai chiều giữa máy khách và máy chủ, phù hợp cho việc truyền sự kiện gần thời gian thực.
Workspace	Không gian làm việc của một tổ chức, dùng để quản lý thành viên, bảng công việc, hoạt động và gói dịch vụ trong cùng phạm vi.

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong hoạt động của các nhóm phát triển phần mềm và nhóm dự án học thuật, thông tin công việc thường được phân tán giữa danh sách tác vụ, lịch, tin nhắn và các tài liệu trao đổi. Khi số lượng thành viên và đầu việc tăng, nhóm cần một cách thống nhất để phân công trách nhiệm, theo dõi trạng thái, quản lý thời hạn và quan sát quan hệ phụ thuộc giữa các công việc.

Trong phạm vi đề tài, bài toán được xác định qua năm nhu cầu chính: (i) biểu diễn trạng thái công việc trực quan thay cho danh sách tác vụ tĩnh; (ii) quản lý ngày bắt đầu, hạn hoàn thành và các góc nhìn theo thời gian; (iii) kiểm soát quan hệ phụ thuộc để tránh chu trình giữa các công việc; (iv) đồng bộ thay đổi giữa những thành viên đang thao tác trên cùng bảng; và (v) hỗ trợ tổng hợp thông tin tiến độ từ dữ liệu công việc đã lưu.

Từ các nhu cầu đó, đề tài xây dựng một hệ thống quản lý công việc tích hợp nhiều chế độ xem, cập nhật gần thời gian thực và hỗ trợ tạo bản nháp nội dung bằng trí tuệ nhân tạo. Mục tiêu của hệ thống là cung cấp một quy trình thống nhất để nhóm nhập, cập nhật và theo dõi dữ liệu công việc; mức cải thiện năng suất hoặc thời gian thao tác không được khẳng định khi chưa có nghiên cứu người dùng tương ứng.

1.2 Mục tiêu và phạm vi đề tài

Các công cụ quản lý công việc hiện có như Trello, Jira và Asana đều cung cấp nhiều chế độ tổ chức và theo dõi công việc. Trello hỗ trợ bảng Kanban, tự động hóa và các góc nhìn Calendar, Timeline, Table, Dashboard theo từng gói dịch vụ [1]. Jira cung cấp bảng, danh sách công việc tồn đọng, lịch, dòng thời gian, báo cáo, tự động hóa, phân quyền và lập kế hoạch phụ thuộc liên nhóm [2]. Asana hỗ trợ danh sách, bảng, lịch, Timeline, Gantt, quan hệ phụ thuộc, tự động hóa và các chức năng AI theo từng gói [3].

Vì vậy, HustFlow không được định hướng như một sản phẩm thay thế toàn diện cho các nền tảng trên. Phạm vi nghiên cứu tập trung vào việc xây dựng một nguyên mẫu thống nhất các góc nhìn Kanban, Lịch, Dòng thời gian và Thống kê với quan hệ phụ thuộc có kiểm tra chu trình, vai trò thành viên ở cấp bảng, cập nhật gần thời gian thực và quy trình AI chỉ sinh bản nháp để người dùng kiểm tra trước khi lưu.

Mục tiêu cụ thể của đề tài này là thiết kế và xây dựng ứng dụng HustFlow nhằm giải quyết các hạn chế nêu trên. Các mục tiêu kỹ thuật và nghiệp vụ chính bao gồm:

(i) xây dựng bảng Kanban kéo thả linh hoạt tích hợp phân quyền vai trò chi tiết; (ii) phát triển các góc nhìn tiến độ đa chiều gồm Lịch biểu, biểu đồ dòng thời gian Gantt và chế độ xem chia đôi màn hình; (iii) thiết lập cơ chế kiểm soát và cảnh báo mối quan hệ phụ thuộc giữa các công việc để tránh vòng lặp; (iv) tích hợp trợ lý trí tuệ nhân tạo để tự động hóa việc khởi tạo mô tả, gợi ý danh sách công việc con và tổng hợp báo cáo tiến độ; và (v) cài đặt hệ thống giới hạn tài nguyên và thanh toán để quản lý việc nâng cấp gói cước.

Phạm vi nghiên cứu và triển khai của đề tài tập trung vào việc phát triển một ứng dụng web phục vụ nhu cầu cộng tác của các nhóm phát triển phần mềm, các nhóm dự án học thuật và các nhà quản lý dự án quy mô vừa và nhỏ. Hệ thống hỗ trợ mô hình đa tổ chức làm việc, trong đó mỗi tổ chức có không gian làm việc riêng biệt, được bảo vệ theo quyền truy cập và đồng bộ hóa thời gian thực qua kết nối WebSocket. Các chi tiết thiết kế giao diện, cấu trúc cơ sở dữ liệu và kịch bản thực nghiệm hệ thống sẽ được phân tích rõ hơn ở các chương sau.

1.3 Định hướng giải pháp

Để hiện thực hóa các mục tiêu đã đề ra, đề tài định hướng xây dựng hệ thống HustFlow dựa trên kiến trúc ứng dụng web hiện đại. Công nghệ cốt lõi được lựa chọn bao gồm: (i) Next.js App Router kết hợp thư viện React để xây dựng giao diện và các luồng xử lý full-stack; (ii) hệ quản trị cơ sở dữ liệu MySQL cùng Prisma ORM để hỗ trợ toàn vẹn dữ liệu; (iii) dịch vụ Pusher làm nền tảng truyền tải dữ liệu thời gian thực qua giao thức WebSocket; (iv) Clerk quản lý quá trình xác thực người dùng và phân cấp tổ chức làm việc; và (v) OpenAI API cung cấp khả năng xử lý ngôn ngữ tự nhiên cho trợ lý AI. Sự kết hợp này hướng tới khả năng bảo trì, tính nhất quán dữ liệu và phạm vi chức năng đã xác định.

Giải pháp kỹ thuật của HustFlow tổ chức thông tin công việc theo cấu trúc phân cấp chặt chẽ từ không gian tổ chức đến các bảng công việc, các cột danh sách và các thẻ tác vụ chi tiết. Ở khía cạnh quản lý trạng thái ứng dụng, hệ thống sử dụng TanStack Query (React Query) [4] để quản lý và lưu bộ nhớ đệm dữ liệu phía máy khách, kết hợp với các sự kiện Pusher được phát đi mỗi khi có thay đổi dữ liệu trên máy chủ. Khi một thành viên thực hiện thao tác kéo thả thẻ, chỉnh sửa thuộc tính hoặc gửi bình luận, máy chủ sẽ xử lý thay đổi và gửi tín hiệu đồng bộ. Các máy khách khác đang tham gia cùng bảng sẽ nhận được tín hiệu này để tự động cập nhật bộ nhớ đệm mà không cần tải lại toàn bộ trang web.

Đồ án đã xây dựng được ứng dụng web HustFlow với các nhóm chức năng quản lý công việc đa chiều, cộng tác thời gian thực, kiểm soát phụ thuộc, hỗ trợ AI và nâng cấp gói dịch vụ. Các giải pháp được đánh giá trong phạm vi thiết kế, mã

nguồn và những kiểm tra kỹ thuật đã thực hiện; đề án chưa sử dụng các kết quả này để suy diễn thành số liệu hiệu năng, khả năng chịu tải hoặc mức cải thiện năng suất khi chưa có thực nghiệm tương ứng. Phân tích chi tiết về các đóng góp kỹ thuật được trình bày tại Chương 5.

1.4 Bố cục đề án

Báo cáo đề án tốt nghiệp này ngoài chương mở đầu được cấu trúc thành năm chương tiếp theo như dưới đây.

Chương 2 tập trung vào phần Khảo sát hiện trạng và Phân tích yêu cầu hệ thống. Chương này đánh giá chi tiết các quy trình quản lý công việc thực tế và phân tích các ưu nhược điểm của các giải pháp hiện có, từ đó xây dựng các yêu cầu chức năng và phi chức năng cho hệ thống HustFlow. Các yêu cầu chức năng được mô hình hóa chi tiết thông qua biểu đồ ca sử dụng tổng quát và các biểu đồ phân rã ca sử dụng, kèm theo đặc tả luồng sự kiện cho các chức năng cốt lõi của ứng dụng.

Chương 3 trình bày về Nền tảng lý thuyết và Công nghệ sử dụng. Chương này giới thiệu chi tiết cơ sở khoa học của các công nghệ được áp dụng như mô hình Next.js App Router, thư viện React, hệ quản trị cơ sở dữ liệu MySQL, Prisma ORM, cơ chế Pusher và API của OpenAI. Đồng thời, nội dung chương cũng đưa ra các lập luận so sánh kỹ thuật để minh chứng cho sự phù hợp của các công nghệ đã chọn đối với yêu cầu bài toán.

Chương 4 mô tả chi tiết quá trình Phân tích thiết kế, Triển khai và Đánh giá hệ thống. Nội dung chương bao gồm thiết kế kiến trúc hệ thống tổng thể, thiết kế biểu đồ gói, biểu đồ tuần tự cho các luồng nghiệp vụ chính, sơ đồ quan hệ thực thể cơ sở dữ liệu và thống kê mã nguồn dự án. Chương này cũng trình bày thiết kế và kết quả kiểm thử chức năng, kết quả kiểm tra kỹ thuật cùng môi trường và mô hình chạy thử nghiệm cục bộ.

Chương 5 giới thiệu chi tiết ba giải pháp và đóng góp nổi bật của đề án. Nội dung gồm giải pháp đồng bộ dữ liệu thời gian thực có kiểm soát tại phần 5.1, giải pháp phát hiện chu trình trong quan hệ phụ thuộc tại phần 5.2, và giải pháp tích hợp trợ lý trí tuệ nhân tạo theo nguyên tắc sinh bản nháp có ràng buộc tại phần 5.3.

Chương 6 đưa ra Kết luận và Hướng phát triển của đề tài. Chương này tổng kết lại những kết quả đạt được, tự đánh giá các ưu điểm và hạn chế còn tồn tại của hệ thống HustFlow. Từ đó, đề tài đề xuất định hướng nghiên cứu tiếp theo và các giải pháp cải tiến công nghệ nhằm nâng cao hiệu suất và trải nghiệm người dùng trong tương lai.

Như vậy, chương này đã phác thảo những nét khái quát nhất về đề tài nghiên cứu

và xây dựng hệ thống HustFlow. Thông qua việc phân tích bối cảnh thực tế và các khó khăn hiện hữu trong hoạt động phối hợp nhóm, đề tài đã xác định mục tiêu xây dựng một ứng dụng trực quan, tích hợp các chế độ xem đa chiều và hỗ trợ từ trí tuệ nhân tạo. Định hướng giải pháp kỹ thuật được lựa chọn để đáp ứng các nhóm yêu cầu chính của đề tài. Những nội dung tổng quan này sẽ làm cơ sở để tiến hành phân tích chi tiết các yêu cầu nghiệp vụ và khảo sát hiện trạng thực tế trong Chương 2.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã trình bày bối cảnh, mục tiêu và định hướng giải pháp của đề tài xây dựng hệ thống HustFlow. Trên cơ sở đó, chương này tập trung khảo sát các công cụ quản lý công việc phổ biến, chỉ ra các khoảng trống cần giải quyết và phân tích yêu cầu nghiệp vụ cho hệ thống. Nội dung chương gồm bốn phần chính: khảo sát hiện trạng tại phần 2.1, tổng quan chức năng và quy trình nghiệp vụ tại phần 2.2, đặc tả các use case cốt lõi tại phần 2.3, và yêu cầu phi chức năng tại phần 2.4.

2.1 Khảo sát hiện trạng

Các công cụ quản lý công việc trực tuyến như Trello, Jira và Asana cung cấp nhiều cách tiếp cận để tổ chức nhiệm vụ, theo dõi tiến độ và phối hợp giữa các thành viên. Phần khảo sát này đối chiếu thông tin chức năng được các nhà cung cấp công bố chính thức, qua đó xác định phạm vi chức năng của HustFlow và tránh xem nguyên mẫu như một sản phẩm thay thế toàn diện.

Trello tổ chức công việc theo bảng, danh sách và thẻ; người dùng có thể di chuyển thẻ giữa các danh sách để phản ánh trạng thái thực hiện. Trang công bố gói dịch vụ cho thấy Trello có tự động hóa tích hợp, ứng dụng di động và các chế độ xem Calendar, Timeline, Table, Dashboard, Map; một số chế độ xem và chức năng AI thuộc các gói cao hơn. Trong phạm vi so sánh của đề án, HustFlow học theo mô hình bảng nhưng tập trung cài đặt quan hệ phụ thuộc giữa các thẻ ngay trong mô hình dữ liệu và kiểm tra chu trình tại máy chủ [1].

Jira cung cấp backlog, bảng, danh sách, lịch, dòng thời gian, báo cáo và bảng điều khiển; đồng thời hỗ trợ tự động hóa, vai trò, phân quyền và các chức năng AI theo gói dịch vụ. Gói cao hơn bổ sung lập kế hoạch và quản lý phụ thuộc liên nhóm cùng các cam kết hỗ trợ và mức dịch vụ. HustFlow không hướng tới mức cấu hình và quy mô vận hành đó; nguyên mẫu chỉ cài đặt các luồng cần thiết cho phạm vi tổ chức, bảng và thẻ đã xác định trong đề án [2].

Asana hỗ trợ danh sách, bảng kiểu Kanban, lịch, Timeline và Gantt; các chế độ xem theo thời gian có thể biểu diễn nhiệm vụ, mốc và quan hệ phụ thuộc. Nền tảng cũng công bố chức năng báo cáo, tự động hóa và AI theo từng gói. HustFlow sử dụng các nhóm chức năng tương tự ở quy mô nguyên mẫu, nhưng phần được nghiên cứu tập trung vào cách kết hợp dữ liệu công việc với kiểm tra phụ thuộc, phân quyền phía máy chủ và quy trình AI sinh bản nháp có bước xác nhận của người dùng [3].

Từ việc đối chiếu tài liệu sản phẩm, HustFlow được giới hạn cho bài toán quản

lý công việc của nhóm vừa và nhỏ trong phạm vi đồ án. Hệ thống ưu tiên bảng Kanban, cập nhật gần thời gian thực, các chế độ xem Lịch, Dòng thời gian và Thống kê, kiểm tra chu trình phụ thuộc, trợ lý AI sinh bản nháp và mô hình gói dịch vụ đơn giản. Phạm vi này là cơ sở xác định các nhóm chức năng và xây dựng use case ở các phần tiếp theo, không phải kết luận rằng HustFlow có độ bao phủ hoặc chất lượng vận hành tương đương các sản phẩm thương mại.

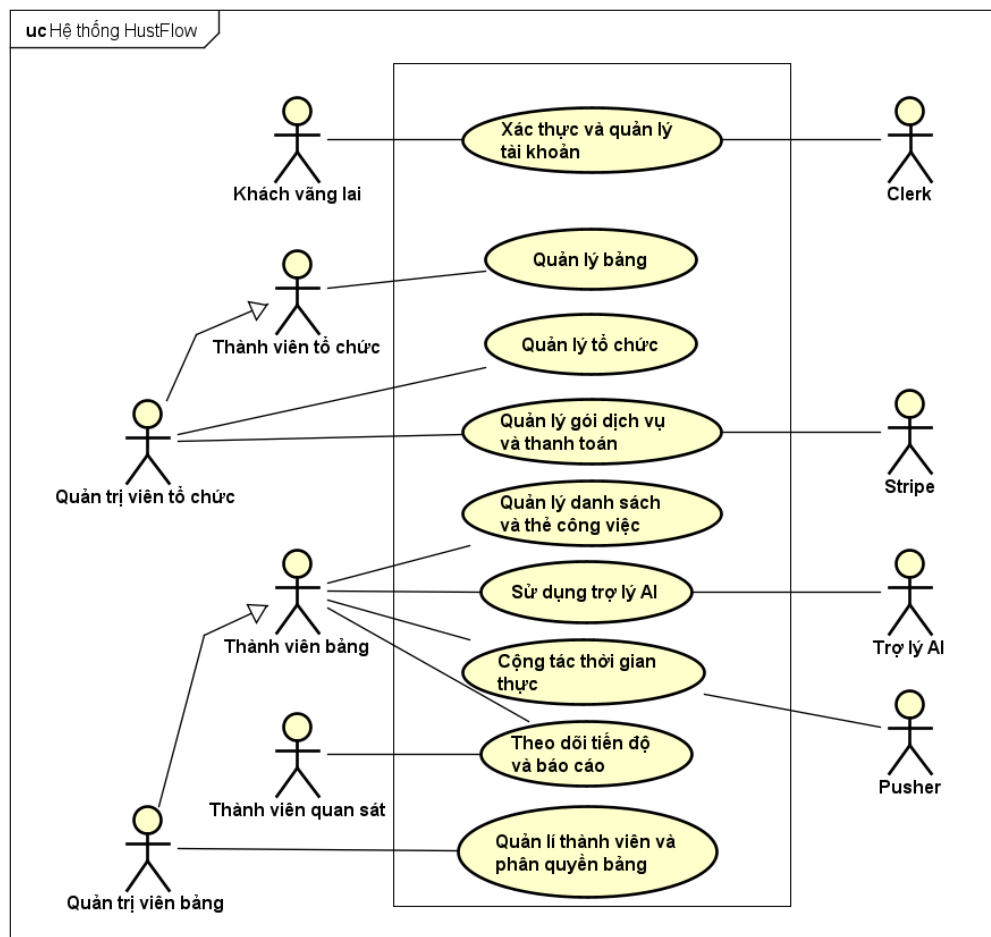
2.2 Tổng quan chức năng

HustFlow tổ chức dữ liệu theo mô hình nhiều tổ chức làm việc. Trong mỗi tổ chức, người dùng có thể tạo các bảng công việc, mời thành viên vào bảng, phân quyền truy cập và quản lý công việc thông qua các thẻ tác vụ. Hệ thống không chỉ hỗ trợ thao tác bảng Kanban cơ bản mà còn cung cấp các chế độ xem lịch, tiến độ, thống kê, cùng các công cụ cộng tác như bình luận, thông báo, tệp đính kèm và trợ lý trí tuệ nhân tạo.

2.2.1 Biểu đồ use case tổng quát

Để mô hình hóa toàn diện các yêu cầu chức năng của hệ thống HustFlow, biểu đồ use case tổng quát được xây dựng với sự phân cấp rõ ràng giữa các vai trò người dùng và phân định phạm vi hoạt động của họ theo hai cấp độ: cấp tổ chức và cấp bảng.

Biểu đồ use case tổng quát của HustFlow được trình bày trong Hình 2.1.



Hình 2.1: Biểu đồ use case tổng quát của hệ thống HustFlow

Trước khi tham gia các hoạt động theo phạm vi tổ chức hoặc bảng, khách vắng lai thực hiện use case Xác thực và quản lý tài khoản. Use case này bao gồm đăng ký, đăng nhập, đăng xuất và quản lý phiên làm việc; Clerk là dịch vụ bên ngoài hỗ trợ thực hiện các chức năng xác thực. Sau khi xác thực thành công và tham gia một tổ chức, người dùng được gán các vai trò tương ứng để sử dụng các chức năng trong hệ thống.

Hình 2.1 cho thấy hệ thống được thiết kế với các tác nhân chính tương ứng với hai phạm vi nghiệp vụ của HustFlow. Ở cấp tổ chức, thành viên tổ chức là người dùng đã thuộc một tổ chức làm việc và có quyền thực hiện các thao tác cơ bản như tạo bảng công việc, truy cập các bảng được phép tham gia và quản lý công việc trong phạm vi quyền hạn của mình. Quản trị viên tổ chức là vai trò có quyền hạn cao hơn, kế thừa các chức năng của thành viên tổ chức và chịu trách nhiệm quản lý các tài nguyên chung ở cấp tổ chức. Vai trò này có thể cập nhật thông tin tổ chức, kiểm soát gói dịch vụ, theo dõi giới hạn sử dụng và thực hiện thanh toán thông qua Stripe khi tổ chức cần nâng cấp lên gói trả phí.

Ở cấp bảng, thành viên bảng là tác nhân trực tiếp tham gia vào quá trình vận

hành công việc hằng ngày. Người dùng thuộc vai trò này có thể tạo và cập nhật thẻ công việc, kéo thả thẻ giữa các danh sách, trao đổi trong thẻ, đính kèm tài liệu, theo dõi tiến độ qua nhiều chế độ xem và sử dụng các chức năng hỗ trợ bởi trợ lý trí tuệ nhân tạo. Quản trị viên bảng kế thừa quyền của thành viên bảng, đồng thời được bổ sung quyền quản lý thành viên và phân quyền truy cập trong phạm vi bảng. Bên cạnh đó, thành viên quan sát chỉ được theo dõi tiến độ và xem các báo cáo liên quan, không có quyền thay đổi dữ liệu hay thực hiện các thao tác chỉnh sửa. Cách phân chia này giúp hệ thống kiểm soát rõ ràng trách nhiệm của từng người dùng, đồng thời hạn chế rủi ro thay đổi dữ liệu ngoài phạm vi được phép.

Ngoài các tác nhân là người dùng trực tiếp, HustFlow còn tương tác với một số tác nhân hỗ trợ bên ngoài. Clerk cung cấp dịch vụ xác thực, quản lý phiên làm việc và thông tin định danh người dùng. Dịch vụ này là nền tảng để hệ thống xác định người dùng trước khi kiểm tra quyền truy cập theo tổ chức và bảng. Pusher cung cấp dịch vụ truyền sự kiện thời gian thực qua các kênh riêng tư. Dịch vụ này được sử dụng để đồng bộ thay đổi giao diện và chuyển thông báo mới tới người dùng đang mở ứng dụng. Trợ lý trí tuệ nhân tạo đóng vai trò xử lý ngôn ngữ tự nhiên và hỗ trợ tự động hóa một số thao tác liên quan đến thẻ công việc, chẳng hạn như trích xuất công việc từ nội dung thô, đề xuất danh sách kiểm tra hoặc tóm tắt nội dung. Stripe được sử dụng như cổng thanh toán trực tuyến, chịu trách nhiệm xử lý các giao dịch nâng cấp gói dịch vụ của tổ chức. Các tác nhân này không thay thế vai trò của người dùng trong hệ thống, mà cung cấp các dịch vụ hỗ trợ để hoàn thiện luồng nghiệp vụ chính.

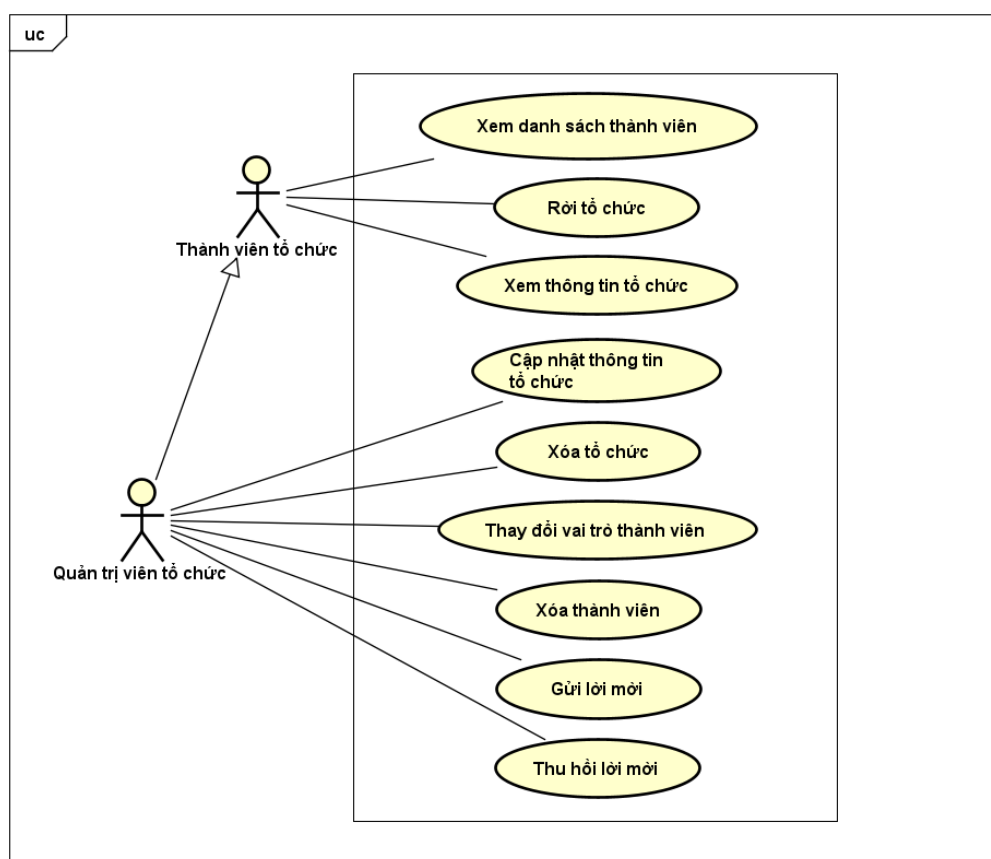
Từ các tác nhân trên, các use case cốt lõi của HustFlow tập trung vào những nhóm chức năng phục vụ trực tiếp quá trình quản lý công việc theo mô hình bảng. Trước hết, Quản lý tổ chức cho phép quản trị viên tổ chức thiết lập và cập nhật thông tin tổ chức, làm nền tảng cho việc quản lý tài nguyên dùng chung. Quản lý bảng hỗ trợ người dùng tạo và quản lý các bảng Kanban trong không gian làm việc, bao gồm việc cấu hình các chế độ xem và truy cập các bảng mà người dùng có quyền tham gia. Trên từng bảng cụ thể, Quản lý thành viên và phân quyền bảng cho phép quản trị viên bảng thêm hoặc loại bỏ thành viên, đồng thời gán các vai trò quản trị viên bảng, thành viên bảng hoặc thành viên quan sát để kiểm soát mức độ truy cập.

Các nghiệp vụ xử lý công việc được thể hiện chủ yếu qua Quản lý danh sách và thẻ công việc, trong đó thành viên bảng có thể tạo danh sách, tạo thẻ, cập nhật tiêu đề, mô tả, thời gian, trạng thái, tệp đính kèm và di chuyển thẻ giữa các danh sách để phản ánh tiến độ thực hiện. Song song với đó, Cộng tác thời gian thực hỗ trợ

quá trình phối hợp thông qua bình luận, biểu cảm, thông báo và các tương tác trực tiếp trong thẻ. Theo dõi tiến độ và báo cáo cung cấp khả năng theo dõi dữ liệu bảng qua các góc nhìn bảng Kanban, lịch, biểu đồ Gantt và thống kê, giúp cả thành viên bảng và thành viên quan sát nắm được tình trạng thực hiện công việc. Ngoài ra, Sử dụng trợ lý AI cho phép người dùng gọi các chức năng thông minh như tạo thẻ tự động từ nội dung thô, đề xuất danh sách kiểm tra và tóm tắt nội dung; trong khi Quản lý gói dịch vụ và thanh toán phục vụ việc kiểm tra hạn mức, quản lý gói dịch vụ và nâng cấp gói trả phí thông qua Stripe. Tổng thể các use case này tạo thành phạm vi chức năng chính của HustFlow và là cơ sở cho phân phân rã use case ở các mục tiếp theo.

2.2.2 Biểu đồ use case phân rã

a, Biểu đồ use case phân rã Quản lý tổ chức



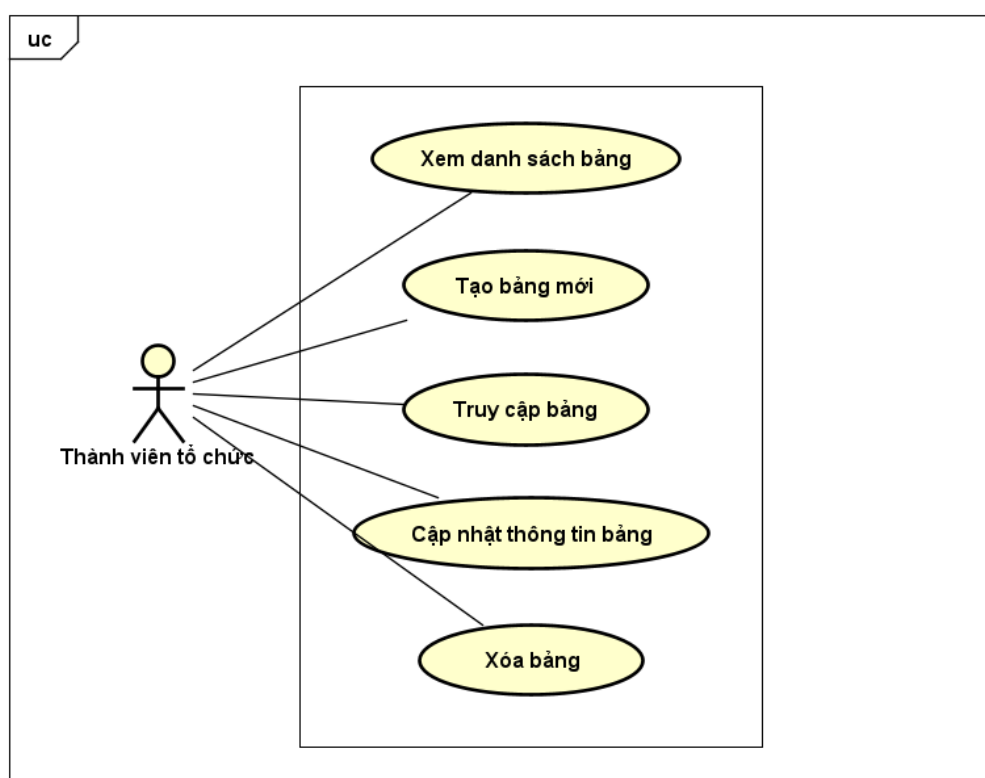
Hình 2.2: Biểu đồ use case phân rã Quản lý tổ chức

Hình 2.2 mô tả chi tiết biểu đồ use case phân rã Quản lý tổ chức, được tổ chức thành ba nhóm nghiệp vụ chính: quản lý thông tin tổ chức, quản lý thành viên và quản lý lời mời. Đối với quản lý thông tin tổ chức, thành viên tổ chức có thể xem thông tin tổ chức và thực hiện rời khỏi tổ chức, trong khi quản trị viên tổ chức có quyền cao hơn để cập nhật thông tin tổ chức (như tên và logo/ảnh đại diện) cũng như thực hiện xóa tổ chức. Đối với quản lý thành viên, thành viên tổ chức được

quyền xem danh sách thành viên, còn quản trị viên tổ chức có quyền thay đổi vai trò thành viên (giữa quản trị viên và thành viên) hoặc xóa thành viên khỏi tổ chức. Đối với quản lý lời mời, các chức năng gửi lời mời tham gia qua email (có gán kèm vai trò ban đầu) và thu hồi lời mời đang chờ xử lý chỉ dành riêng cho quản trị viên tổ chức.

Cách tách biệt chi tiết này giúp làm rõ ranh giới quyền hạn giữa các vai trò ở cấp tổ chức. Việc phân cấp bảo vệ tính toàn vẹn dữ liệu dùng chung, ngăn chặn các thao tác thay đổi cấu hình trái phép từ người dùng thông thường, đồng thời giúp định hình cấu trúc giao diện cài đặt tổ chức một cách trực quan và mạch lạc.

b, Biểu đồ use case phân rã Quản lý bảng



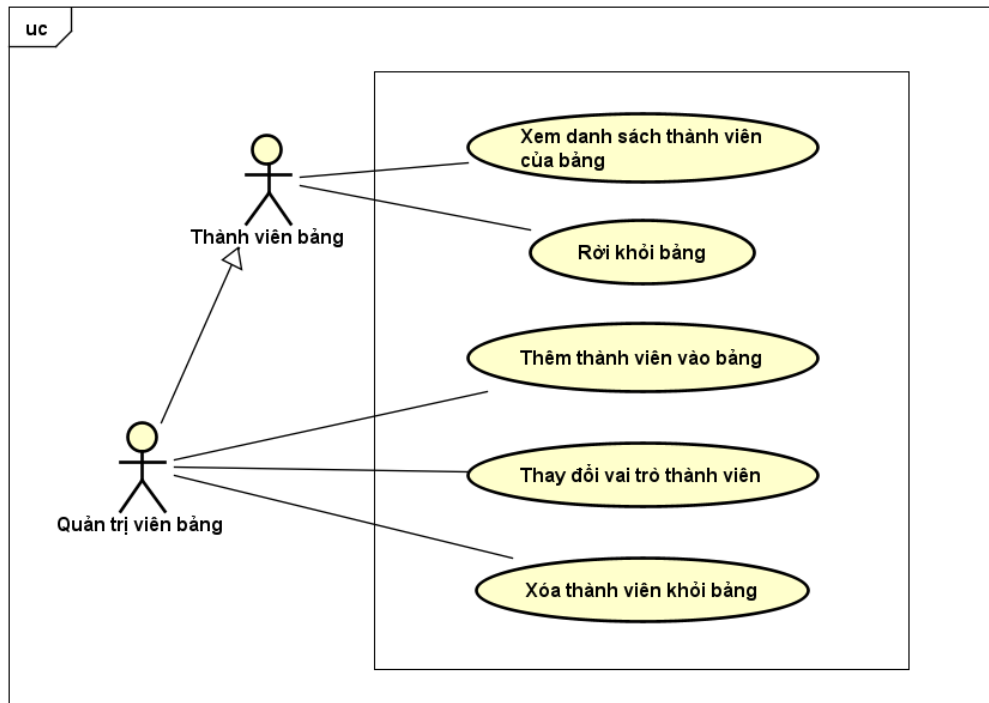
Hình 2.3: Biểu đồ use case phân rã Quản lý bảng

Hình 2.3 cho thấy use case Quản lý bảng do thành viên tổ chức kích hoạt, phản ánh quá trình tạo và truy cập các bảng công việc trong một tổ chức. Người dùng có thể tạo bảng mới khi tổ chức còn trong hạn mức cho phép, xem danh sách các bảng mà mình tham gia, mở bảng đang làm việc và cập nhật một số thông tin cơ bản của bảng nếu có quyền phù hợp. Đây là cầu nối giữa phạm vi tổ chức và phạm vi xử lý công việc hằng ngày.

Trong phần phân rã, Quản lý bảng cần được tách khỏi Quản lý thành viên và phân quyền bảng. Quản lý bảng tập trung vào vòng đời của bảng, còn việc thêm thành viên hoặc gán vai trò cho từng người dùng thuộc về use case riêng của quản

trị viên bảng. Sự phân tách này phù hợp với biểu đồ tổng quát, trong đó tác nhân thành viên tổ chức liên kết với Quản lý bảng, còn quản trị viên bảng liên kết với Quản lý thành viên và phân quyền bảng.

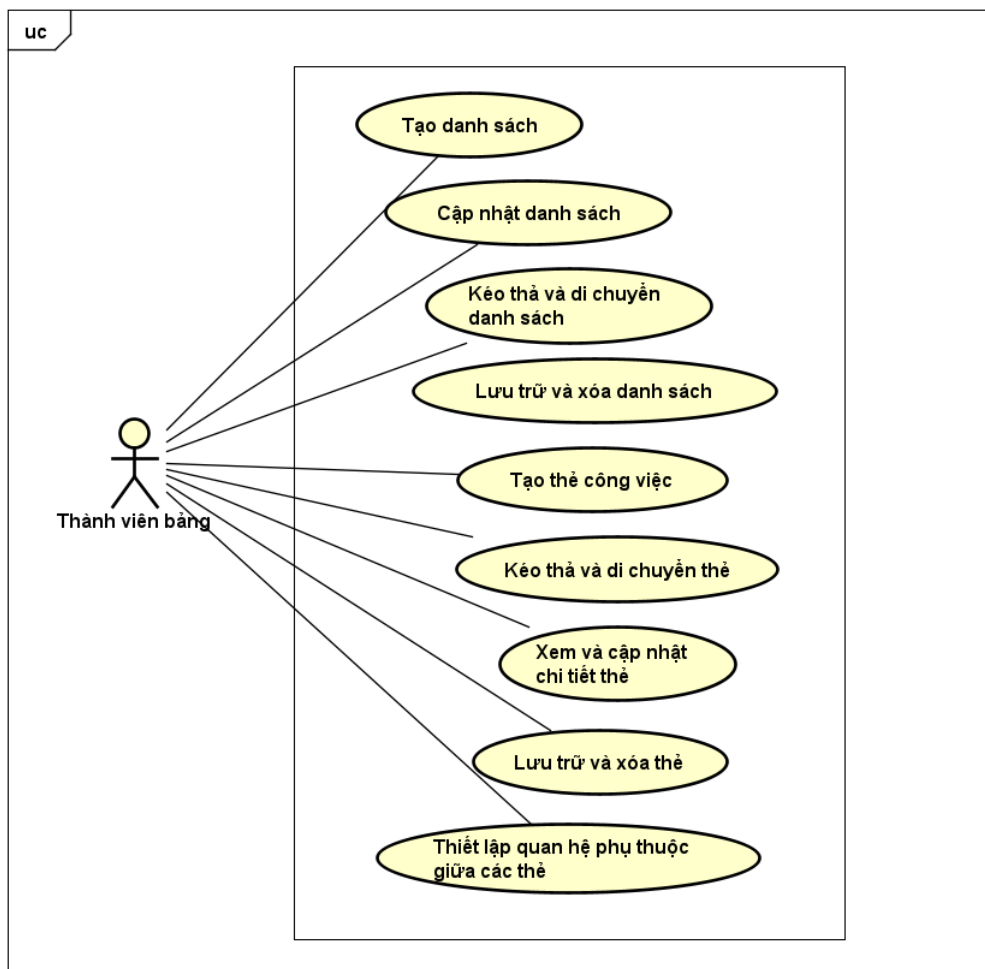
c, Biểu đồ use case phân rã Quản lý thành viên và phân quyền bảng



Hình 2.4: Biểu đồ use case phân rã Quản lý thành viên và phân quyền bảng

Hình 2.4 cho thấy use case Quản lý thành viên và phân quyền bảng trong phạm vi một bảng công việc. Nhóm chức năng này bao gồm xem danh sách thành viên hiện có, rời khỏi bảng, thêm thành viên từ tổ chức vào bảng, thay đổi vai trò trong bảng và thu hồi quyền truy cập khi cần thiết. Các vai trò được sử dụng trong phạm vi bảng gồm quản trị viên bảng, thành viên bảng và thành viên quan sát.

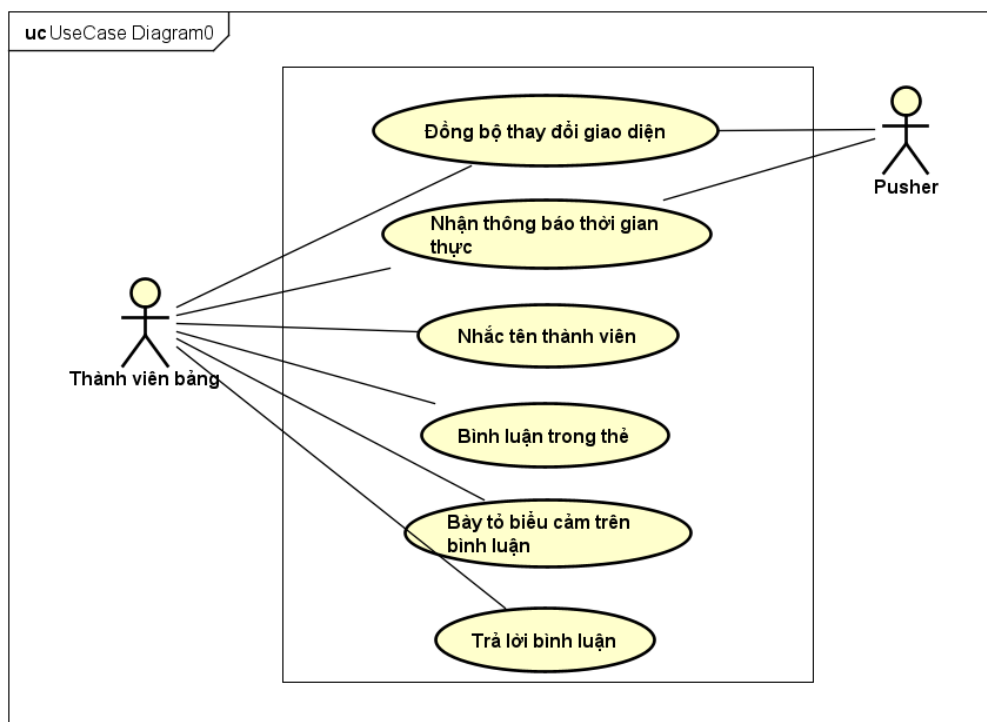
Việc phân rã use case này cần nhấn mạnh rằng quyền quản trị bảng không đồng nhất với quyền quản trị tổ chức. Một người dùng có thể là thành viên tổ chức nhưng chỉ là thành viên quan sát ở một bảng cụ thể, hoặc có thể là quản trị viên của một bảng nhưng không có quyền thay đổi thiết lập chung của tổ chức. Khi một thành viên rời khỏi bảng, hệ thống xóa tư cách thành viên trong phạm vi bảng, gỡ các phân công thẻ và mục checklist liên quan khỏi giao diện, đồng thời ghi nhận một nhật ký hoạt động cấp bảng. Trường hợp người rời là quản trị viên cuối cùng của bảng, hệ thống không cho phép rời và yêu cầu chuyển quyền quản trị cho người khác trước để bảo đảm bảng luôn còn người chịu trách nhiệm quản trị.

d, Biểu đồ use case phân rã Quản lý danh sách và thẻ công việc**Hình 2.5:** Biểu đồ use case phân rã Quản lý danh sách và thẻ công việc

Hình 2.5 cho thấy use case Quản lý danh sách và thẻ công việc do thành viên bảng kích hoạt và là nhóm chức năng cốt lõi trong HustFlow. Người dùng có thể tạo danh sách, đổi tên danh sách, sắp xếp danh sách, tạo thẻ công việc, cập nhật nội dung thẻ và di chuyển thẻ giữa các danh sách để phản ánh trạng thái thực hiện. Thẻ công việc lưu trữ các thông tin chi tiết như tiêu đề, mô tả, người phụ trách, nhãn, ngày bắt đầu, hạn chót, nhắc việc, danh sách kiểm tra, tệp đính kèm và trạng thái hoàn thành.

Khi phân rã sâu hơn, nhóm chức năng này cũng bao gồm thao tác thiết lập quan hệ phụ thuộc giữa các thẻ. Trước khi lưu một quan hệ phụ thuộc mới, hệ thống cần kiểm tra để tránh vòng lặp phụ thuộc, bởi vòng lặp có thể làm sai lệch logic tiến độ và khiến các thẻ chặn lẫn nhau. Nhóm use case này vì vậy không chỉ xử lý thao tác nhập liệu, mà còn đảm bảo tính nhất quán nghiệp vụ của bảng công việc.

e, Biểu đồ use case phân rã Cộng tác thời gian thực

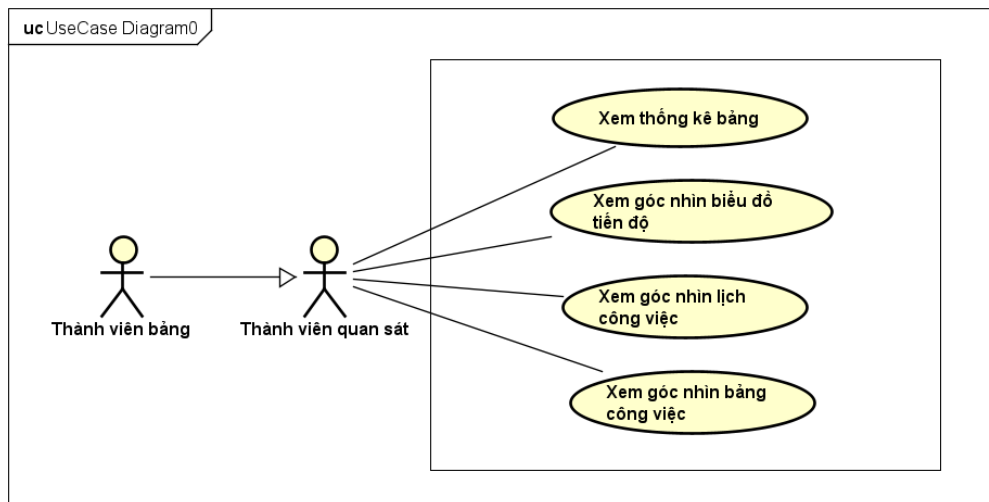


Hình 2.6: Biểu đồ use case phân rã Cộng tác thời gian thực

Hình 2.6 cho thấy use case Cộng tác thời gian thực do thành viên bảng kích hoạt, phục vụ quá trình phối hợp giữa nhiều người dùng trên cùng một bảng hoặc cùng một thẻ. Các chức năng phân rã gồm bình luận trong thẻ, trả lời bình luận, bày tỏ biểu cảm, nhắc đến thành viên khác, nhận thông báo và đồng bộ thay đổi tới các giao diện đang mở. Pusher là tác nhân hỗ trợ bên ngoài cho hai chức năng đồng bộ thay đổi giao diện và nhận thông báo thời gian thực. Sau khi nghiệp vụ như bình luận, trả lời hoặc nhắc tên thành viên được xử lý và dữ liệu được lưu, hệ thống phát sự kiện qua Pusher để các giao diện liên quan cập nhật mà không cần tải lại trang. Nhờ đó, người dùng có thể theo dõi diễn biến công việc mà không cần tải lại trang hoặc trao đổi qua công cụ bên ngoài.

Nhóm chức năng này gắn chặt với Quản lý danh sách và thẻ công việc nhưng vẫn cần được tách thành use case riêng vì mục tiêu nghiệp vụ khác nhau. Quản lý danh sách và thẻ tập trung vào dữ liệu công việc, còn Cộng tác thời gian thực tập trung vào tương tác giữa các thành viên và khả năng đồng bộ trạng thái. Việc tách riêng giúp phần thiết kế hệ thống sau này làm rõ các cơ chế thông báo, nhật ký hoạt động và cập nhật thời gian thực.

f, Biểu đồ use case phân rã Theo dõi tiến độ và báo cáo

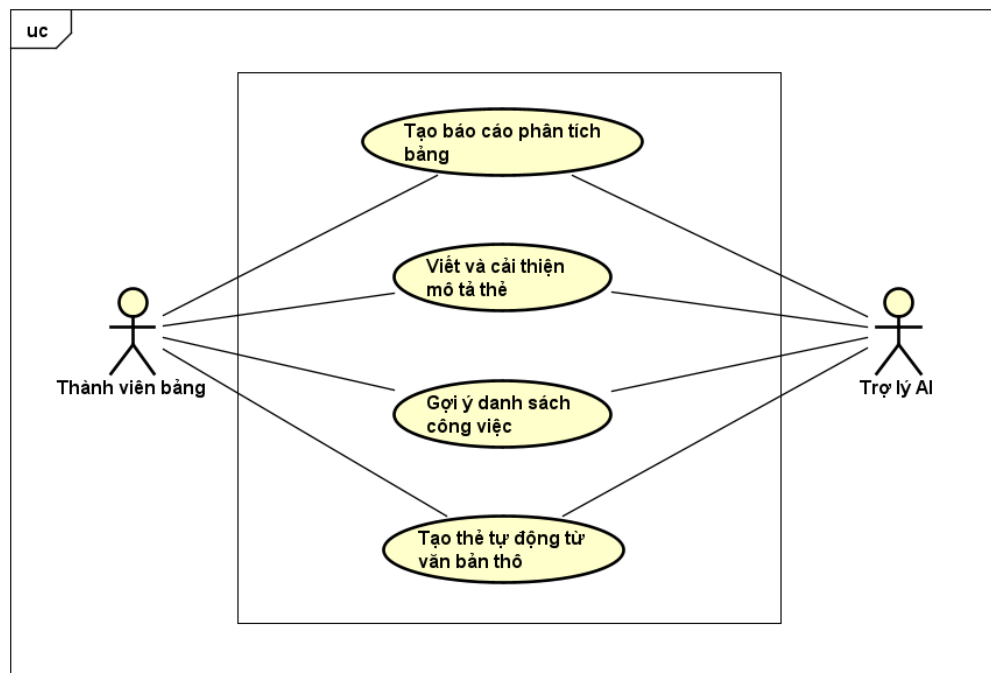


Hình 2.7: Biểu đồ use case phân rã Theo dõi tiến độ và báo cáo

Hình 2.7 cho thấy use case Theo dõi tiến độ và báo cáo được kích hoạt bởi thành viên bảng và thành viên quan sát. Nhóm chức năng này cho phép người dùng quan sát dữ liệu bảng qua nhiều góc nhìn như bảng Kanban, lịch, biểu đồ Gantt và thống kê. Người dùng có thể theo dõi số lượng thẻ theo trạng thái, các thẻ đến hạn, tiến độ hoàn thành danh sách kiểm tra, phân bổ công việc theo thành viên và các thông tin tổng hợp phục vụ đánh giá tình trạng bảng.

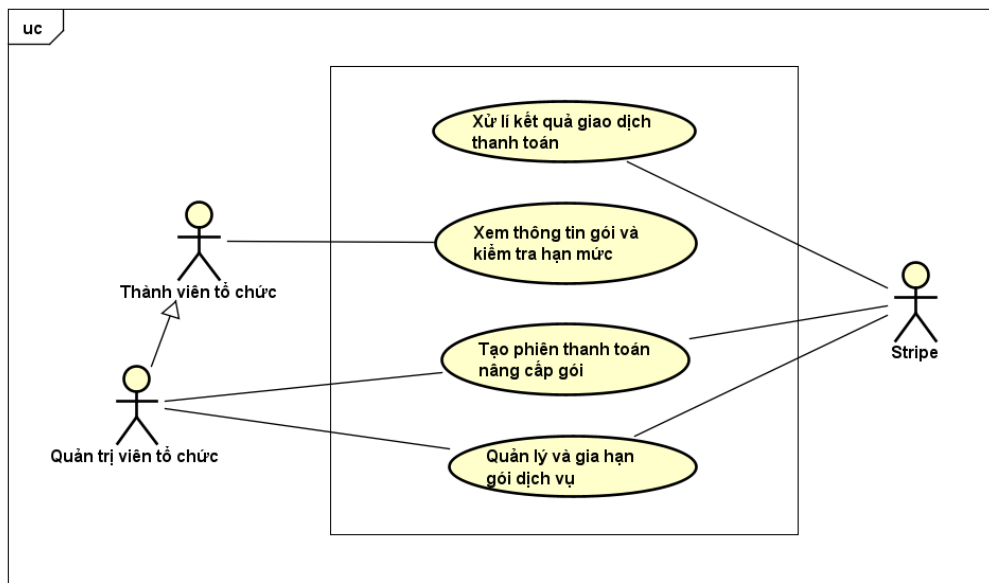
Điểm cần lưu ý trong phân rã là thành viên quan sát chỉ có quyền khai thác thông tin, không có quyền chỉnh sửa dữ liệu công việc. Trong khi đó, thành viên bảng vừa có thể theo dõi tiến độ vừa có thể quay lại thao tác cập nhật thẻ nếu phát hiện vấn đề trong quá trình làm việc. Nhóm use case này vì vậy đóng vai trò hỗ trợ ra quyết định và giúp các thành viên nắm được bức tranh tổng quan của bảng.

g, Biểu đồ use case phân rã Sử dụng trợ lý AI

**Hình 2.8:** Biểu đồ use case phân rã Sử dụng trợ lý AI

Hình 2.8 cho thấy use case Sử dụng trợ lý AI do thành viên bảng kích hoạt và có liên kết với tác nhân hỗ trợ bên ngoài là Trợ lý AI. Các chức năng phân rã gồm tạo thẻ tự động từ nội dung thô, gợi ý danh sách kiểm tra, hỗ trợ viết hoặc cải thiện mô tả thẻ và tạo báo cáo bảng dựa trên dữ liệu thống kê. Người dùng cung cấp dữ liệu đầu vào hoặc chọn phạm vi dữ liệu cần phân tích, sau đó hệ thống gửi ngữ cảnh phù hợp tới trợ lý.

Kết quả do trợ lý tạo ra được hiển thị dưới dạng bản nháp hoặc gợi ý để người dùng rà soát trước khi lưu. Cách thiết kế này giúp tận dụng khả năng hỗ trợ của trí tuệ nhân tạo nhưng vẫn giữ quyền quyết định cuối cùng cho người dùng. Use case này cũng cần được kiểm soát bởi quyền truy cập bảng, nhằm bảo đảm trợ lý chỉ xử lý dữ liệu mà người dùng hiện tại được phép xem hoặc chỉnh sửa.

h, Biểu đồ use case phân rã Quản lý gói dịch vụ và thanh toán**Hình 2.9:** Biểu đồ use case phân rã Quản lý gói dịch vụ và thanh toán

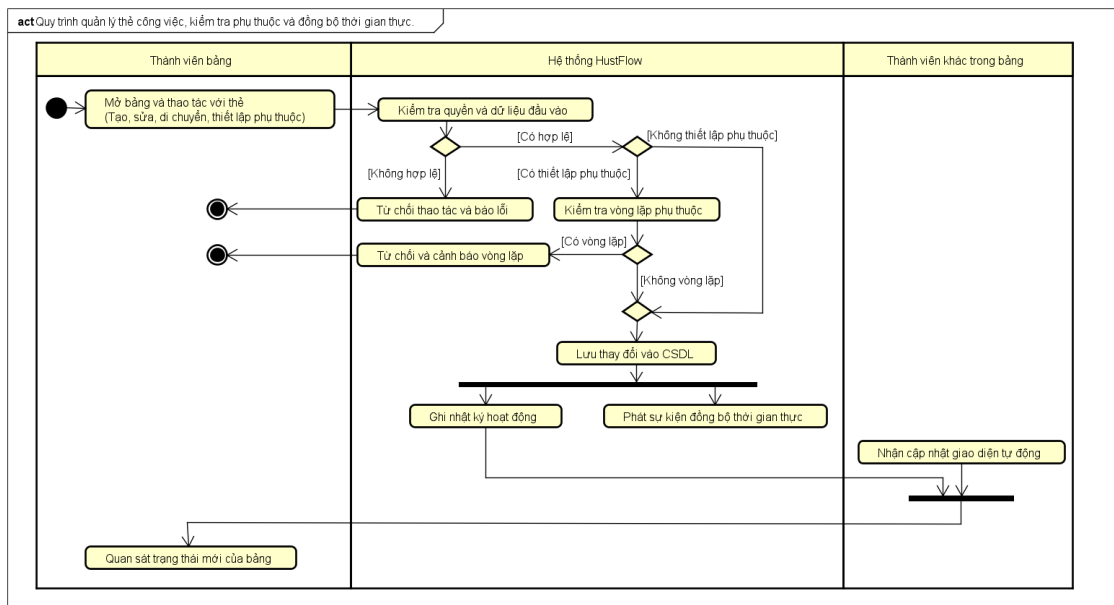
Hình 2.9 cho thấy use case Quản lý gói dịch vụ và thanh toán do quản trị viên tổ chức kích hoạt và có liên kết với tác nhân hỗ trợ bên ngoài là Stripe. Nhóm chức năng này bao gồm xem trạng thái gói dịch vụ, kiểm tra hạn mức tạo bảng, tạo phiên thanh toán, chuyển người dùng sang Stripe để hoàn tất giao dịch và xử lý kết quả thanh toán trả về. Thành viên tổ chức không có quyền thanh toán chỉ được xem trạng thái gói hoặc nhận thông báo khi tổ chức đạt giới hạn tài nguyên.

Khi phân rã, use case này cần thể hiện rõ bước xác thực quyền quản trị viên tổ chức trước khi tạo phiên thanh toán. Sau khi Stripe gửi sự kiện xác nhận giao dịch, HustFlow kiểm tra tính hợp lệ của sự kiện rồi mới cập nhật trạng thái thuê bao và mở rộng hạn mức tạo bảng. Việc tách riêng use case thanh toán khỏi các nghiệp vụ quản lý công việc giúp hệ thống giảm phụ thuộc giữa logic tài chính và logic vận hành bảng.

2.2.3 Quy trình nghiệp vụ

Trong các nghiệp vụ của HustFlow, quy trình nổi bật và quan trọng nhất là quy trình quản lý thẻ công việc kết hợp kiểm tra phụ thuộc và đồng bộ thời gian thực. Đây không phải là luồng sự kiện của một use case đơn lẻ, mà là quy trình phối hợp nhiều nhóm chức năng như quản lý danh sách và thẻ công việc, phân quyền bảng, kiểm soát quan hệ phụ thuộc, ghi nhận hoạt động, thông báo và cộng tác thời gian thực. Quy trình này được chọn để mô hình hóa vì nó phản ánh trực tiếp giá trị cốt lõi của hệ thống: hỗ trợ người dùng quản lý tiến độ công việc trên bảng Kanban một cách nhất quán, an toàn và cập nhật tức thời cho nhiều thành viên cùng làm việc.

Biểu đồ hoạt động của quy trình được trình bày trong Hình 2.10.



Hình 2.10: Biểu đồ hoạt động quy trình quản lý thẻ công việc, kiểm tra phụ thuộc và đồng bộ thời gian thực

Dựa trên Hình 2.10, quy trình bắt đầu khi thành viên bảng mở một bảng công việc và thực hiện thao tác với thẻ, chẳng hạn tạo thẻ mới, cập nhật nội dung, di chuyển thẻ giữa các danh sách, sắp xếp thứ tự thẻ hoặc thiết lập quan hệ phụ thuộc giữa các thẻ. Trước khi ghi nhận thay đổi, hệ thống kiểm tra quyền truy cập của người dùng trong phạm vi bảng và kiểm tra tính hợp lệ của dữ liệu đầu vào. Nếu người dùng không có quyền phù hợp hoặc dữ liệu không hợp lệ, thao tác bị từ chối và hệ thống hiển thị thông báo tương ứng.

Với các thao tác có liên quan đến quan hệ phụ thuộc, HustFlow thực hiện thêm bước kiểm tra vòng lặp phụ thuộc. Nếu quan hệ mới làm phát sinh vòng lặp, hệ thống không lưu thay đổi và cảnh báo để người dùng điều chỉnh lại thao tác. Ngược lại, nếu dữ liệu hợp lệ và không tạo vòng lặp, hệ thống lưu thay đổi vào cơ sở dữ liệu, ghi nhật ký hoạt động và phát sự kiện đồng bộ thời gian thực tới các thành viên khác đang mở cùng bảng. Nhờ đó, trạng thái bảng được cập nhật đồng nhất trên nhiều giao diện, đồng thời các thông báo cần thiết cũng được tạo ra để người dùng theo dõi diễn biến công việc.

Bên cạnh quy trình trên, hệ thống còn có một số quy trình nghiệp vụ đáng chú ý khác như kiểm tra giới hạn số bảng và nâng cấp gói trả phí, hoặc sử dụng trợ lý trí tuệ nhân tạo để hỗ trợ khởi tạo và phân tích công việc. Tuy nhiên, các quy trình này có phạm vi hẹp hơn hoặc phụ thuộc nhiều vào dịch vụ bên ngoài. Vì vậy, trong phạm vi khảo sát và phân tích yêu cầu, quy trình quản lý thẻ công việc được ưu tiên mô hình hóa do liên quan trực tiếp đến hoạt động thường xuyên nhất của người

dùng và tác động đến nhiều thành phần chức năng cốt lõi của HustFlow.

2.3 Đặc tả chức năng

Phần này đặc tả bảy use case cốt lõi của HustFlow được lựa chọn dựa trên mức độ quan trọng và độ phức tạp của quy trình nghiệp vụ, bao gồm: tạo bảng mới, xem và cập nhật chi tiết thẻ, kéo thả và di chuyển thẻ, thiết lập quan hệ phụ thuộc giữa các thẻ, bình luận trong thẻ, theo dõi tiến độ và báo cáo, cùng với việc tạo thẻ tự động từ văn bản thô.

2.3.1 Đặc tả use case 1: Tạo bảng mới

Bảng 2.1: Đặc tả use case Tạo bảng mới

Mã use case	UC001
Tên use case	Tạo bảng mới
Tác nhân	Thành viên tổ chức, quản trị viên tổ chức
Tiền điều kiện	Người dùng đã đăng nhập và là thành viên của tổ chức tương ứng.
Luồng sự kiện chính	1. Người dùng chọn chức năng tạo bảng mới trên giao diện không gian làm việc của tổ chức. 2. Hệ thống hiển thị hộp thoại yêu cầu nhập tiêu đề bảng và chọn ảnh nền. 3. Người dùng nhập tiêu đề bảng, chọn ảnh nền từ danh sách được cung cấp và xác nhận gửi yêu cầu. 4. Hệ thống kiểm tra quyền hạn của người dùng trong tổ chức và truy xuất thông tin gói dịch vụ hiện tại. 5. Hệ thống kiểm tra hạn mức số lượng bảng tối đa được phép tạo của gói dịch vụ. 6. Nếu dữ liệu hợp lệ và tổ chức còn hạn mức, hệ thống tạo bảng trong cơ sở dữ liệu, gán người tạo làm quản trị viên bảng và liên kết bảng với tổ chức hiện tại. 7. Hệ thống thông báo tạo bảng thành công và chuyển hướng người dùng vào giao diện làm việc của bảng mới.

Luồng sự kiện thay thế	5a. Nếu tổ chức đã đạt giới hạn bảng của gói hiện tại, hệ thống không tạo bảng và hiển thị thông báo yêu cầu nâng cấp gói dịch vụ. 3a. Nếu tiêu đề bảng bị bỏ trống hoặc không hợp lệ, hệ thống hiển thị lỗi tại trường nhập liệu và yêu cầu người dùng nhập lại.
Hậu điều kiện	Bảng mới được khởi tạo thành công trong tổ chức, người tạo được gán vai trò quản trị viên bảng và danh sách bảng hiển thị được cập nhật.

2.3.2 Đặc tả use case 2: Xem và cập nhật chi tiết thẻ

Bảng 2.2: Đặc tả use case Xem và cập nhật chi tiết thẻ

Mã use case	UC002
Tên use case	Xem và cập nhật chi tiết thẻ
Tác nhân	Thành viên bảng, quản trị viên bảng
Tiền điều kiện	Người dùng đã truy cập vào bảng công việc với vai trò thành viên bảng hoặc quản trị viên bảng.
Luồng sự kiện chính	1. Người dùng chọn một thẻ công việc trên bảng Kanban. 2. Hệ thống hiển thị hộp thoại chi tiết thẻ với các thông tin hiện có. 3. Người dùng cập nhật các thông tin cần thiết như mô tả, thành viên phụ trách, nhãn, ngày bắt đầu, hạn chót, nhắc việc, checklist hoặc tệp đính kèm. 4. Hệ thống kiểm tra quyền chỉnh sửa và tính hợp lệ của dữ liệu đầu vào. 5. Hệ thống lưu thay đổi vào cơ sở dữ liệu và cập nhật nhật ký hoạt động của bảng. 6. Hệ thống phát sự kiện đồng bộ thời gian thực tới các thành viên khác đang cùng truy cập bảng.

Luồng sự kiện thay thế	4a. Nếu người dùng có vai trò thành viên quan sát, hệ thống chỉ cho phép xem và vô hiệu hóa các khả năng sửa đổi. 3a. Nếu tệp đính kèm vượt quá giới hạn dung lượng hoặc sai định dạng, hệ thống chặn tải lên, hiển thị lỗi và giữ nguyên dữ liệu cũ. 4b. Nếu dữ liệu nhập không hợp lệ, hệ thống hiển thị lỗi tại trường tương ứng và không lưu thay đổi.
Hậu điều kiện	Thẻ công việc được lưu với thông tin mới nhất, lịch sử hoạt động được ghi nhận và trạng thái thẻ được đồng bộ tới các thành viên liên quan.

2.3.3 Đặc tả use case 3: Kéo thả và di chuyển thẻ

Bảng 2.3: Đặc tả use case Kéo thả và di chuyển thẻ

Mã use case	UC003
Tên use case	Kéo thả và di chuyển thẻ
Tác nhân	Thành viên bảng, quản trị viên bảng
Tiền điều kiện	Người dùng đang mở giao diện bảng ở chế độ Kanban và có quyền chỉnh sửa trên bảng.
Luồng sự kiện chính	1. Người dùng nhấn giữ một thẻ công việc trên bảng Kanban. 2. Người dùng kéo thẻ đến vị trí mới trong cùng danh sách hoặc sang một danh sách khác rồi thả thẻ. 3. Giao diện áp dụng hiển thị lạc quan để di chuyển thẻ ngay lập tức. 4. Hệ thống gửi yêu cầu cập nhật vị trí thẻ, danh sách nguồn, danh sách đích và thứ tự mới về máy chủ. 5. Máy chủ kiểm tra quyền chỉnh sửa của người dùng đối với bảng. 6. Hệ thống cập nhật thứ tự sắp xếp và danh sách sở hữu của thẻ trong cơ sở dữ liệu. 7. Hệ thống phát sự kiện đồng bộ thời gian thực tới các thành viên khác đang mở bảng.

Luồng sự kiện thay thế	5a. Nếu người dùng chỉ có quyền xem bảng, máy chủ từ chối lưu thay đổi và giao diện trả thẻ về vị trí ban đầu. 6a. Nếu kết nối mạng hoặc máy chủ bị gián đoạn, hệ thống khôi phục vị trí cũ của thẻ và hiển thị thông báo lỗi.
Hậu điều kiện	Thứ tự sắp xếp và danh sách sở hữu thẻ được cập nhật chính xác; giao diện của các thành viên đang mở bảng được đồng bộ tức thời.

2.3.4 Đặc tả use case 4: Thiết lập quan hệ phụ thuộc giữa các thẻ

Bảng 2.4: Đặc tả use case Thiết lập quan hệ phụ thuộc giữa các thẻ

Mã use case	UC004
Tên use case	Thiết lập quan hệ phụ thuộc giữa các thẻ
Tác nhân	Thành viên bảng, quản trị viên bảng
Tiền điều kiện	Bảng công việc hiện tại có ít nhất hai thẻ công việc hợp lệ đang hoạt động và người dùng có quyền chỉnh sửa bảng.
Luồng sự kiện chính	1. Người dùng mở hộp thoại chi tiết của thẻ đích. 2. Người dùng chọn chức năng thêm quan hệ phụ thuộc. 3. Hệ thống hiển thị danh sách các thẻ công việc khác trong cùng bảng để người dùng chọn làm thẻ tiền đề. 4. Người dùng chọn thẻ tiền đề và xác nhận. 5. Hệ thống duyệt đồ thị phụ thuộc hiện tại để kiểm tra liên kết mới có tạo chu trình hay không. 6. Nếu không phát hiện vòng lặp, hệ thống lưu quan hệ phụ thuộc vào cơ sở dữ liệu. 7. Hệ thống ghi nhật ký hoạt động và gửi cập nhật thời gian thực đến các thành viên đang mở bảng.
Luồng sự kiện thay thế	5a. Nếu liên kết mới tạo vòng lặp phụ thuộc, hệ thống từ chối lưu và hiển thị cảnh báo để người dùng cấu hình lại. 4a. Nếu thẻ tiền đề vừa bị xóa hoặc lưu trữ, hệ thống báo lỗi dữ liệu không tồn tại và yêu cầu tải lại danh sách thẻ. 4b. Nếu người dùng chọn chính thẻ hiện tại làm thẻ tiền đề, hệ thống từ chối thao tác.

Hậu điều kiện	Quan hệ phụ thuộc được ghi nhận thành công, trạng thái phụ thuộc được cập nhật trên giao diện thẻ và đồng bộ theo thời gian thực.
----------------------	---

2.3.5 Đặc tả use case 5: Bình luận trong thẻ

Bảng 2.5: Đặc tả use case Bình luận trong thẻ

Mã use case	UC005
Tên use case	Bình luận trong thẻ
Tác nhân	Thành viên bảng, quản trị viên bảng
Tiền điều kiện	Người dùng có quyền truy cập vào thẻ công việc tương ứng.
Luồng sự kiện chính	1. Người dùng mở hộp thoại chi tiết thẻ và di chuyển đến khu vực thảo luận. 2. Người dùng nhập nội dung bình luận vào ô soạn thảo. 3. Người dùng có thể gõ ký tự @ kèm tên thành viên để nhắc tên trong bình luận. 4. Hệ thống hiển thị danh sách gợi ý thành viên phù hợp; người dùng chọn thành viên cần nhắc tên. 5. Người dùng gửi bình luận. 6. Hệ thống kiểm tra quyền gửi bình luận và phân tích nội dung để trích xuất các thành viên được nhắc tên. 7. Hệ thống lưu bình luận, tạo thông báo cho các thành viên được nhắc tên và ghi nhận hoạt động liên quan. 8. Hệ thống phát sự kiện thời gian thực để hiển thị bình luận mới trên giao diện của người dùng khác đang mở thẻ.
Luồng sự kiện thay thế	5a. Nếu nội dung bình luận trống hoặc chỉ chứa khoảng trắng, hệ thống chặn gửi và yêu cầu người dùng nhập nội dung hợp lệ. 7a. Nếu thành viên được nhắc tên không còn quyền truy cập thẻ, hệ thống vẫn lưu bình luận nhưng không tạo thông báo nhắc tên cho thành viên đó.
Hậu điều kiện	Bình luận được hiển thị trong lịch sử thảo luận của thẻ, thông báo được gửi tới thành viên được nhắc tên và giao diện thảo luận được cập nhật thời gian thực.

2.3.6 Đặc tả use case 6: Theo dõi tiến độ và báo cáo

Bảng 2.6: Đặc tả use case Theo dõi tiến độ và báo cáo

Mã use case	UC006
Tên use case	Theo dõi tiến độ và báo cáo
Tác nhân	Thành viên bảng, quản trị viên bảng, thành viên quan sát
Tiền điều kiện	Người dùng có quyền truy cập vào bảng công việc tương ứng.
Luồng sự kiện chính	1. Người dùng chọn chế độ hiển thị như lịch, biểu đồ Gantt hoặc thống kê/báo cáo từ thanh công cụ của bảng. 2. Hệ thống truy xuất dữ liệu thẻ, danh sách, thành viên và quan hệ phụ thuộc trong bảng. 3. Hệ thống tính toán và định dạng dữ liệu theo chế độ xem được chọn. 4. Với chế độ lịch, hệ thống sắp xếp thẻ theo ngày bắt đầu và hạn chót. 5. Với chế độ Gantt, hệ thống biểu diễn thẻ trên dòng thời gian và thể hiện quan hệ phụ thuộc giữa các thẻ. 6. Với chế độ thống kê, hệ thống tổng hợp số thẻ theo trạng thái, số thẻ quá hạn, số thẻ hoàn thành và phân bố công việc theo thành viên. 7. Hệ thống hiển thị giao diện báo cáo hoặc tiến độ trực quan để người dùng khai thác thông tin. 8. Người dùng có thể lọc dữ liệu theo nhãn, thành viên hoặc khoảng thời gian.
Luồng sự kiện thay thế	2a. Nếu bảng chưa có danh sách hoặc thẻ công việc, hệ thống hiển thị trạng thái trống và hướng dẫn người dùng tạo dữ liệu trước. 7a. Nếu người dùng có vai trò thành viên quan sát, hệ thống vẫn hiển thị báo cáo nhưng khóa các thao tác chỉnh sửa trực tiếp trên lịch hoặc dòng thời gian.
Hậu điều kiện	Báo cáo, biểu đồ, giao diện Gantt và lịch hiển thị thông tin phản ánh dữ liệu hiện tại của bảng công việc.

2.3.7 Đặc tả use case 7: Tạo thẻ tự động từ văn bản thô

Bảng 2.7: Đặc tả use case Tạo thẻ tự động từ văn bản thô

Mã use case	UC007
Tên use case	Tạo thẻ tự động từ văn bản thô
Tác nhân	Thành viên bảng, quản trị viên bảng, trợ lý AI
Tiền điều kiện	Người dùng có quyền chỉnh sửa trên bảng và bảng hiện tại có ít nhất một danh sách đang hoạt động để tạo thẻ.
Luồng sự kiện chính	1. Người dùng chọn chức năng tạo thẻ bằng AI trên bảng công việc. 2. Hệ thống hiển thị hộp thoại yêu cầu nhập văn bản mô tả công việc tự do. 3. Người dùng nhập văn bản mô tả và yêu cầu AI phân tích. 4. Hệ thống kiểm tra quyền chỉnh sửa, kiểm tra độ dài nội dung đầu vào, đóng gói yêu cầu cùng ngữ cảnh bảng hiện tại và gửi tới dịch vụ trợ lý AI. 5. Trợ lý AI phân tích nội dung và trả về bản nháp có cấu trúc gồm tiêu đề, mô tả, danh sách checklist gợi ý và có thể kèm hạn chót, người phụ trách, nhãn hoặc danh sách đích. 6. Hệ thống kiểm tra, chuẩn hóa dữ liệu trả về; các giá trị không hợp lệ bị loại bỏ hoặc thay bằng giá trị dự phòng, sau đó bản nháp thẻ được hiển thị cho người dùng xem trước. 7. Người dùng rà soát, chỉnh sửa tiêu đề, mô tả, checklist, hạn chót, người phụ trách, nhãn và danh sách đích nếu cần rồi xác nhận tạo thẻ. 8. Hệ thống kiểm tra lại quyền chỉnh sửa và danh sách đích, lưu thẻ công việc cùng các checklist, người phụ trách và nhãn hợp lệ vào cơ sở dữ liệu. 9. Hệ thống ghi nhật ký hoạt động, phát sự kiện đồng bộ thời gian thực, làm mới dữ liệu bảng, mở thẻ vừa tạo và thông báo thành công.

Luồng sự kiện thay thế	3a. Nếu nội dung trống hoặc vượt quá giới hạn cho phép, hệ thống chặn yêu cầu phân tích và hiển thị thông báo lỗi phù hợp. 4a. Nếu người dùng không còn quyền chỉnh sửa hoặc bảng không có danh sách đang hoạt động, hệ thống từ chối thao tác. 4b. Nếu dịch vụ AI lỗi, quá tải hoặc quá thời gian phản hồi, hệ thống hiển thị thông báo lỗi và cho phép người dùng thử lại hoặc tạo thẻ thủ công. 5a. Nếu kết quả trả về không đúng cấu trúc JSON yêu cầu, hệ thống báo lỗi xử lý và cho phép gửi lại yêu cầu phân tích. 6a. Nếu một số gợi ý như người phụ trách, nhân hoặc danh sách đích không hợp lệ, hệ thống bỏ qua các gợi ý đó hoặc thay bằng giá trị mặc định trong phạm vi bảng. 7a. Nếu người dùng hủy bản nháp, hệ thống không tạo thẻ mới. 8a. Nếu danh sách đích không còn tồn tại khi người dùng xác nhận, hệ thống từ chối lưu và yêu cầu chọn lại danh sách.
Hậu điều kiện	Sau khi người dùng phê duyệt bản nháp, thẻ công việc được tạo trong danh sách đã chọn; các checklist, người phụ trách và nhân hợp lệ được gắn vào thẻ, nhật ký hoạt động được ghi nhận và dữ liệu bảng được cập nhật.

2.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng đã được mô tả thông qua các use case, HustFlow cần đáp ứng một số yêu cầu phi chức năng để hệ thống có thể vận hành ổn định trong môi trường cộng tác nhóm. Các yêu cầu này tập trung vào năm khía cạnh chính: (i) hiệu năng, (ii) bảo mật và phân quyền, (iii) độ tin cậy và tính toàn vẹn dữ liệu, (iv) tính dễ dùng, cùng (v) khả năng bảo trì và mở rộng. Việc xác định rõ các yêu cầu phi chức năng giúp quá trình lựa chọn công nghệ, thiết kế kiến trúc và kiểm thử hệ thống ở các chương sau có cơ sở cụ thể hơn.

2.4.1 Tính hiệu năng

HustFlow là hệ thống làm việc cộng tác, do đó hiệu năng cần được đánh giá trước hết ở các thao tác được sử dụng thường xuyên. Các thao tác như mở bảng, mở chi tiết thẻ, tạo thẻ mới, kéo thả thẻ giữa các danh sách, cập nhật danh sách kiểm tra và gửi bình luận phải có thời gian phản hồi đủ ngắn để không làm gián đoạn

luồng làm việc của người dùng. Trong điều kiện mạng ổn định, các thay đổi quan trọng trên bảng cần được đồng bộ tới những thành viên liên quan gần như theo thời gian thực (near real-time), đặc biệt đối với các thao tác ảnh hưởng trực tiếp đến trạng thái công việc.

Đối với các màn hình có lượng dữ liệu lớn như bảng Kanban, lịch, biểu đồ Gantt và thống kê, hệ thống cần hạn chế tải lại toàn bộ dữ liệu khi không cần thiết, đồng thời chỉ hiển thị hoặc tính toán những thông tin phục vụ trực tiếp cho chế độ xem hiện tại. Các thao tác sử dụng trợ lý trí tuệ nhân tạo có thể có thời gian xử lý dài hơn thao tác thông thường, nhưng giao diện phải thể hiện rõ trạng thái đang xử lý, có cơ chế giới hạn thời gian chờ và trả về thông báo phù hợp khi dịch vụ bên ngoài không phản hồi.

2.4.2 Bảo mật và phân quyền

Hệ thống phải yêu cầu người dùng xác thực trước khi truy cập các tài nguyên thuộc không gian làm việc, bảng và thẻ công việc. Mọi yêu cầu đọc hoặc ghi dữ liệu cần được kiểm tra theo ngữ cảnh tổ chức, bảng và vai trò của người dùng, tránh trường hợp người dùng truy cập vào tài nguyên không thuộc phạm vi được cấp quyền. Cơ chế phân quyền phải được áp dụng nhất quán cho các vai trò quản trị viên, thành viên và thành viên quan sát trong các nghiệp vụ như tạo bảng, cập nhật thẻ, mời thành viên, bình luận, quản lý tệp đính kèm, sử dụng trợ lý trí tuệ nhân tạo và xem báo cáo.

Đối với dữ liệu nhạy cảm, hệ thống không được hiển thị thông tin nội bộ của máy chủ, câu lệnh truy vấn hoặc dấu vết lỗi kỹ thuật cho người dùng cuối. Các chức năng thanh toán phải được thực hiện thông qua cổng thanh toán Stripe; chỉ quản trị viên tổ chức mới có quyền tạo phiên thanh toán hoặc quản lý thuê bao. Trạng thái gói dịch vụ chỉ được cập nhật khi hệ thống nhận được sự kiện hợp lệ từ Stripe, nhằm hạn chế rủi ro thay đổi trái phép thông tin thanh toán.

2.4.3 Độ tin cậy và tính toàn vẹn dữ liệu

Các thao tác cập nhật dữ liệu cần bảo đảm tính nhất quán giữa giao diện người dùng và cơ sở dữ liệu. Khi người dùng thay đổi thứ tự thẻ, cập nhật danh sách kiểm tra, thiết lập quan hệ phụ thuộc, thêm thành viên hoặc thay đổi vai trò, hệ thống phải kiểm tra dữ liệu đầu vào và chỉ ghi nhận thay đổi hợp lệ. Trong trường hợp thao tác thất bại, giao diện cần thông báo lỗi rõ ràng và khôi phục trạng thái phù hợp, tránh để người dùng tiếp tục làm việc trên dữ liệu hiển thị sai.

HustFlow cũng cần duy trì ranh giới rõ ràng giữa dữ liệu đang hoạt động và dữ liệu đã lưu trữ. Các thẻ hoặc danh sách đã lưu trữ không được xuất hiện trong các thao tác chỉnh sửa thông thường, nhưng vẫn có thể được khôi phục hoặc xóa vĩnh

viễn bởi người dùng có quyền phù hợp. Những thay đổi quan trọng như tạo, cập nhật, xóa, lưu trữ, khôi phục thẻ hoặc thay đổi thành viên cần được ghi vào nhật ký hoạt động để phục vụ theo dõi lịch sử và hỗ trợ kiểm tra khi có sự cố.

2.4.4 Tính dễ dùng

Giao diện của HustFlow cần trực quan đối với nhóm người dùng phổ thông, bao gồm sinh viên, nhóm nghiên cứu và nhóm phát triển nhỏ. Các thao tác chính như kéo thả thẻ, tạo danh sách, gán thành viên, thêm nhãn, đặt hạn chót, bình luận và xem báo cáo cần được bố trí dễ nhận biết. Hệ thống phải có phản hồi rõ ràng cho các trạng thái đang tải, thành công, thất bại hoặc không đủ quyền, giúp người dùng hiểu được kết quả của thao tác mà không cần diễn giải kỹ thuật.

Do hệ thống hỗ trợ nhiều góc nhìn công việc, hệ thống cần hỗ trợ người dùng chuyển đổi linh hoạt giữa các chế độ xem: bảng Kanban, lịch, biểu đồ Gantt, chế độ xem chia đôi (bảng kết hợp lịch) và thống kê mà vẫn giữ được ngữ cảnh của bảng hiện tại. Các thông báo lỗi liên quan đến phân quyền, dữ liệu không hợp lệ, thanh toán hoặc trợ lý trí tuệ nhân tạo cần được trình bày bằng tiếng Việt rõ ràng. Giao diện cũng cần thích ứng với các kích thước màn hình phổ biến, đặc biệt tại những khu vực có mật độ thông tin cao như bảng Kanban, lịch và biểu đồ Gantt.

2.4.5 Tính bảo trì và mở rộng

Hệ thống cần được tổ chức theo hướng tách biệt rõ giữa các nhóm nghiệp vụ như quản lý bảng, quản lý thẻ, danh sách kiểm tra, bình luận, thông báo, quan hệ phụ thuộc, trợ lý trí tuệ nhân tạo và thanh toán. Ranh giới xử lý rõ ràng giúp giảm phụ thuộc giữa các phần của hệ thống, đồng thời tạo điều kiện bổ sung chức năng mới như bộ lọc nâng cao, loại báo cáo mới, hình thức thông báo mới hoặc tác vụ trí tuệ nhân tạo mới mà không ảnh hưởng lớn đến những chức năng đã ổn định.

Ở mức kỹ thuật, hệ thống cần duy trì cơ chế kiểm tra dữ liệu đầu vào trước khi ghi vào cơ sở dữ liệu, bao gồm cả dữ liệu do người dùng nhập và dữ liệu do dịch vụ trí tuệ nhân tạo trả về. Việc sử dụng cơ sở dữ liệu quan hệ thông qua Prisma và MySQL đòi hỏi mô hình dữ liệu phải phản ánh đúng quan hệ giữa tổ chức, bảng, thành viên, danh sách, thẻ, bình luận, tệp đính kèm và thanh toán. Các lỗi phía máy chủ cần được ghi nhận ở mức phù hợp để hỗ trợ kiểm tra, bảo trì và cải tiến hệ thống trong quá trình phát triển lâu dài.

Chương 2 đã khảo sát một số công cụ quản lý công việc phổ biến, từ đó xác định những yêu cầu nghiệp vụ cần thiết cho HustFlow trong phạm vi nhóm làm việc vừa và nhỏ. Trên cơ sở kết quả khảo sát, chương này đã trình bày mô hình use case tổng quát, các nhóm use case phân rã và bảy use case cốt lõi được đặc tả chi tiết của hệ thống, bao gồm: (i) tạo bảng mới, (ii) xem và cập nhật chi tiết thẻ, (iii) kéo thả và

di chuyển thẻ, (iv) thiết lập quan hệ phụ thuộc giữa các thẻ, (v) bình luận trong thẻ, (vi) theo dõi tiến độ và báo cáo, cùng với (vii) tạo thẻ tự động từ văn bản thô. Bên cạnh đó, các yêu cầu phi chức năng về hiệu năng, bảo mật, độ tin cậy dữ liệu, tính dễ dùng và khả năng bảo trì cũng được xác định để làm rõ các ràng buộc khi triển khai hệ thống. Những kết quả phân tích trong chương này là cơ sở để lựa chọn nền tảng lý thuyết và công nghệ sử dụng, nội dung sẽ được trình bày trong Chương 3.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã trình bày kết quả khảo sát hiện trạng, các nhóm use case cốt lõi và các yêu cầu phi chức năng của hệ thống HustFlow. Từ những yêu cầu đó, chương này trình bày các nền tảng lý thuyết và công nghệ được lựa chọn để hiện thực hóa hệ thống. Nội dung chương tập trung vào hai nhóm chính: nền tảng lý thuyết phục vụ mô hình hóa dữ liệu và kiểm soát phụ thuộc công việc tại phần 3.1, sau đó là các công nghệ phát triển ứng dụng được sử dụng để đáp ứng yêu cầu về giao diện, lưu trữ, cộng tác thời gian thực, xác thực, thanh toán và hỗ trợ trí tuệ nhân tạo tại phần 3.2. Với mỗi công nghệ, chương phân tích yêu cầu cụ thể ở Chương 2 mà công nghệ đó giải quyết, các lựa chọn thay thế có thể sử dụng và lý do lựa chọn trong phạm vi đề án.

3.1 Nền tảng lý thuyết

HustFlow là hệ thống quản lý công việc theo mô hình bảng Kanban, trong đó dữ liệu có quan hệ phân cấp và các thẻ công việc có thể phụ thuộc lẫn nhau. Vì vậy, hai nền tảng lý thuyết cần được làm rõ trước khi lựa chọn công nghệ là: (i) đồ thị có hướng phi chu trình để kiểm soát quan hệ phụ thuộc giữa các thẻ, và (ii) mô hình cơ sở dữ liệu quan hệ để bảo đảm tính toàn vẹn của dữ liệu nghiệp vụ.

3.1.1 Đồ thị có hướng phi chu trình và phát hiện chu trình

Use case UC004 ở Chương 2 yêu cầu hệ thống cho phép thiết lập quan hệ phụ thuộc giữa các thẻ công việc. Nếu các quan hệ này tạo thành vòng lặp, ví dụ thẻ A chặn thẻ B, thẻ B chặn thẻ C và thẻ C lại chặn thẻ A, người dùng sẽ không thể xác định công việc nào cần hoàn thành trước. Để tránh trạng thái bế tắc đó, các quan hệ phụ thuộc trong một bảng cần được mô hình hóa dưới dạng đồ thị có hướng phi chu trình (Directed Acyclic Graph - DAG), một cấu trúc dữ liệu thường dùng cho các bài toán có ràng buộc thứ tự trước sau [5].

Hệ thống biểu diễn tập thẻ trên một bảng bằng đồ thị có hướng $G = (V, E)$, trong đó V là tập đỉnh đại diện cho các thẻ công việc, còn E là tập cạnh có hướng đại diện cho quan hệ phụ thuộc:

$$V = \{v_1, v_2, \dots, v_n\} \quad (3.1)$$

$$E = \{(u, v) \mid u, v \in V, u \text{ chặn } v\} \quad (3.2)$$

Theo định nghĩa ở công thức (3.2), cạnh (u, v) thể hiện rằng thẻ u là điều kiện cần hoàn thành trước khi thẻ v có thể được xử lý. Khi người dùng thêm một quan hệ

phụ thuộc mới, hệ thống cần kiểm tra xem cạnh mới đó có tạo ra chu trình dạng $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$ hay không.

Trong HustFlow, bài toán được giải quyết bằng phép duyệt theo chiều rộng (Breadth-First Search - BFS) để kiểm tra tính khả đạt. Giả sử hệ thống chuẩn bị thêm cạnh $u \rightarrow v$, trong đó u là thẻ chặn và v là thẻ bị chặn. Thuật toán bắt đầu từ v , lần lượt duyệt các thẻ có thể đi tới theo quan hệ phụ thuộc hiện có. Nếu gặp u , cạnh mới sẽ khép kín một chu trình và bị từ chối. Hàng đợi lưu các đỉnh chưa xử lý, còn tập đỉnh đã thăm bảo đảm mỗi thẻ chỉ được xét một lần. Thuật toán có độ phức tạp thời gian $O(V + E)$ và độ phức tạp không gian $O(V)$ [5], phù hợp với quy mô một bảng công việc trong hệ thống.

Các hướng tiếp cận thay thế gồm DFS và thuật toán Kahn dựa trên sắp xếp topo; các thuật toán này đều có thể xử lý đồ thị trong thời gian tuyến tính theo số đỉnh và cạnh [5]. BFS được lựa chọn cho cài đặt hiện tại vì thao tác thêm cạnh chỉ cần xác định có tồn tại đường đi từ thẻ bị chặn tới thẻ chặn hay không. Thuật toán có thể dừng khi tìm thấy thẻ đích, không cần tính lại thứ tự topo của toàn bộ bảng và dễ kết hợp với danh sách kẻ được tạo từ các quan hệ đang lưu. Đây là quyết định thiết kế trong phạm vi HustFlow nhằm đáp ứng yêu cầu về độ tin cậy dữ liệu ở phần 2.4.3 và yêu cầu phản hồi ở phần 2.4.1.

Chi tiết về cách ánh xạ quan hệ phụ thuộc, chuỗi kiểm tra phía máy chủ và mã giả BFS trong HustFlow được phân tích tại phần 5.2.

3.1.2 Mô hình cơ sở dữ liệu quan hệ và toàn vẹn dữ liệu

Cấu trúc dữ liệu của HustFlow có nhiều quan hệ chặt chẽ: tổ chức sở hữu bảng, bảng chứa danh sách, danh sách chứa thẻ, thẻ có checklist, bình luận, nhãn, tệp đính kèm và quan hệ phụ thuộc. Với đặc điểm này, mô hình cơ sở dữ liệu quan hệ là nền tảng phù hợp vì dữ liệu được tổ chức thành các bảng có lược đồ rõ ràng, được liên kết bằng khóa chính, khóa ngoại và các ràng buộc toàn vẹn [6].

Yêu cầu phi chức năng tại phần 2.4.3 nhấn mạnh tính nhất quán của dữ liệu khi người dùng cập nhật thẻ, danh sách, thành viên, vai trò hoặc thanh toán. Trong mô hình quan hệ, các ràng buộc khóa chính, khóa ngoại, ràng buộc duy nhất và giao dịch ACID giúp hạn chế các trạng thái dữ liệu không hợp lệ. Ví dụ, khi xóa một bảng, các danh sách và thẻ thuộc bảng đó cũng cần được xử lý nhất quán để tránh bản ghi mồ côi. Quan hệ này có thể được mô tả như sau:

$$\text{Delete}(\text{Board}) \implies \forall \text{List} \in \text{Board} : \text{Delete}(\text{List}) \implies \forall \text{Card} \in \text{List} : \text{Delete}(\text{Card}) \quad (3.3)$$

Công thức (3.3) cho thấy cơ chế xóa lan truyền cần được quản lý ở mức cơ sở dữ

liệu hoặc lớp truy cập dữ liệu, thay vì chỉ dựa vào xử lý thủ công ở giao diện.

Một lựa chọn thay thế là cơ sở dữ liệu phi quan hệ dạng tài liệu, chẳng hạn MongoDB. Cách tiếp cận này linh hoạt khi lưu các đối tượng JSON lồng nhau và có thể thuận tiện cho một số màn hình đọc dữ liệu thể. Tuy nhiên, HustFlow có nhiều quan hệ nhiều-nhiều và các ràng buộc phân quyền, phụ thuộc, thanh toán cần kiểm soát chặt chẽ. Nếu sử dụng mô hình tài liệu, nhiều ràng buộc sẽ phải tự triển khai ở tầng ứng dụng, làm tăng nguy cơ bất đồng bộ dữ liệu. Vì vậy, mô hình quan hệ được lựa chọn để bảo đảm dữ liệu có cấu trúc rõ ràng, dễ kiểm tra và phù hợp với yêu cầu bảo trì, mở rộng ở phần 2.4.5.

3.2 Các công nghệ phát triển ứng dụng

Trên cơ sở các yêu cầu đã xác định ở Chương 2, HustFlow được triển khai bằng nhóm công nghệ phục vụ các vai trò khác nhau trong hệ thống: Next.js và React cho giao diện và xử lý full-stack, MySQL và Prisma cho lưu trữ dữ liệu, Pusher cho đồng bộ thời gian thực, Clerk cho xác thực và phân quyền, Stripe cho thanh toán, OpenAI API cho trợ lý trí tuệ nhân tạo và @hello-pangea/dnd cho tương tác kéo thả. Các công nghệ này khớp với định hướng kỹ thuật đã nêu ở Chương 1 và được phân tích cụ thể trong các mục sau.

3.2.1 Next.js và React

Next.js [7] là framework phát triển ứng dụng web dựa trên React [8], hỗ trợ kết hợp giao diện người dùng với xử lý phía máy chủ trong cùng một dự án. Trong HustFlow, Next.js và React được sử dụng để xây dựng giao diện bảng Kanban, lịch, biểu đồ tiến độ, trang quản trị tổ chức và các luồng thao tác với thẻ công việc.

Công nghệ này giải quyết trực tiếp yêu cầu về tính dễ dùng ở phần 2.4.4, yêu cầu hiệu năng ở phần 2.4.1 và yêu cầu bảo trì ở phần 2.4.5. React giúp chia giao diện thành các thành phần nhỏ có thể tái sử dụng, phù hợp với các màn hình có nhiều vùng tương tác như bảng Kanban, modal chi tiết thẻ và khu vực bình luận. Next.js App Router hỗ trợ tổ chức mã nguồn theo tuyến đường, kết xuất phía máy chủ và Server Actions, nhờ đó các thao tác như tạo bảng, cập nhật thẻ hoặc gửi bình luận có thể được xử lý gần với lớp dữ liệu hơn.

Một phương án thay thế là xây dựng ứng dụng trang đơn bằng React kết hợp Vite. Phương án này nhẹ và phù hợp với các ứng dụng chỉ cần giao diện phía máy khách, nhưng HustFlow cần nhiều xử lý phía máy chủ như kiểm tra phân quyền, truy vấn dữ liệu, xử lý thanh toán và gọi API bên ngoài. Một phương án khác là tách backend bằng Spring Boot hoặc Laravel và frontend bằng React. Cách này mạnh trong các hệ thống lớn, nhưng làm tăng số lượng dự án cần duy trì trong phạm vi đề án. Vì vậy, Next.js được lựa chọn vì cân bằng giữa tốc độ phát triển, khả năng tổ

chức mã nguồn và khả năng triển khai các luồng full-stack trong một hệ sinh thái thống nhất.

3.2.2 MySQL và Prisma ORM

HustFlow sử dụng hệ quản trị cơ sở dữ liệu quan hệ MySQL [9] để lưu trữ dữ liệu lâu bền, kết hợp Prisma ORM [10] làm lớp truy cập dữ liệu trong mã nguồn TypeScript. MySQL lưu trữ các thực thể chính như tổ chức, bảng, thành viên, danh sách, thẻ, checklist, bình luận, tệp đính kèm và thông tin thuê bao. Prisma giúp ánh xạ các bảng này thành các kiểu dữ liệu và hàm truy vấn an toàn hơn trong mã nguồn.

Cặp công nghệ này giải quyết yêu cầu toàn vẹn dữ liệu ở phần 2.4.3 và yêu cầu bảo trì ở phần 2.4.5. Thay vì viết nhiều câu SQL thủ công, Prisma cho phép định nghĩa lược đồ dữ liệu tập trung trong `schema.prisma`, từ đó sinh client truy vấn có kiểm tra kiểu. Điều này làm giảm lỗi khi thay đổi mô hình dữ liệu và giúp mã nguồn tăng nghiệp vụ dễ đọc hơn.

PostgreSQL là lựa chọn thay thế phù hợp nếu hệ thống cần nhiều tính năng nâng cao như kiểu dữ liệu JSONB mạnh, truy vấn phân tích phức tạp hoặc extension chuyên dụng. Tuy nhiên, với phạm vi HustFlow, MySQL đã đáp ứng đầy đủ các yêu cầu về quan hệ, chỉ mục và giao dịch, đồng thời quen thuộc và dễ triển khai trong môi trường phát triển. Ở lớp ORM, TypeORM hoặc Sequelize cũng có thể được sử dụng, nhưng Prisma có ưu thế ở khả năng sinh kiểu TypeScript trực tiếp từ lược đồ, phù hợp với mục tiêu giảm lỗi kiểu dữ liệu giữa backend và frontend.

3.2.3 Pusher cho đồng bộ thời gian thực

Pusher [11] là dịch vụ truyền thông điệp thời gian thực dựa trên WebSocket. Trong HustFlow, Pusher được sử dụng để phát tán sự kiện khi người dùng kéo thả thẻ, cập nhật thông tin thẻ hoặc gửi bình luận. Các thao tác này liên quan trực tiếp tới UC003, UC005 và yêu cầu đồng bộ gần thời gian thực ở phần 2.4.1.

Luồng xử lý tổng quát là: người dùng thực hiện thao tác trên giao diện, Server Action kiểm tra quyền và cập nhật dữ liệu, sau đó máy chủ gửi sự kiện qua Pusher tới những trình duyệt đang mở cùng bảng công việc. Khi nhận sự kiện, giao diện phía khách cập nhật dữ liệu tương ứng mà không yêu cầu người dùng tải lại trang. Cách làm này giúp trải nghiệm cộng tác của HustFlow nhất quán với mục tiêu đã nêu ở phần khảo sát hiện trạng.

Giải pháp thay thế là tự vận hành máy chủ WebSocket bằng Socket.io. Cách này cho phép kiểm soát sâu hơn nhưng yêu cầu duy trì tiến trình máy chủ kết nối lâu dài, xử lý mở rộng kết nối và triển khai hạ tầng riêng. Server-Sent Events cũng

có thể dùng cho một số luồng thông báo một chiều, nhưng không phù hợp bằng WebSocket cho các tương tác cộng tác dày đặc. Do đó ưu tiên triển khai ứng dụng trên kiến trúc web hiện đại và giảm chi phí vận hành hạ tầng thời gian thực, Pusher là lựa chọn phù hợp hơn.

Cơ chế kết hợp kênh riêng tư, payload có kiểu, cập nhật bộ nhớ đệm và các tầng phục hồi của hệ thống được trình bày chi tiết tại phần 5.1.

3.2.4 Clerk cho xác thực và phân quyền

Clerk [12] là dịch vụ xác thực và quản lý người dùng. Trong HustFlow, Clerk đảm nhiệm đăng ký, đăng nhập, quản lý phiên làm việc, tổ chức và vai trò thành viên ở cấp tổ chức. Công nghệ này giải quyết yêu cầu bảo mật và phân quyền tại phần 2.4.2, đặc biệt là yêu cầu mọi thao tác đọc ghi dữ liệu phải được kiểm tra theo ngữ cảnh tổ chức, bảng và vai trò người dùng.

Nếu tự xây dựng xác thực bằng Auth.js hoặc một thư viện tương tự, hệ thống sẽ có quyền kiểm soát dữ liệu người dùng tốt hơn, nhưng phải tự xử lý nhiều vấn đề phụ trợ như bảo mật mật khẩu, phiên đăng nhập, mời thành viên, xác thực hai yếu tố và quản lý tổ chức. Những phần này không phải đóng góp chính của đề án. Clerk được lựa chọn vì cung cấp sẵn các cơ chế xác thực phổ biến, tích hợp tốt với Next.js và cho phép đề án tập trung hơn vào nghiệp vụ quản lý công việc, phân quyền bảng và xử lý dữ liệu.

3.2.5 Stripe cho thanh toán gói dịch vụ

Stripe [13] là nền tảng thanh toán trực tuyến hỗ trợ tạo phiên thanh toán, quản lý thuê bao và gửi sự kiện webhook. Trong HustFlow, Stripe được dùng cho nhóm chức năng quản lý gói dịch vụ và thanh toán đã được mô tả trong phần phân rã use case ở Chương 2, đặc biệt tại Hình 2.9 và yêu cầu bảo mật thanh toán ở phần 2.4.2.

Khi quản trị viên tổ chức yêu cầu nâng cấp gói, hệ thống tạo phiên thanh toán và chuyển người dùng sang Stripe Checkout. Sau khi giao dịch hoàn tất, Stripe gửi webhook về HustFlow để hệ thống kiểm tra tính hợp lệ của sự kiện rồi cập nhật trạng thái thuê bao trong cơ sở dữ liệu. Thiết kế này giúp hệ thống không trực tiếp lưu trữ thông tin thẻ thanh toán, giảm rủi ro bảo mật và phù hợp với yêu cầu chỉ cập nhật gói dịch vụ khi nhận được sự kiện hợp lệ từ cổng thanh toán.

Các lựa chọn thay thế gồm VNPAY, MoMo hoặc chuyển khoản ngân hàng thủ công. Các cổng nội địa thuận tiện hơn cho thị trường Việt Nam, nhưng việc quản lý thuê bao định kỳ, môi trường kiểm thử và tài liệu tích hợp trong phạm vi đề án không đồng nhất bằng Stripe. Chuyển khoản thủ công dễ triển khai ban đầu nhưng không đáp ứng tốt yêu cầu tự động cập nhật trạng thái gói dịch vụ. Vì vậy, Stripe

được chọn để triển khai luồng Checkout, webhook và gia hạn thuê bao cơ bản trong phạm vi đồ án.

Phạm vi quyền quản lý thanh toán, cách nổi trạng thái thuê bao với giới hạn tài nguyên và các giới hạn của webhook hiện tại được trình bày trong phần thiết kế và kết quả xây dựng hệ thống tại Chương 4.

3.2.6 OpenAI API cho trợ lý trí tuệ nhân tạo

OpenAI API [14] cung cấp khả năng gọi mô hình ngôn ngữ lớn qua HTTP API. Trong HustFlow, API này được gọi từ phía máy chủ để hỗ trợ UC007, bao gồm phân tích văn bản tự do, tạo bản nháp thẻ, đề xuất checklist và hỗ trợ một số tác vụ tổng hợp nội dung. Công nghệ này liên quan trực tiếp tới yêu cầu tính dễ dùng ở phần 2.4.4, vì nó giảm số thao tác nhập liệu thủ công cho người dùng.

Khi người dùng nhập mô tả công việc dạng tự do, hệ thống gửi nội dung và ngữ cảnh cần thiết tới API, sau đó nhận về dữ liệu có cấu trúc để hiển thị bản nháp cho người dùng kiểm tra trước khi lưu. Việc gọi API từ máy chủ giúp bảo vệ khóa truy cập, đồng thời cho phép kiểm tra lại dữ liệu đầu ra trước khi ghi vào cơ sở dữ liệu, phù hợp với yêu cầu kiểm tra dữ liệu ở phần 2.4.5.

Các phương án thay thế gồm Google Gemini API hoặc tự triển khai mô hình mã nguồn mở như Llama thông qua hạ tầng riêng. Gemini có thể đáp ứng nhiều tác vụ ngôn ngữ tương tự, nhưng việc giữ một API thống nhất và dễ kiểm soát đầu ra có cấu trúc giúp giảm độ phức tạp triển khai. Mô hình mã nguồn mở cho phép kiểm soát dữ liệu tốt hơn, nhưng đòi hỏi tài nguyên vận hành lớn và không phù hợp với phạm vi triển khai của đồ án. Do đó, OpenAI API được lựa chọn để cân bằng giữa chất lượng kết quả, thời gian phát triển và chi phí hạ tầng.

Các lớp kiểm soát đầu ra, đối chiếu định danh và quy trình xác nhận trước khi ghi dữ liệu được trình bày tại phần 5.3.

3.2.7 @hello-pangea/dnd cho tương tác kéo thả

Thư viện @hello-pangea/dnd [15] hỗ trợ xây dựng tương tác kéo thả trong ứng dụng React. Trong HustFlow, thư viện này được dùng cho bảng Kanban, nơi người dùng cần di chuyển thẻ giữa các danh sách và sắp xếp lại thứ tự công việc. Đây là thao tác trung tâm của UC003 và là một phần quan trọng của yêu cầu tính dễ dùng ở phần 2.4.4.

Các lựa chọn thay thế gồm React DnD và dnd-kit. React DnD linh hoạt nhưng ở mức trừu tượng thấp, đòi hỏi tự xử lý nhiều chi tiết về vùng kéo, vùng thả và trạng thái giao diện. dnd-kit hiện đại và có khả năng tùy biến cao, nhưng cần nhiều cấu hình hơn cho bố cục Kanban gồm nhiều danh sách ngang và nhiều thẻ dọc. Trong

phạm vi HustFlow, @hello-pangea/dnd được lựa chọn vì cung cấp sẵn mô hình kéo thả danh sách phù hợp với Kanban, giúp giảm khối lượng cài đặt giao diện trong khi vẫn đáp ứng yêu cầu thao tác trực quan.

Chương này đã trình bày các nền tảng lý thuyết và công nghệ chính được sử dụng trong HustFlow. Về lý thuyết, đồ thị có hướng phi chu trình và thuật toán phát hiện chu trình được dùng để kiểm soát quan hệ phụ thuộc giữa các thẻ, còn mô hình cơ sở dữ liệu quan hệ được dùng để bảo đảm tính toàn vẹn dữ liệu. Về công nghệ, Next.js, React, MySQL, Prisma, Pusher, Clerk, Stripe, OpenAI API và @hello-pangea/dnd được lựa chọn vì tương ứng với các yêu cầu cụ thể đã phân tích ở Chương 2, đồng thời phù hợp với phạm vi triển khai của đề án. Các lựa chọn này là cơ sở để Chương 4 tiếp tục trình bày kiến trúc, thiết kế dữ liệu, luồng xử lý và kết quả triển khai hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã trình bày nền tảng lý thuyết và các công nghệ được lựa chọn để xây dựng HustFlow. Trên cơ sở đó, chương này cụ thể hóa các lựa chọn công nghệ thành kiến trúc, mô hình dữ liệu và các luồng xử lý của hệ thống. Nội dung chương gồm năm phần chính: thiết kế kiến trúc tại phần 4.1, thiết kế chi tiết tại phần 4.2, kết quả xây dựng ứng dụng tại phần 4.3, thiết kế và kết quả kiểm thử tại phần 4.4, và môi trường chạy thử nghiệm cục bộ tại phần 4.5. Các mô tả trong chương được đối chiếu với lược đồ Prisma, các Server Action, Route Handler và thành phần giao diện trong mã nguồn hiện tại.

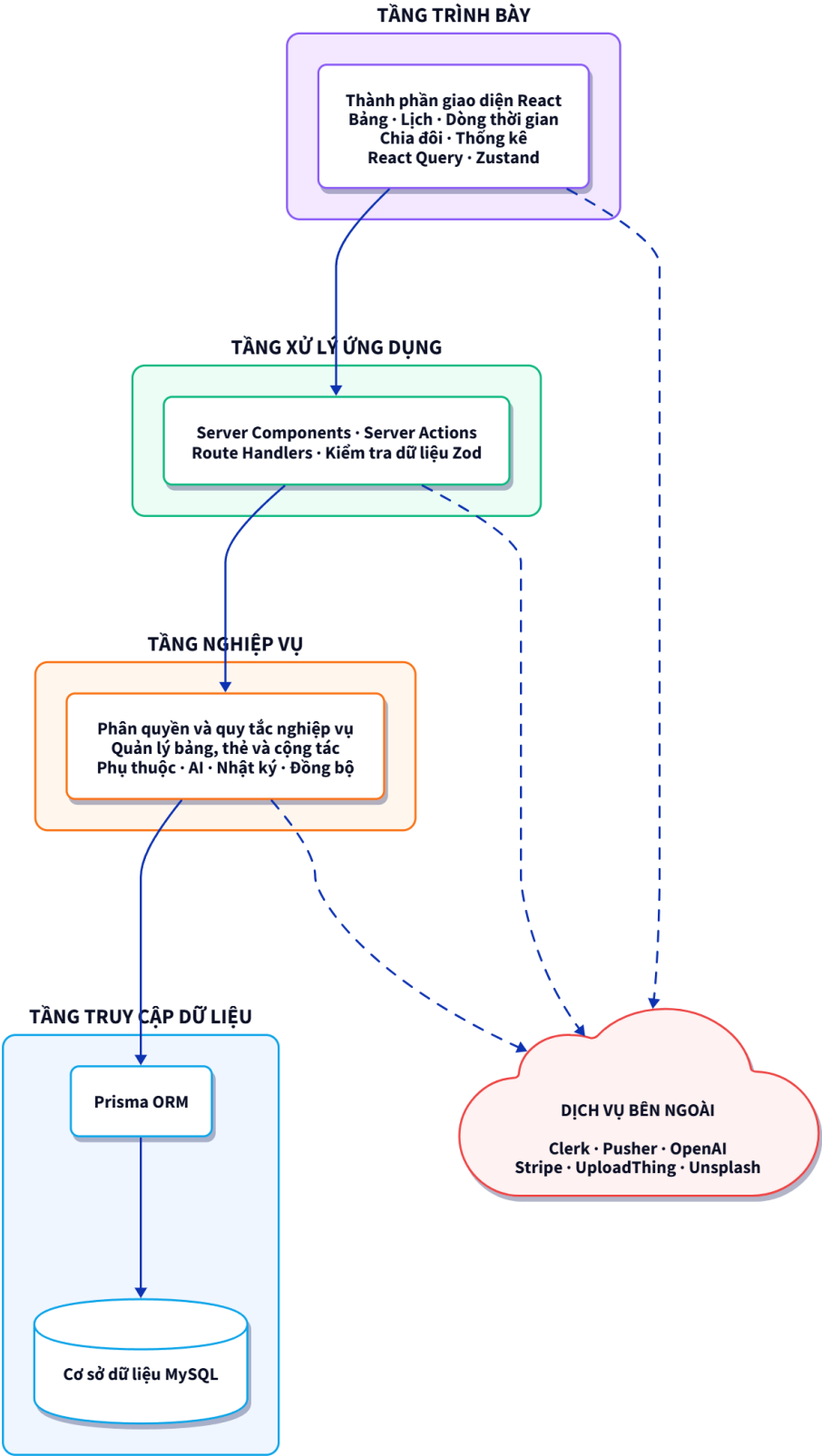
4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

HustFlow được xây dựng theo kiến trúc nguyên khối mô-đun trên nền tảng Next.js App Router. Toàn bộ chức năng được triển khai trong một ứng dụng, nhưng mã nguồn được chia thành các mô-đun theo miền nghiệp vụ và được tổ chức thành bốn tầng trách nhiệm: trình bày, xử lý ứng dụng, nghiệp vụ và truy cập dữ liệu. Kiến trúc phân tầng giúp phân tách giao diện, điều phối yêu cầu, quy tắc nghiệp vụ và cơ chế lưu trữ [16]. Trong HustFlow, các tầng biểu diễn sự phân chia trách nhiệm, không phải ràng buộc mọi lời gọi phải lần lượt đi qua đủ bốn tầng.

Kiến trúc này phù hợp với phạm vi đồ án vì các chức năng quản lý bảng, thẻ, thành viên và cộng tác có quan hệ dữ liệu chặt chẽ. Một ứng dụng thống nhất cho phép dùng chung kiểu dữ liệu, lược đồ Prisma và các mô-đun nghiệp vụ, đồng thời giảm số lượng thành phần phải triển khai và phối hợp. So với microservice, phương án này tránh yêu cầu vận hành nhiều dịch vụ và xử lý giao tiếp phân tán khi quy mô hệ thống chưa cần đến mức phân tách đó [17].

Kiến trúc tổng thể của HustFlow được trình bày trong Hình 4.1. Ngoài bốn tầng nội bộ, hình còn thể hiện nhóm dịch vụ bên ngoài được hệ thống sử dụng.



Hình 4.1: Kiến trúc tổng thể của hệ thống HustFlow

Các tầng trong Hình 4.1 được ánh xạ vào mã nguồn HustFlow như sau. Tầng trình bày gồm Client Component, React Component, hook và năm chế độ xem của bảng. Tầng xử lý ứng dụng gồm Server Component, 57 Server Action, Route Handler, webhook và cơ chế kiểm tra dữ liệu đầu vào. Tầng nghiệp vụ gồm các quy tắc và mô-đun về phân quyền, bảng-thẻ, phụ thuộc, AI, nhật ký và đồng bộ thời gian thực; các thành phần này được đặt trong *lib* hoặc được điều phối tại Server Action và Route Handler tương ứng. Tầng truy cập dữ liệu gồm lược đồ Prisma, Prisma Client, mô-đun *lib/db.ts* và MySQL.

Do đặc điểm full-stack của Next.js, mã nguồn không áp dụng chiều phụ thuộc tầng một cách nghiêm ngặt. Server Component có thể đọc dữ liệu ban đầu qua Prisma; Server Action, Route Handler và mô-đun nghiệp vụ phía máy chủ có thể đọc hoặc ghi dữ liệu sau khi kiểm tra phiên, quyền truy cập và dữ liệu cần thiết. Các Server Action sử dụng lược đồ Zod thông qua `createSafeAction`, còn Route Handler kiểm tra yêu cầu theo từng điểm cuối. Client Component không nhập Prisma và không truy cập trực tiếp MySQL.

Clerk, Pusher, OpenAI, Stripe, UploadThing và Unsplash là các dịch vụ bên ngoài được gọi qua SDK hoặc mô-đun tích hợp tương ứng. Các mũi tên nét đứt tới nhóm dịch vụ bên ngoài trong hình chỉ biểu diễn quan hệ tích hợp tổng quát; chúng không hàm ý mọi tầng đều sử dụng toàn bộ các dịch vụ được liệt kê.

4.1.2 Thiết kế tổng quan

Biểu đồ gói ở Hình 4.2 mô tả cách tổ chức các nhóm mã nguồn chính và chiều phụ thuộc giữa chúng. Mũi tên nét đứt biểu diễn quan hệ phụ thuộc, trong đó phần tử tại điểm xuất phát sử dụng thành phần do phần tử mà đầu mũi tên hướng tới cung cấp. Chiều phụ thuộc kiến trúc chủ đạo đi từ tầng trình bày tới tầng xử lý ứng dụng, tầng nghiệp vụ và tầng truy cập dữ liệu. Các package ở tầng dưới không phụ thuộc ngược vào giao diện hoặc trang cụ thể; các contract dùng chung được đặt tại tầng nghiệp vụ để tránh phụ thuộc vòng giữa *actions* và *lib*. Bên cạnh package, biểu đồ sử dụng stereotype để phân biệt mô-đun mã nguồn, lược đồ, client được sinh, cơ sở dữ liệu và dịch vụ ngoài.

lib/boards, mô-đun phân quyền cùng *package types*. Package *lib/cards* chứa contract, quy tắc ngày, thông điệp nhật ký và service của nghiệp vụ thẻ. Các schema và kiểu đầu vào dùng chung được đặt tại đây để *actions* có thể sử dụng mà *lib/cards* không phải phụ thuộc ngược vào tầng Server Action. Ngoài hai package được vẽ, các thư mục *checklists*, *comments*, *notifications* và *analytics* cũng đóng gói logic theo từng miền nhưng được lược bỏ để hình không quá phức tạp. Mô-đun *permissions* đại diện cho *permissions.ts* và *organization-permissions.ts*, thực hiện kiểm tra quyền ở cấp bảng và tổ chức. Package *types* mô tả dữ liệu dùng chung giữa API, giao diện và các mô-đun nghiệp vụ.

Tầng truy cập dữ liệu gồm mô-đun *lib/db.ts*, lược đồ *prisma/schema.prisma*, Prisma Client được sinh tự động và MySQL. Lược đồ Prisma định nghĩa các model và sinh mã Prisma Client trong quá trình generate. Mô-đun *lib/db.ts* nhập, khởi tạo và tái sử dụng một instance Prisma Client; client này thực hiện truy vấn tới MySQL. Các mũi tên “sinh mã”, “sử dụng” và “truy vấn” trong hình lần lượt biểu diễn ba quan hệ trên, không hàm ý *lib/db.ts* đọc trực tiếp tệp schema khi hệ thống đang chạy.

HustFlow có một số đường phụ thuộc trực tiếp tới *lib/db.ts* do đặc điểm full-stack của Next.js. Server Component trong *app* có thể đọc dữ liệu ban đầu; Server Action, Route Handler và các mô-đun nghiệp vụ trong *lib* có thể đọc hoặc ghi dữ liệu. Đây là các đường truy cập phía máy chủ, không phải việc Client Component bỏ qua các tầng kiểm soát. Client Component không được nhập hoặc sử dụng Prisma trực tiếp, còn mọi thao tác với dữ liệu nhạy cảm vẫn phải kiểm tra quyền trước khi truy vấn hoặc thay đổi dữ liệu.

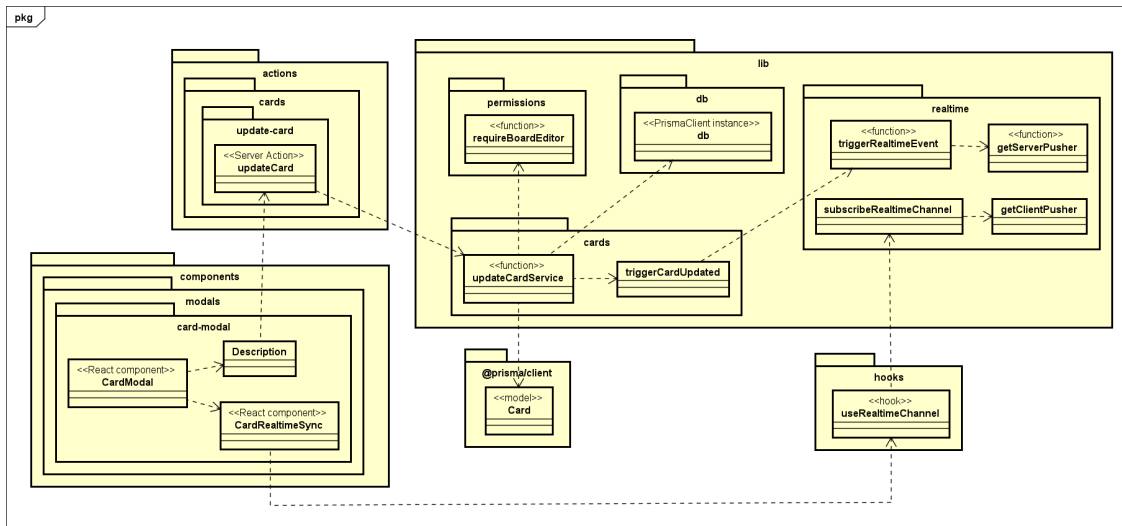
Các mũi tên tới nhóm dịch vụ bên ngoài biểu diễn quan hệ tích hợp tổng quát, không có nghĩa mọi package trong một tầng đều sử dụng tất cả dịch vụ được liệt kê. Clerk cung cấp xác thực và quản lý tổ chức; Pusher hỗ trợ đồng bộ thời gian thực; OpenAI cung cấp chức năng AI; Stripe xử lý thanh toán; UploadThing lưu trữ tệp; Unsplash cung cấp ảnh nền. Tầng trình bày, tầng xử lý ứng dụng và tầng nghiệp vụ sử dụng các dịch vụ phù hợp thông qua SDK hoặc mô-đun tích hợp tương ứng.

Như vậy, thiết kế duy trì chiều phụ thuộc chính từ trên xuống, không tạo phụ thuộc vòng giữa các package và không để tầng dữ liệu gọi ngược lên tầng trên. Các đường truy cập trực tiếp tới *lib/db.ts* được thể hiện riêng để phản ánh đúng mã nguồn, nhưng vẫn tuân thủ ranh giới máy khách–máy chủ và quy tắc phân quyền.

Các bất biến phân quyền ở cấp bảng, quyền quản trị tổ chức và mối liên hệ với giới hạn dịch vụ được kiểm tra tại các Server Action và mô-đun quyền tương ứng.

4.1.3 Thiết kế chi tiết gói quản lý thẻ và cộng tác

Thẻ công việc là thực thể trung tâm của HustFlow. Hình 4.3 tập trung mô tả luồng cập nhật nội dung thẻ và đồng bộ thay đổi theo thời gian thực giữa các thành viên. Biểu đồ chỉ trình bày tên các thành phần chủ đạo và lược bỏ thuộc tính, tham số cùng chi tiết cài đặt để bảo đảm tính rõ ràng. Do hệ thống được xây dựng chủ yếu theo hướng hàm và React Component, stereotype được sử dụng để phân biệt React Component, Server Action, hook, function, Prisma Client instance và model dữ liệu.



Hình 4.3: Thiết kế chi tiết nhóm gói quản lý thẻ và cộng tác

Trong nhóm gói *components/modals/card-modal*, CardModal hiển thị thành phần Description và sử dụng CardRealtimeSync. Khi người dùng lưu mô tả, Description gọi Server Action updateCard trong nhóm gói *actions/cards/update-card*. Action xác thực phiên làm việc, kiểm tra dữ liệu đầu vào và chuyển yêu cầu tới hàm updateCardService thuộc *lib/cards*.

updateCardService gọi requireBoardEditor trong *lib/permissions* để kiểm tra quyền chỉnh sửa bảng. Sau khi được cấp quyền, service sử dụng instance db của Prisma Client trong *lib/db* để đọc và cập nhật model Card do @prisma/client cung cấp. Khi cập nhật thành công, service gọi triggerCardUpdated để tạo và phát thông báo về thay đổi của thẻ.

Ở phía máy chủ, triggerCardUpdated gọi triggerRealtimeEvent. Hàm này lấy Pusher server instance thông qua getServerPusher rồi phát sự kiện tới kênh riêng của bảng và kênh riêng của thẻ. Ở phía trình duyệt, CardRealtimeSync sử dụng hook useRealtimeChannel; hook gọi subscribeRealtimeChannel, sau đó hàm này sử dụng Pusher client instance do getClientPusher cung cấp để đăng ký kênh thời gian thực. Khi nhận sự kiện do thành viên khác thực hiện, CardRealtimeSync vô

hiệu hóa các truy vấn liên quan để giao diện tải lại dữ liệu mới nhất.

Các mũi tên nét đứt trong biểu đồ biểu diễn quan hệ phụ thuộc, trong đó thành phần ở điểm xuất phát sử dụng hoặc gọi thành phần mà mũi tên hướng tới. Biểu đồ không sử dụng quan hệ kế thừa, thực thi, kết tập hoặc hợp thành vì các quan hệ này không phát sinh trong phạm vi luồng cập nhật và đồng bộ được lựa chọn. Việc chỉ biểu diễn các quan hệ thực sự tồn tại giúp thiết kế phản ánh đúng mã nguồn và tránh làm biểu đồ quá phức tạp.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

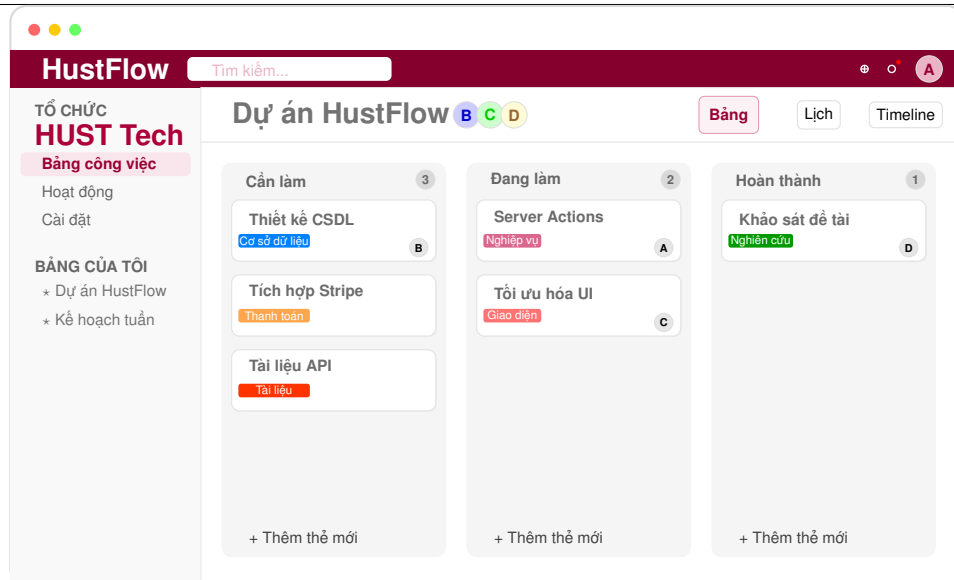
HustFlow là ứng dụng quản lý công việc có mật độ thông tin cao, do đó giao diện được thiết kế theo định hướng ưu tiên máy tính xách tay và máy tính để bàn. Các chế độ Kanban, Lịch, Dòng thời gian và Chia đôi cần không gian ngang để người dùng quan sát đồng thời nhiều nhóm công việc. Thiết bị di động vẫn được hỗ trợ cho các tác vụ xem nhanh, cập nhật thẻ và theo dõi thông báo, nhưng không được xem là môi trường tối ưu cho hoạt động lập kế hoạch tổng thể.

a, Môi trường hiển thị mục tiêu

Độ phân giải tối thiểu của môi trường làm việc chính là 1280×720 pixel; độ phân giải khuyến nghị là Full HD 1920×1080 pixel. Màn hình có kích thước vật lý khoảng 13–24 inch và tỷ lệ 16:9 hoặc 16:10 phù hợp với bố cục nhiều cột. Hệ thống không có yêu cầu xử lý đồ họa chuyên dụng; độ sâu màu 24 bit, tương ứng khoảng 16,7 triệu màu, đủ để hiển thị nền tảng, nhãn, trạng thái và biểu đồ. Chuột và bàn phím là phương thức nhập chính, đồng thời các điều khiển cơ bản vẫn hỗ trợ thao tác chạm. Trình duyệt cần hỗ trợ JavaScript, CSS Grid, Flexbox và kết nối HTTPS.

Giao diện được tổ chức theo hai phạm vi điều hướng. Ở phạm vi tổ chức, thanh điều hướng trên cùng cung cấp truy cập tới tìm kiếm, thông báo và tài khoản; thanh bên chứa danh sách tổ chức và các chức năng quản trị. Ở phạm vi bảng, thanh điều hướng thứ hai hiển thị tên bảng, thành viên, bộ lọc và bộ chuyển đổi chế độ xem. Cấu trúc này giúp người dùng phân biệt thao tác ảnh hưởng toàn tổ chức với thao tác chỉ tác động đến bảng hiện tại.

Hình 4.4 mô tả khung bố cục của màn hình làm việc chính. Đây là bản phác thảo mức độ chi tiết thấp nhằm thể hiện thứ bậc thông tin và vị trí điều khiển, không phải ảnh chụp giao diện của sản phẩm sau khi cài đặt. Vùng nội dung trung tâm có thể thay bằng Kanban, Lịch, Dòng thời gian, Chia đôi hoặc Thống kê mà không làm thay đổi ngữ cảnh điều hướng.



Hình 4.4: Bản phác thảo bố cục không gian làm việc chính

b, Các quy ước thiết kế

Các điều khiển được chuẩn hóa theo mục đích và tần suất sử dụng. Nút hành động chính dùng màu tím nhận diện và chỉ xuất hiện một lần trong mỗi cụm thao tác; nút phụ dùng nền trung tính; hành động phá hủy dùng màu đỏ và yêu cầu xác nhận. Nút trên thanh công cụ có chiều cao khoảng 32 px để bảo đảm mật độ thông tin, trong khi điều khiển chính trong biểu mẫu và modal có chiều cao khoảng 40–44 px. Cỡ chữ nội dung phổ biến nằm trong khoảng 13–16 px; tiêu đề, nhãn phụ và metadata được phân cấp bằng kích thước, độ đậm và độ tương phản thay vì sử dụng quá nhiều màu.

Màu tím được dùng cho hành động chính và thành phần nhận diện; nhóm màu trung tính được dùng cho nền, đường viền và nội dung phụ. Màu đỏ chỉ biểu thị lỗi, dữ liệu quá hạn hoặc thao tác nguy hiểm. Trạng thái không chỉ được truyền đạt bằng màu sắc mà còn bằng nhãn, biểu tượng hoặc thông điệp văn bản. Mọi thành phần tương tác cần có trạng thái hover và focus để nhận biết; thứ tự focus được bố trí theo chiều đọc để hỗ trợ thao tác bàn phím.

Phản hồi thao tác được đặt theo phạm vi của vấn đề. Lỗi kiểm tra dữ liệu xuất hiện ngay dưới trường nhập tương ứng. Lỗi nghiệp vụ ở mức biểu mẫu được hiển thị trong biểu mẫu hoặc modal. Kết quả thành công, lỗi hệ thống và thông báo không gắn với một trường cụ thể được hiển thị bằng toast ở vùng ít che khuất nội dung. Trong thời gian xử lý, nút gửi bị vô hiệu hóa và hiển thị trạng thái chờ; vùng dữ liệu đang tải sử dụng skeleton. Những thao tác xóa hoặc lưu trữ dữ liệu phải qua hộp thoại xác nhận.

Quyền thao tác cũng được phản ánh trong thiết kế. Vai trò **ADMIN** có thể quản

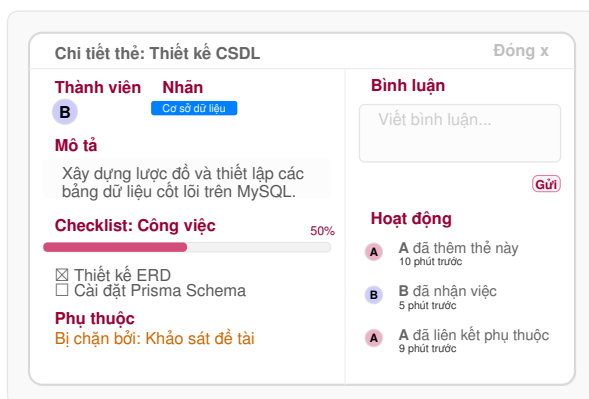
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

lý bảng và thành viên; **MEMBER** có thể tạo, sửa và sắp xếp công việc; **VIEWER** chỉ theo dõi tiến độ. Các điều khiển cập nhật và trợ lý AI không xuất hiện với người dùng chỉ có quyền xem. Việc ẩn điều khiển chỉ nhằm làm rõ khả năng thao tác; mọi yêu cầu thay đổi dữ liệu vẫn được kiểm tra quyền ở phía máy chủ.

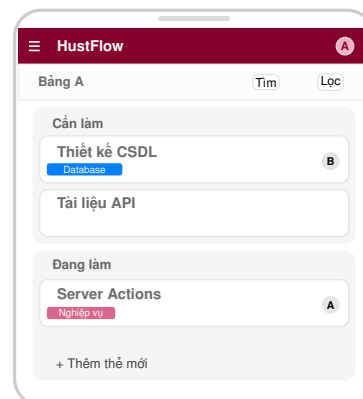
c, Thiết kế thích ứng và các màn hình quan trọng

Thiết kế thích ứng sử dụng các mốc chiều rộng của Tailwind CSS. Từ breakpoint md, thanh bên tổ chức rộng 256 px được hiển thị cố định; ở màn hình nhỏ, thanh bên được thay bằng nút mở menu. Danh sách bảng thay đổi từ hai cột lên ba cột tại sm và bốn cột tại lg. Tên chế độ xem được ẩn trên màn hình hẹp và chỉ giữ biểu tượng; từ lg, cả biểu tượng và nhãn được hiển thị. Modal chi tiết thẻ dùng một cột trên màn hình nhỏ và lưới 12 cột từ md, trong đó nội dung thẻ chiếm bảy cột, hoạt động và bình luận chiếm năm cột.

Hình 4.5 minh họa hai tình huống quan trọng: cập nhật chi tiết thẻ trên desktop và theo dõi bảng trên điện thoại. Các hình chỉ mô tả bố cục dự kiến, vùng thông tin và thứ tự thao tác; màu sắc, nội dung mẫu và tỷ lệ thành phần không được xem là kết quả giao diện cuối cùng.



(a) Modal chi tiết thẻ trên desktop



(b) Bảng thu gọn trên điện thoại

Hình 4.5: Bản phác thảo giao diện cho các tình huống sử dụng quan trọng

Trên desktop, modal chi tiết thẻ ưu tiên nội dung công việc ở vùng bên trái và hoạt động cộng tác ở vùng bên phải, giúp người dùng vừa chỉnh sửa dữ liệu vừa theo dõi lịch sử trao đổi. Trên điện thoại, các điều khiển ít dùng được thu vào menu, danh sách chuyển sang bố cục dọc và nội dung được cuộn theo một trục. Các chế độ cần quan sát tổng thể như Dòng thời gian hoặc Chia đôi vẫn truy cập được nhưng phù hợp hơn với màn hình lớn.

4.2.2 Thiết kế lớp

Mã nguồn HustFlow sử dụng TypeScript theo phong cách hàm và mô-đun thay vì cài đặt toàn bộ nghiệp vụ bằng từ khóa class. Tuy nhiên, ở mức thiết kế hướng

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

đối tượng, thuộc tính và hành vi vẫn được phân chia theo trách nhiệm. Phần này lựa chọn bốn lớp chủ đạo gồm Board, Card, BoardMember và CardApplicationService. Ba lớp đầu là các lớp thực thể được ánh xạ thành model Prisma; lớp CardApplicationService đóng gói các ca sử dụng liên quan đến thẻ. Khi cài đặt, phương thức của lớp dịch vụ được tách thành các hàm và Server Action để phù hợp với mô hình thực thi của Next.js. Cách ánh xạ này không làm thay đổi trách nhiệm và hợp đồng đã xác định trong thiết kế.

a, Thuộc tính của các lớp chủ đạo

Board biểu diễn phạm vi cộng tác của một tổ chức, Card biểu diễn đơn vị công việc và BoardMember biểu diễn tư cách cùng vai trò của người dùng trong bảng. Các thuộc tính lưu trữ của ba lớp thực thể được trình bày trong Bảng 4.1. Thuộc tính của CardApplicationService là các phụ thuộc db, permissionService, auditLogger và realtimePublisher; trong mã nguồn, các phụ thuộc này được cung cấp qua các mô-đun dùng chung thay vì được giữ trong một đối tượng dịch vụ. Các trường quan hệ của lớp thực thể được ánh xạ thành khóa ngoại hoặc bảng nối ở phần 4.2.3.

Bảng 4.1: Thiết kế thuộc tính của các lớp thực thể chủ đạo

Lớp	Thuộc tính	Kiểu Prisma	Ý nghĩa
Board	id	String	Khóa định danh UUID của bảng.
Board	orgId	String	Định danh tổ chức Clerk sở hữu bảng.
Board	title	String	Tên hiển thị của bảng.
Board	imageId	String	Định danh ảnh nền từ nguồn ảnh.
Board	imageThumb Url	String @db.Text	URL ảnh thu nhỏ dùng ở danh sách bảng.
Board	imageFullUrl	String @db.Text	URL ảnh đầy đủ dùng làm nền bảng.
Board	imageUser Name	String @db.Text	Tên tác giả ảnh để ghi nguồn.
Board	imageLink HTML	String @db.Text	Liên kết ghi nhận nguồn ảnh.
Board	createdAt	DateTime	Thời điểm tạo, mặc định là thời gian hiện tại.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Lớp	Thuộc tính	Kiểu Prisma	Ý nghĩa
Board	updatedAt	DateTime	Thời điểm cập nhật gần nhất do Prisma duy trì.
Card	id	String	Khóa định danh UUID của thẻ.
Card	title	String	Tiêu đề công việc.
Card	order	Int	Vị trí của thẻ trong danh sách.
Card	description	String? @db.Text	Mô tả Markdown, có thể để trống.
Card	description UpdatedAt	DateTime	Mốc phiên bản dùng để phát hiện ghi đè mô tả.
Card	startDate, dueDate	DateTime?	Khoảng thời gian dự kiến của công việc.
Card	isCompleted	Boolean	Trạng thái hoàn thành, mặc định là false.
Card	reminder	String?	Quy tắc nhắc việc được người dùng lựa chọn.
Card	reminder SetAt	DateTime?	Thời điểm cấu hình nhắc việc gần nhất.
Card	archivedAt	DateTime?	Thời điểm lưu trữ mềm của thẻ.
Card	archivedBy ListId	String?	Danh sách nguồn khi thẻ bị lưu trữ cùng danh sách.
Card	listId	String	Khóa ngoại tới danh sách chứa thẻ.
Card	createdAt, updatedAt	DateTime	Thời điểm tạo và cập nhật gần nhất.
Board Member	id	String	Khóa định danh UUID của quan hệ thành viên bảng.
Board Member	boardId	String	Khóa ngoại tới bảng.
Board Member	userId	String	Định danh người dùng Clerk.
Board Member	userName	String @db.Text	Tên được chụp lại để hiển thị trong bảng.
Board Member	userImage	String @db.Text	URL ảnh đại diện.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Lớp	Thuộc tính	Kiểu Prisma	Ý nghĩa
Board Member	userEmail	String? @db.Text	Thư điện tử dùng cho tìm kiếm và gợi ý gán việc.
Board Member	role	BoardMember Role	Vai trò ADMIN, MEMBER hoặc VIEWER.
Board Member	createdAt, updatedAt	DateTime	Thời điểm tạo và cập nhật quan hệ thành viên.

Board có quan hệ một-nhiều với List, BoardMember và Label. Card thuộc một List và có các tập quan hệ người phụ trách, bình luận, nhãn, checklist, tệp đính kèm cùng hai chiều phụ thuộc. BoardMember liên kết với thẻ qua CardAssignee và có thể được gán cho nhiều ChecklistItem. Các quan hệ tập hợp này không tạo cột trực tiếp trong bảng chủ mà được Prisma ánh xạ qua khóa ngoại hoặc bảng nối.

b, Phương thức và trách nhiệm của các lớp

Các phương thức ở mức thiết kế được trình bày trong Bảng 4.2. Do các lớp thực thể Prisma chủ yếu lưu trạng thái, phương thức làm thay đổi dữ liệu được điều phối bởi CardApplicationService. Các phương thức kiểm tra trạng thái đơn giản vẫn được gắn với lớp sở hữu thông tin tương ứng. Trong mã nguồn, các phương thức này được hiện thực bằng hàm thuần, service hoặc Server Action; kiểu Action-State<TInput,TOutput> thống nhất kết quả với các trường fieldErrors, error, errorCode và data.

Bảng 4.2: Phương thức và trách nhiệm của bốn lớp chủ đạo

Lớp	Phương thức thiết kế	Đầu vào/dầu ra	Trách nhiệm và điều kiện chính
Board	containsCard hasActiveList	Nhận định danh thẻ hoặc danh sách; trả giá trị đúng/sai.	Xác định thẻ hoặc danh sách còn hoạt động có thuộc đúng bảng và tổ chức hay không. Khi cài đặt, điều kiện này được đưa vào truy vấn Prisma.
Card	validateDate Range canUpdate Description isBlocked	Sử dụng trạng thái thẻ; trả kết quả kiểm tra hoặc mã lỗi.	Bảo đảm ngày bắt đầu không sau hạn hoàn thành, phát hiện xung đột phiên bản mô tả và xác định các phụ thuộc chưa hoàn thành.
BoardMember	canView canEdit canAdminister	Sử dụng thuộc tính role; trả giá trị đúng/sai.	Ánh xạ ADMIN, MEMBER, VIEWER thành quyền xem, chỉnh sửa và quản trị; không tin cậy vai trò do máy khách gửi lên.
Card Application Service	updateCard moveCards create Dependency analyzeSmart Capture	Nhận lệnh, định danh người dùng và tổ chức; trả ActionState hoặc bản nháp.	Kiểm tra lược đồ và quyền, phối hợp các lớp thực thể, ghi dữ liệu khi cần, tạo nhật ký và phát sự kiện thời gian thực. Phương thức analyzeSmart-Capture chỉ tạo bản nháp, không tự ghi dữ liệu.

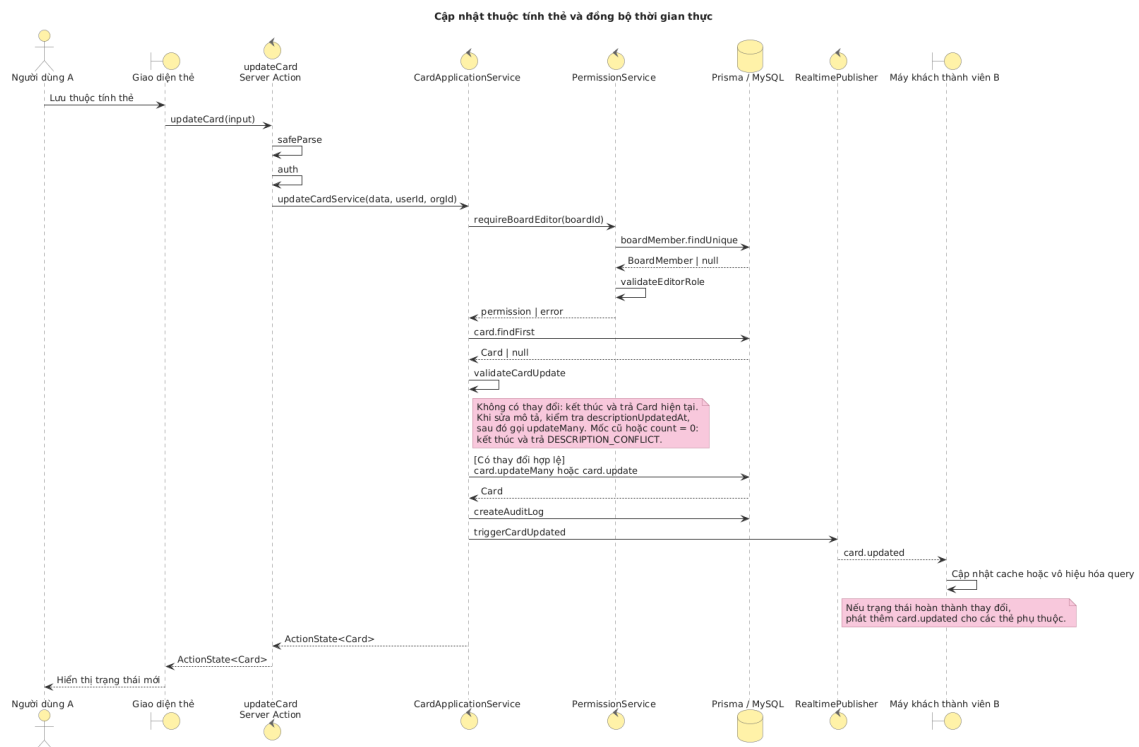
Trong ánh xạ cài đặt, CardApplicationService.updateCard() tương ứng với updateCardService; createDependency() tương ứng với Server Action createCardDependency; và analyzeSmartCapture() tương ứng với Server Action cùng tên. Các phương thức quyền của BoardMember được hiện thực qua nhóm hàm requireBoardMember, requireBoardEditor và requireBoardAdmin. Dữ liệu đầu vào được createSafeAction kiểm tra bằng lược đồ Zod trước khi chuyển tới hàm xử lý.

Định danh người dùng và tổ chức được lấy từ phiên Clerk ở phía máy chủ, không lấy từ giá trị quyền do máy khách tự khai báo. Dữ liệu bảng hoặc thẻ sau đó được truy vấn kèm điều kiện orgId, boardId và trạng thái lưu trữ. Quy tắc này vừa duy trì bất biến của các lớp, vừa giảm nguy cơ truy cập chéo tổ chức khi một định danh hợp lệ bị đưa vào yêu cầu không thuộc phạm vi hiện tại.

c, Luồng cập nhật thuộc tính và đồng bộ thẻ

Hình 4.6 mô tả luồng cập nhật thuộc tính của thẻ và đồng bộ kết quả tới các thành viên đang mở cùng bảng. Luồng kéo thả và sắp xếp thẻ sử dụng Server Action riêng nên không được gộp vào biểu đồ này.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.6: Biểu đồ trình tự cập nhật thuộc tính thẻ và đồng bộ thời gian thực

Giao diện gửi dữ liệu tới Server Action `updateCard`. Hàm `createSafeAction` kiểm tra đầu vào bằng lược đồ Zod trước khi handler lấy `userId` và `orgId` từ phiên Clerk. Handler chuyển yêu cầu tới `updateCardService`; service kiểm tra quyền chỉnh sửa, tải trạng thái hiện tại, xác định các trường thay đổi và áp dụng quy tắc về khoảng ngày, nhắc việc cùng phiên bản mô tả. Nếu không có trường nào thay đổi, service trả ngay dữ liệu hiện tại. Khi mô tả thay đổi, service kiểm tra mốc `descriptionUpdatedAt`, sau đó dùng `updateMany` kèm điều kiện phiên bản để phát hiện cập nhật đồng thời; mốc phiên bản cũ hoặc không có bản ghi được cập nhật đều dẫn tới lỗi `DESCRIPTION_CONFLICT`. Các thuộc tính khác được cập nhật bằng `update`. Sau cập nhật, các thông báo nhắc việc không còn hợp lệ được xóa khi cần. Thao tác này không nằm trong một giao dịch Prisma bao gồm cả nhật ký. Sau khi cập nhật thành công, hệ thống ghi nhật ký và phát sự kiện `card.updated` tới kênh bảng và kênh thẻ. Máy khách nhận sự kiện sẽ cập nhật cache hoặc vô hiệu hóa các query liên quan. Riêng khi trạng thái hoàn thành thay đổi, service truy vấn các quan hệ phụ thuộc và phát thêm sự kiện cập nhật cho các thẻ liên quan.

Đối với kéo thả, giao diện gọi Server Action `updateCardOrder`, không gọi `updateCard`. Action này nhận danh sách thẻ cùng `listId` và `order` mới, kiểm tra các danh sách đích và thẻ đều thuộc cùng một bảng, sau đó cập nhật các thẻ thay đổi trong một giao dịch Prisma. Nếu có thẻ chuyển sang danh sách khác, hệ thống phát `card.moved`; nếu chỉ thay đổi thứ tự, hệ thống phát `card.reordered`. Việc tách riêng

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

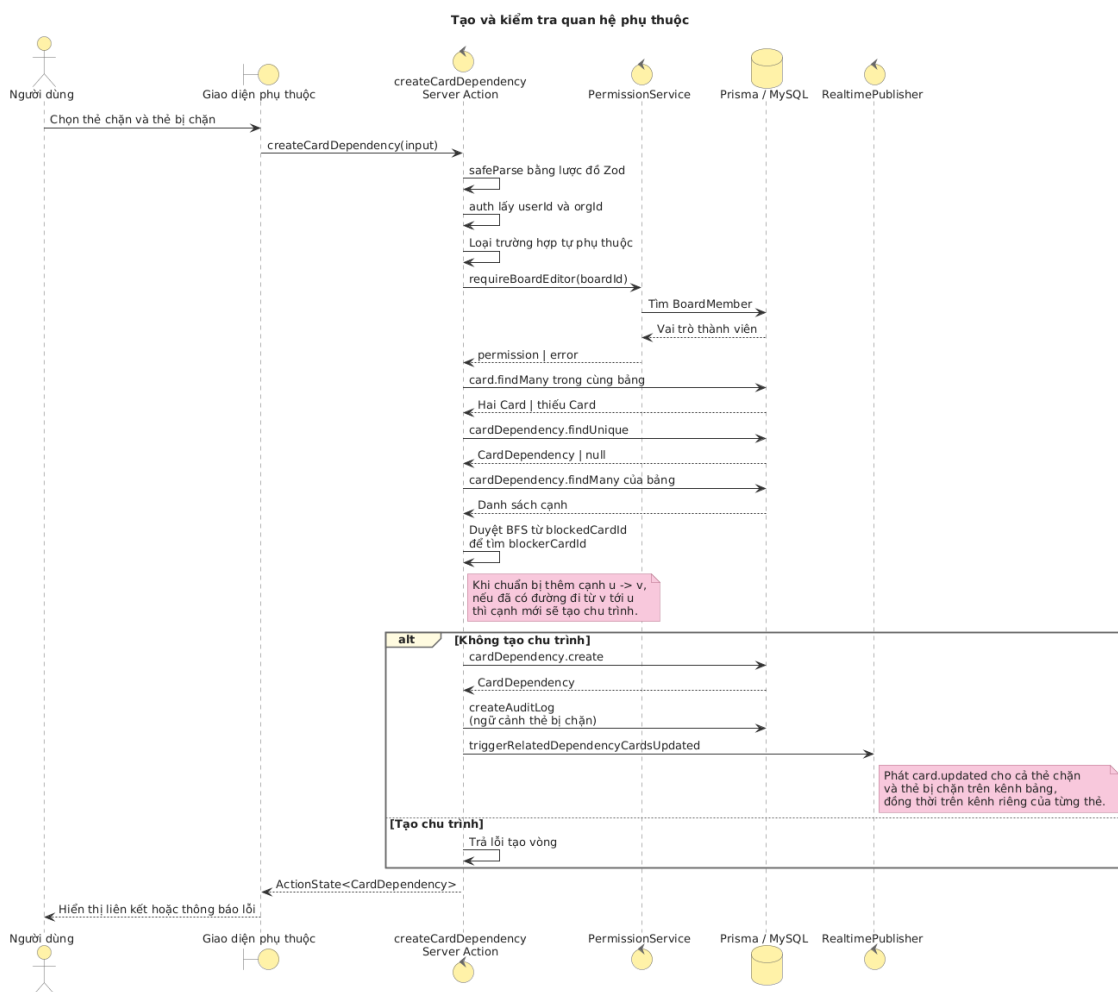
hai luồng phản ánh đúng trách nhiệm của từng Server Action và giúp máy khách xử lý đúng phạm vi thay đổi.

Kênh Pusher là kênh riêng tư. Điểm cuối `/api/pusher/auth` kiểm tra người dùng trước khi cấp quyền đăng ký kênh người dùng, tổ chức, bảng hoặc thẻ. Với kênh bảng, người dùng phải tồn tại trong BoardMember; với kênh thẻ, thẻ phải thuộc một bảng mà người dùng đang là thành viên. Do đó, tên kênh không đủ để truy cập dữ liệu thời gian thực.

Chiến lược cập nhật gia tăng, loại sự kiện trùng và phục hồi khi payload hoặc kết nối không đầy đủ được trình bày tại phần 5.1.

d, Luồng kiểm soát quan hệ phụ thuộc

Quan hệ phụ thuộc được biểu diễn bởi CardDependency, trong đó blockerCardId là công việc phải hoàn thành trước và blockedCardId là công việc bị chặn. Trong cài đặt hiện tại, toàn bộ luồng kiểm tra và tạo quan hệ được điều phối trực tiếp bởi Server Action `createCardDependency`, thay vì qua một service riêng. Luồng tạo liên kết được trình bày trong Hình 4.7.



Hình 4.7: Biểu đồ trình tự tạo và kiểm tra quan hệ phụ thuộc

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

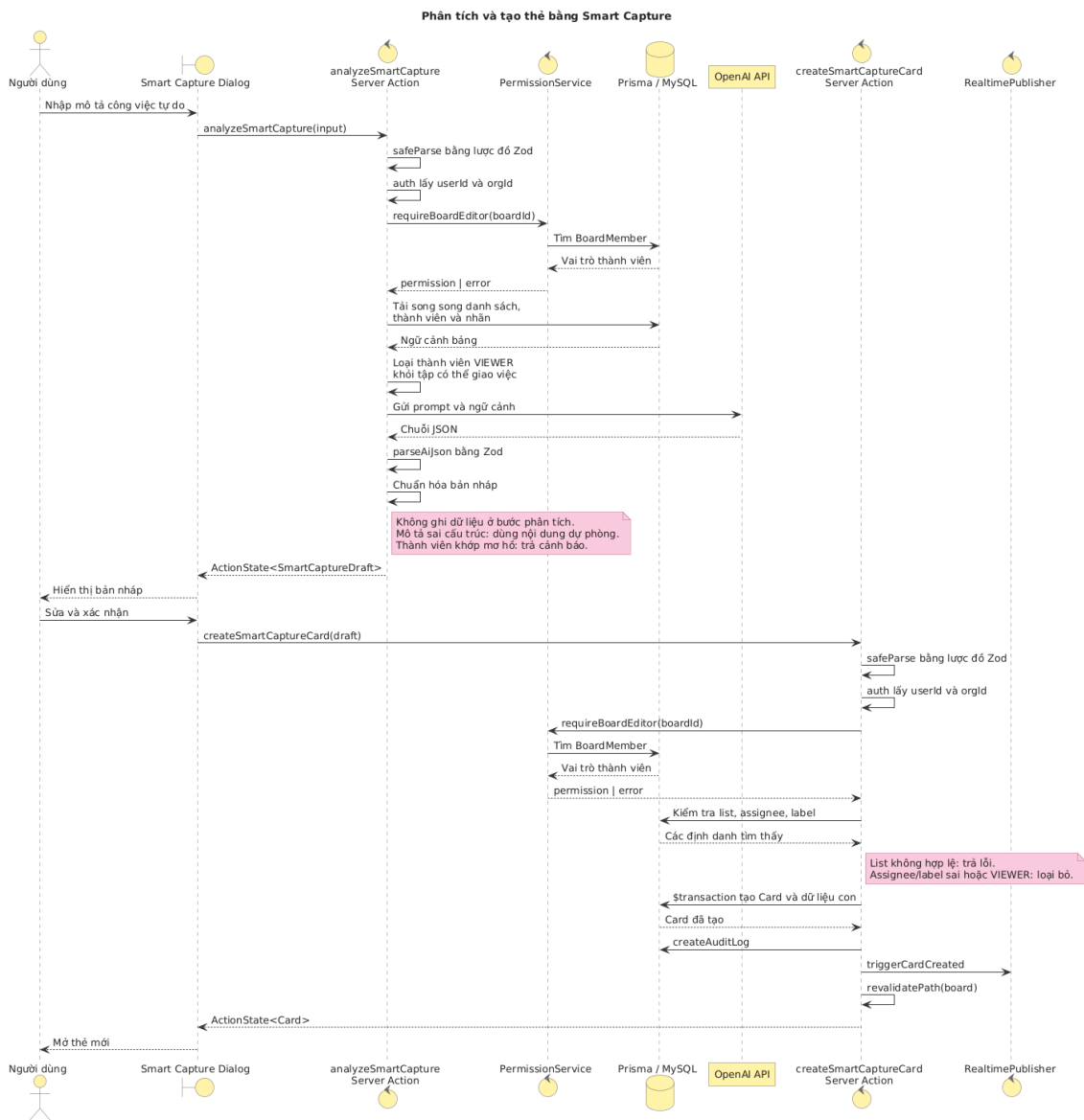
Sau khi dữ liệu đầu vào được kiểm tra bằng Zod và phiên Clerk được xác thực, máy chủ loại trường hợp một thẻ phụ thuộc vào chính nó, kiểm tra quyền chỉnh sửa và truy vấn hai thẻ trong phạm vi bảng hiện tại. Hai thẻ chỉ được chấp nhận khi chưa bị lưu trữ và không thuộc danh sách đã lưu trữ. Trước khi ghi, Server Action tiếp tục kiểm tra cạnh trùng và xác định cạnh mới có tạo chu trình trong đồ thị phụ thuộc hay không.

Khi quan hệ hợp lệ được tạo, hệ thống ghi nhật ký dưới ngữ cảnh thẻ bị chặn và phát sự kiện cập nhật cho hai thẻ liên quan. Khi trạng thái hoàn thành của một thẻ thay đổi, các thẻ có quan hệ với nó cũng được yêu cầu làm mới để giao diện phản ánh dữ liệu mới. Cách ánh xạ đồ thị, thuật toán kiểm tra chu trình, kết quả và giới hạn xử lý đồng thời của giải pháp được phân tích tại phần 5.2.

e, Luồng Smart Capture

Smart Capture chuyển văn bản tự do thành bản nháp thẻ có cấu trúc. Trong cài đặt hiện tại, hai giai đoạn được điều phối bởi hai Server Action `analyzeSmartCapture` và `createSmartCaptureCard`, không đi qua một `CardApplicationService` riêng. Luồng xử lý được trình bày trong Hình 4.8.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.8: Biểu đồ trình tự phân tích và tạo thẻ bằng Smart Capture

Ở giai đoạn phân tích, máy chủ kiểm tra quyền, chuẩn bị ngữ cảnh của bảng và gửi văn bản đầu vào tới mô hình OpenAI. Phản hồi được kiểm tra trước khi hiển thị thành bản nháp có cấu trúc. Người dùng có thể chỉnh sửa bản nháp; chỉ sau khi xác nhận, createSmartCaptureCard mới kiểm tra lại dữ liệu và tạo thẻ cùng các quan hệ liên quan.

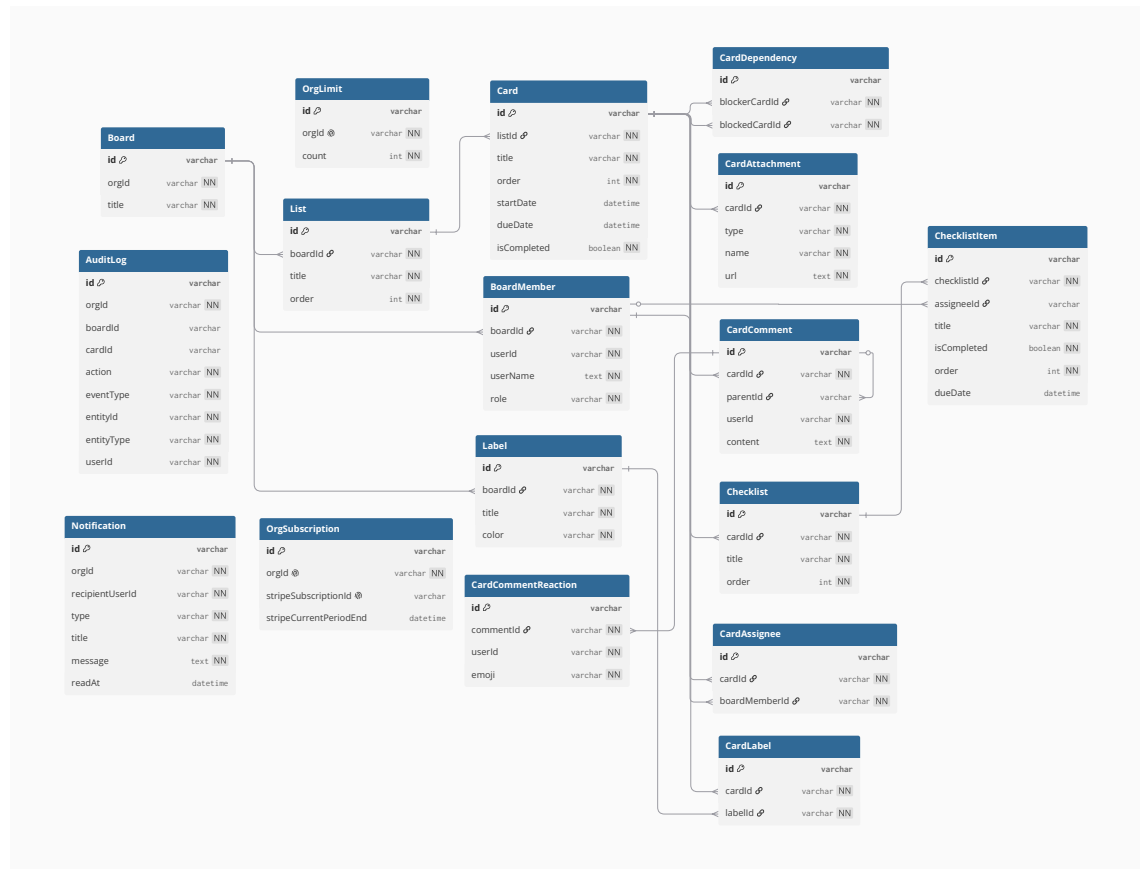
Kết quả triển khai cho phép dữ liệu do mô hình sinh ra đi qua quy trình “phân tích–xác nhận” thay vì được ghi trực tiếp vào cơ sở dữ liệu. Các lớp kiểm soát đầu ra, cách xử lý dữ liệu mơ hồ và nguyên tắc giới hạn vai trò của AI được phân tích tại phần 5.3.

4.2.3 Thiết kế cơ sở dữ liệu

Lược đồ dữ liệu của HustFlow gồm 17 mô hình dữ liệu (model) Prisma và được ánh xạ sang MySQL. Hình 4.9 trình bày các thực thể, khóa chính, khóa ngoại và

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

lượng quan hệ. Để sơ đồ có thể đọc rõ khi in, hình chỉ giữ các thuộc tính định danh, thuộc tính nghiệp vụ quan trọng và các ràng buộc duy nhất; các trường nội dung dài và trường thời gian được lược bớt.



Hình 4.9: Sơ đồ thực thể liên kết của cơ sở dữ liệu HustFlow

Nhóm quản lý công việc gồm Board, List, Card và CardDependency. Một bảng chứa nhiều danh sách; một danh sách chứa nhiều thẻ. CardDependency có hai khóa ngoại cùng trỏ tới Card nhưng sử dụng hai tên quan hệ khác nhau để biểu diễn hướng chặn. Các chỉ mục theo boardId, listId, trạng thái lưu trữ và thứ tự hỗ trợ các truy vấn hiển thị Kanban.

Nhóm phân quyền và phân loại gồm BoardMember, CardAssignee, Label và CardLabel. Hai bảng nối CardAssignee và CardLabel triển khai quan hệ nhiều-nhiều. Ràng buộc duy nhất theo cặp khóa ngăn cùng một thành viên hoặc nhãn được gắn lắp vào một thẻ. Thông tin tổ chức và người dùng Clerk được lưu dưới dạng định danh chuỗi thay vì khóa ngoại, vì nguồn dữ liệu chính của chúng nằm ngoài MySQL của ứng dụng.

Nhóm nội dung chi tiết gồm Checklist, ChecklistItem, CardComment, CardCommentReaction và CardAttachment. Bình luận hỗ trợ trả lời lồng thông qua quan hệ tự tham chiếu parentId. Phản ứng có ràng buộc duy nhất theo bình luận, người dùng và emoji. Tập đính kèm phân biệt liên kết và tập bảng enum Attach-

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

mentType; tệp vật lý được lưu bởi UploadThing, còn cơ sở dữ liệu lưu siêu dữ liệu và URL.

Nhóm theo dõi và thương mại gồm Notification, AuditLog, OrgLimit và OrgSubscription. Notification chứa dữ liệu ngữ cảnh đã làm phẳng để thông báo vẫn có thể hiển thị tiêu đề tại thời điểm phát sinh. AuditLog lưu loại sự kiện và thực thể để phục vụ trang hoạt động và báo cáo. OrgLimit theo dõi số bảng của tổ chức, còn OrgSubscription ánh xạ tổ chức với khách hàng và thuê bao Stripe.

Thiết kế vật lý sử dụng các ánh xạ kiểu dữ liệu chính: String sang varchar(191), nội dung dài có @db.Text sang text, DateTime sang datetime(3), Int sang int và Boolean sang tinyint(1). UUID của khóa chính được Prisma Client tạo tại tầng ứng dụng; createdAt nhận giá trị mặc định khi tạo bản ghi, còn updatedAt được Prisma tự động cập nhật. Bảng 4.3 tóm tắt thiết kế của năm bảng cốt lõi mà không lặp lại toàn bộ thuộc tính đã có trong lược đồ.

Bảng 4.3: Thiết kế vật lý của các bảng dữ liệu cốt lõi

Bảng	Cột và kiểu dữ liệu chính	Khóa, quan hệ và chỉ mục
Board	id, orgId, title, imageId: varchar(191); imageThumbUrl, imageFullUrl, imageUserName, imageLinkHTML: text.	id là PK; orgId là định danh tổ chức do Clerk quản lý.
List	id, boardId, title: varchar(191); order: int; archivedAt: datetime(3), cho phép NULL.	FK boardId tham chiếu Board.id; chỉ mục boardId và (boardId, archivedAt, order).
Card	id, title, listId, reminder, archivedByListId: varchar(191); description: text; descriptionUpdatedAt, startDate, dueDate, reminderSetAt, archivedAt: datetime(3); order: int; isCompleted: tinyint(1).	FK listId tham chiếu List.id; chỉ mục listId và (listId, archivedAt, order).
BoardMember	id, boardId, userId: varchar(191); userName, userImage, userEmail: text; role: enum ADMIN, MEMBER, VIEWER.	FK boardId; duy nhất (boardId, userId); chỉ mục riêng theo boardId và userId.
CardDependency	id, blockerCardId, blockedCardId: varchar(191).	Hai FK cùng tham chiếu Card.id; duy nhất theo cặp hai thẻ; chỉ mục theo từng chiều phụ thuộc.

Các chỉ mục ghép được đặt theo điều kiện lọc và thứ tự sắp xếp của truy vấn Kanban. Với List và Card, chỉ mục lần lượt bắt đầu bằng khóa ngoại boardId

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

hoặc listId, sau đó là trạng thái lưu trữ và thứ tự. Ràng buộc duy nhất của BoardMember, CardAssignee, CardLabel, CardDependency và CardCommentReaction vừa bảo vệ tính toàn vẹn dữ liệu, vừa ngăn các liên kết nghiệp vụ bị tạo trùng.

Các quan hệ sở hữu sử dụng onDelete: Cascade. Vì vậy, khi xóa một bảng, các danh sách, thẻ và dữ liệu con có khóa ngoại tương ứng được xóa theo chuỗi, tránh bản ghi mồ côi. Riêng quan hệ từ ChecklistItem tới người phụ trách sử dụng SetNull; khi thành viên bị xóa khỏi bảng, nội dung công việc con vẫn được giữ lại nhưng không còn người phụ trách. Các model Notification, AuditLog, OrgLimit và OrgSubscription lưu định danh ngoài hoặc dữ liệu lịch sử nên không tạo khóa ngoại tới toàn bộ thực thể nghiệp vụ.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Các công cụ và thư viện trong Bảng 4.4 được lựa chọn theo vai trò trực tiếp trong quá trình phát triển hoặc vận hành HustFlow; các package giao diện phụ trợ nhỏ không được liệt kê riêng. Phiên bản Node.js, npm, Visual Studio Code và Git được lấy từ công cụ dòng lệnh; phiên bản thư viện được đối chiếu với cây phụ thuộc đã cài đặt và package-lock.json; phiên bản MySQL được truy vấn bằng SELECT VERSION().

Bảng 4.4: Thư viện và công cụ chính được sử dụng trong HustFlow

Mục đích	Công cụ/thư viện/API và phiên bản	Địa chỉ URL
IDE	Visual Studio Code 1.124.2	https://code.visualstudio.com/
Quản lý phiên bản	Git for Windows 2.47.1.windows.2	https://git-scm.com/
Môi trường chạy	Node.js 20.18.0; npm 10.8.2	https://nodejs.org/ https://www.npmjs.com/
Ngôn ngữ	TypeScript 5.9.3	https://www.typescriptlang.org/
Framework web	Next.js 16.2.1	https://nextjs.org/
Giao diện	React / React DOM 19.2.4	https://react.dev/
Định kiểu	Tailwind CSS 4.2.2	https://tailwindcss.com/
Cơ sở dữ liệu	MySQL 8.0.44; Prisma Client / Prisma CLI 6.19.3	https://www.mysql.com/ https://www.prisma.io/
Xác thực	@clerk/nextjs 7.0.12	https://clerk.com/
Thời gian thực	Pusher 5.3.3; Pusher JS 8.5.0	https://pusher.com/
Dữ liệu phía khách	TanStack React Query 5.100.10	https://tanstack.com/query/
Kiểm tra dữ liệu	Zod 4.3.6	https://zod.dev/
Kéo thả	@hello-pangea/dnd 18.0.1	https://github.com/hello-pangea/dnd
Biểu đồ	Recharts 3.8.1	https://recharts.org/
Trí tuệ nhân tạo	OpenAI REST API v1; gpt-4o-mini mặc định	https://developers.openai.com/api/reference/resources/chat
Thanh toán	Stripe SDK 22.1.1	https://stripe.com/docs
Lưu trữ tệp	UploadThing 7.7.4; @uploadthing/react 7.3.3	https://uploadthing.com/
Ảnh nền	Unsplash JS 7.0.20; Unsplash API	https://unsplash.com/developers

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mục đích	Công cụ/thư viện/API và phiên bản	Địa chỉ URL
Xuất dữ liệu	ExcelJS 4.4.0	https://github.com/exceljs/exceljs
Kiểm tra mã nguồn	ESLint 9.39.4; ESLint Config Next 16.2.1	https://eslint.org/ https://nextjs.org/docs/app/api-reference/config/eslint

Dự án hiện chưa cấu hình framework kiểm thử tự động như Jest, Vitest hoặc Playwright; `npm run lint` và `npm run build` chỉ cung cấp kiểm tra tĩnh, kiểm tra kiểu và khả năng biên dịch, không thay thế các ca kiểm thử chức năng được trình bày tại phần 4.4.

4.3.2 Kết quả đạt được

a, Kết quả xây dựng chức năng

Kết quả chính của đề án là ứng dụng web HustFlow hỗ trợ quản lý công việc và cộng tác theo tổ chức. Người dùng đăng nhập qua Clerk, lựa chọn tổ chức và truy cập các bảng mà mình là thành viên. Danh sách thành viên và vai trò được quản lý riêng cho từng bảng, qua đó phân biệt quyền quản trị, chỉnh sửa và chỉ xem.

Ở cấp tổ chức và bảng, ứng dụng đã hiện thực các chức năng tạo, cập nhật và xóa bảng; thêm hoặc xóa thành viên; thay đổi vai trò; rời bảng; lưu trữ và khôi phục dữ liệu. Giới hạn số bảng được áp dụng theo tổ chức, còn quản trị viên tổ chức có thể nâng cấp và quản lý thuê bao qua Stripe.

Ở cấp công việc, người dùng có thể tạo, sắp xếp, lưu trữ và khôi phục danh sách hoặc thẻ. Thẻ lưu trữ tiêu đề, mô tả Markdown, thời gian thực hiện, trạng thái hoàn thành, nhắc việc, người phụ trách, nhãn, checklist, bình luận, phản ứng, tệp hoặc liên kết đính kèm và quan hệ phụ thuộc. Các thay đổi nghiệp vụ chính được ghi vào nhật ký hoạt động.

Ứng dụng cung cấp năm chế độ xem dùng chung dữ liệu thẻ, gồm Kanban, Lịch, Dòng thời gian, Chia đôi và Thống kê. Lịch hỗ trợ chế độ ngày, tuần và tháng; Dòng thời gian biểu diễn khoảng thực hiện và quan hệ phụ thuộc; Chia đôi hiển thị đồng thời Lịch và Kanban; trang Thống kê tổng hợp tiến độ, công việc quá hạn, phân bổ theo danh sách và tải công việc theo thành viên.

Các chức năng AI gồm Smart Capture, đề xuất checklist, hỗ trợ tạo và cải thiện mô tả thẻ, cùng báo cáo bảng. Lời gọi mô hình được thực hiện phía máy chủ; đầu ra được kiểm tra cấu trúc, giới hạn độ dài và đối chiếu với dữ liệu bảng trước khi sử dụng. Smart Capture chỉ tạo bản nháp và yêu cầu người dùng xác nhận trước khi ghi dữ liệu.

b, Sản phẩm bàn giao

Kết quả bàn giao gồm hai thành phần. Thành phần thứ nhất là gói mã nguồn ứng dụng, bao gồm mã TypeScript, tài nguyên công khai, lược đồ Prisma, cấu hình và file khóa phụ thuộc; các thư mục sinh tự động, file môi trường và dữ liệu bí mật không thuộc gói bàn giao. Thành phần thứ hai là output build production của Next.js, gồm mã máy chủ đã biên dịch, tài nguyên tĩnh và metadata định tuyến. Output này vẫn cần môi trường Node.js, phụ thuộc runtime và các biến môi trường phù hợp khi triển khai.

Bảng 4.5 ghi nhận dung lượng ngày 20/06/2026. Gói nguồn được đo trên các file trong thư mục bkflow thuộc phạm vi quản lý phiên bản, gồm file đã được theo dõi và file mới chưa bị loại bởi .gitignore. Output production chỉ tính .next/server, .next/static, .next/build và các file ở gốc .next; thư mục phát triển và cache không được tính.

Bảng 4.5: Dung lượng các thành phần bàn giao của HustFlow

Thành phần	Số file	Dung lượng	Phạm vi đo
Gói mã nguồn ứng dụng	520	4.931.119 byte (4,70 MiB)	Các file thuộc phạm vi quản lý phiên bản trong thư mục bkflow; không gồm file bị .gitignore loại bỏ.
Output build production	776	75.106.348 byte (71,63 MiB)	Mã máy chủ, tài nguyên tĩnh, metadata build và các file gốc của .next; không gồm dev và cache.

c, Thống kê mã nguồn

Thống kê trong Bảng 4.6 được ghi nhận và phản ánh trạng thái kho mã nguồn tại thời điểm đo. Các số liệu có thể khác với cây làm việc ở thời điểm người đọc đối chiếu nếu mã nguồn tiếp tục được cập nhật. Phạm vi thống kê chỉ tính các file .ts, .tsx, .css và .prisma thuộc phạm vi quản lý phiên bản trong thư mục ứng dụng. Thư mục node_modules, sản phẩm build .next, file khóa gói và tài nguyên ảnh không được tính. Số dòng là số dòng vật lý, được xác định bằng số ký tự xuống dòng cộng một đối với mỗi file không rỗng.

Bảng 4.6: Thống kê mã nguồn HustFlow

Loại file	Số file	Số dòng vật lý	Dung lượng (byte)
TypeScript (.ts)	313	29.033	792.624
React TypeScript (.tsx)	184	28.775	956.011
CSS	1	216	6.476
Prisma schema	1	387	9.554
Tổng	499	58.411	1.764.665

Các chỉ số cấu trúc bổ sung được trình bày trong Bảng 4.7. Do HustFlow chủ yếu sử dụng component hàm, hook và service thay cho hệ thống lớp hướng đối tượng, số file TSX và số mô-đun nghiệp vụ được dùng để phản ánh quy mô cấu trúc thay cho chỉ tiêu “số lớp”.

Bảng 4.7: Thống kê cấu trúc mô-đun của ứng dụng

Thành phần	Số lượng	Tiêu chí đếm
File React TypeScript	184	Các file .tsx thuộc phạm vi quản lý phiên bản.
Mô-đun Server Action	57	Thư mục action có file index.ts.
API Route Handler	19	Các file route.ts trong app/api.
Hook phía khách	10	File TypeScript/TSX trong thư mục hooks.
Mô-đun thư viện	52	File .ts thuộc phạm vi quản lý phiên bản trong thư mục lib.
Model Prisma	17	Các khai báo model trong schema.prisma.
Nhóm thư mục nguồn cấp cao	9	actions, app, components, config, constants, hooks, lib, prisma, types.

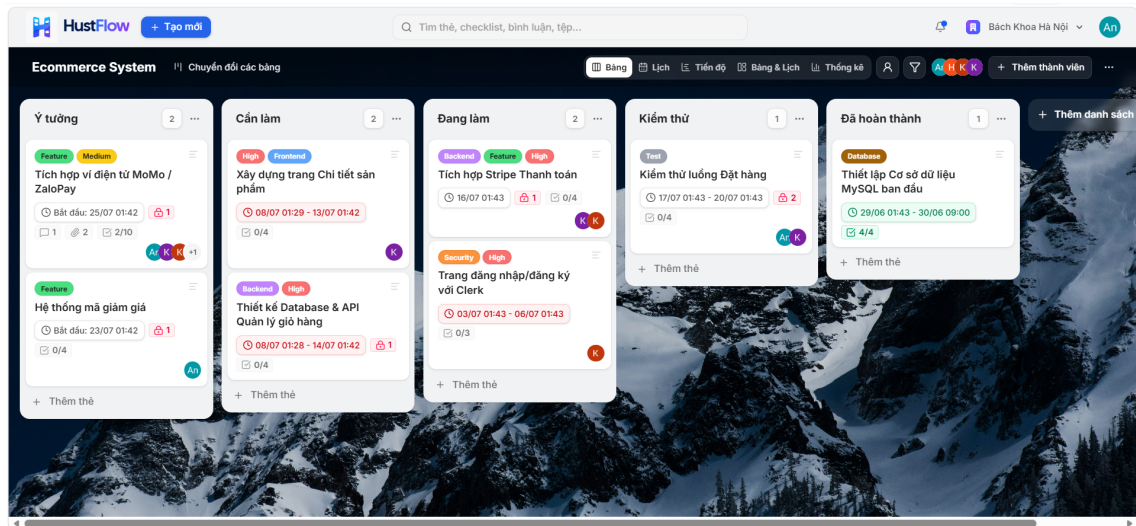
Các số liệu trên chỉ mô tả quy mô cài đặt, không phản ánh trực tiếp chất lượng phần mềm. Phạm vi và tiêu chí đếm được nêu rõ để có thể đo lại khi mã nguồn thay đổi.

4.3.3 Minh họa các chức năng chính

Sáu giao diện dưới đây được lựa chọn để minh họa các chức năng cốt lõi và những đặc điểm nổi bật của HustFlow, gồm quản lý công việc theo Kanban, xử lý chi tiết thẻ, lập kế hoạch theo lịch và dòng thời gian, tạo thẻ bằng AI, cùng theo dõi tiến độ qua thống kê và báo cáo.

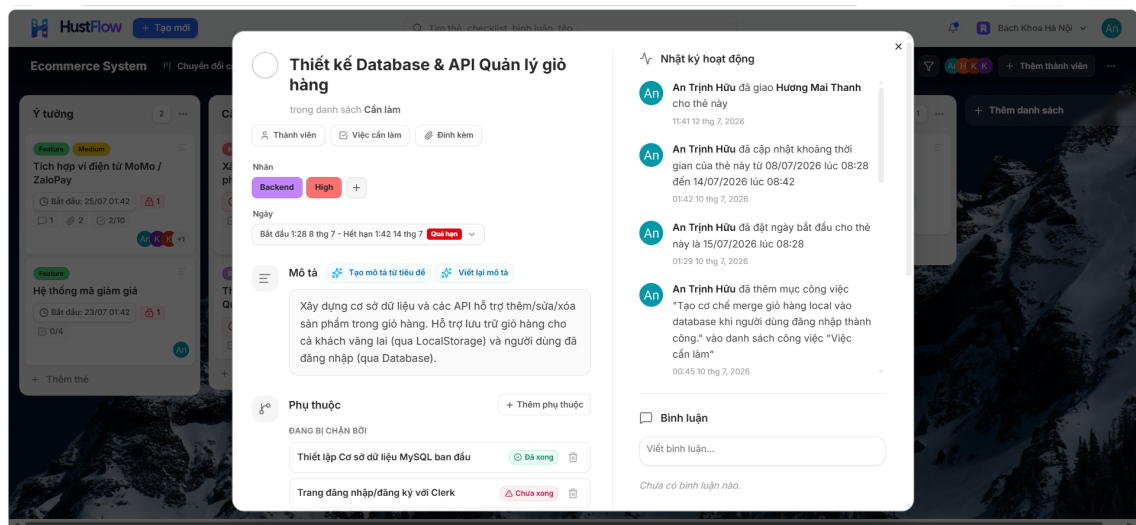
Hình 4.10 trình bày giao diện Kanban tổng quan. Các thẻ được tổ chức theo danh sách và hiển thị những thông tin cần nhận biết nhanh như nhãn, thời hạn, người phụ trách và tiến độ checklist. Người dùng có thể kéo thả thẻ trong cùng danh sách hoặc giữa các danh sách để cập nhật thứ tự và trạng thái công việc.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.10: Giao diện Kanban tổng quan của HustFlow

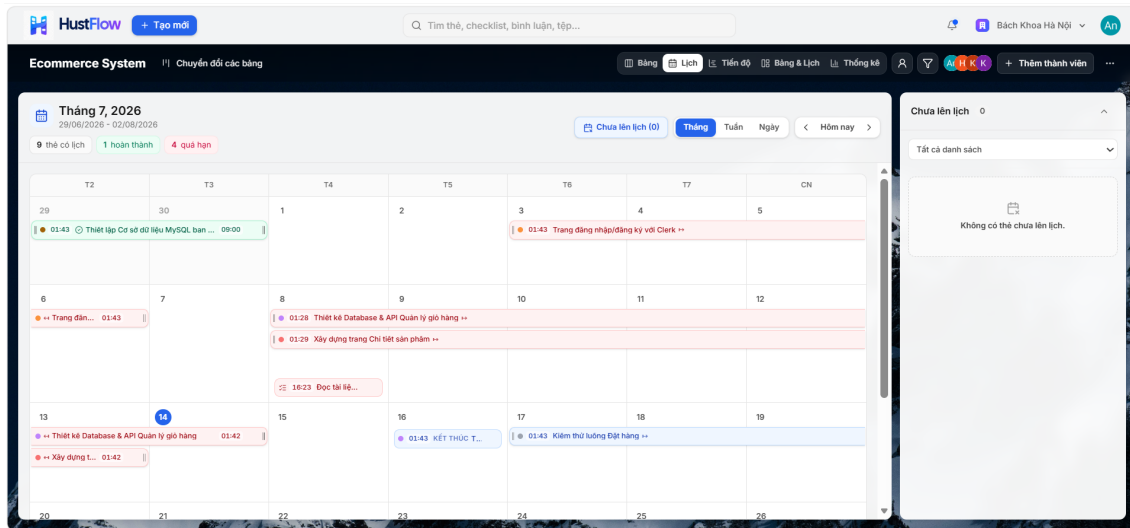
Hình 4.11 trình bày cửa sổ chi tiết thẻ. Giao diện tập trung các thuộc tính và hoạt động liên quan đến một công việc, gồm mô tả, thời gian thực hiện, người phụ trách, nhãn, checklist, tệp hoặc liên kết đính kèm, quan hệ phụ thuộc, bình luận và nhật ký hoạt động. Cửa sổ này cho phép người dùng cập nhật và trao đổi về công việc mà không phải rời khỏi bảng.



Hình 4.11: Giao diện chi tiết thẻ công việc

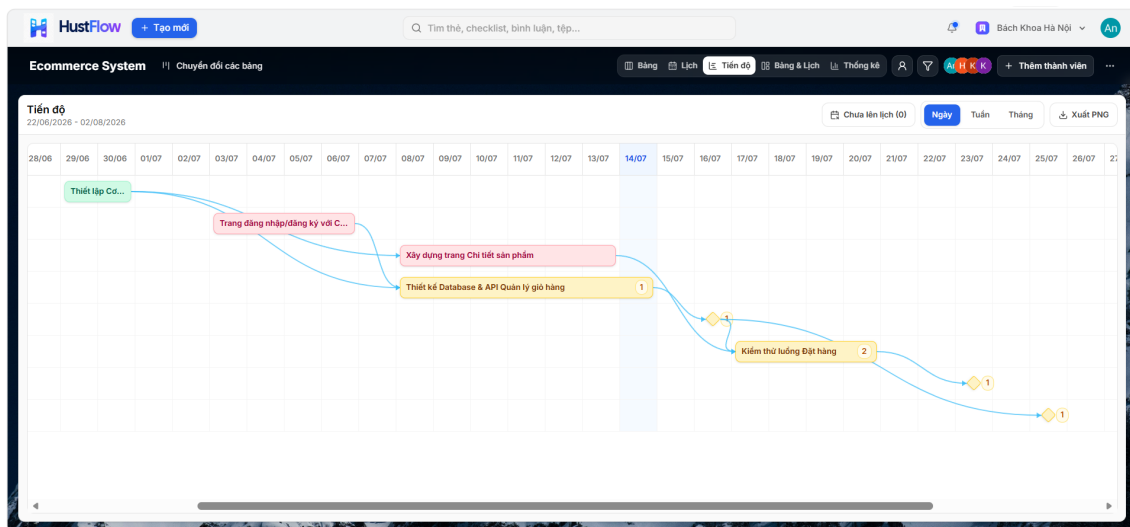
Hình 4.12 minh họa chế độ Lịch cùng khung “Chưa lên lịch”. Lịch hỗ trợ các chế độ xem ngày, tuần và tháng để theo dõi công việc theo thời gian. Khung “Chưa lên lịch” tập hợp những thẻ chưa có cả ngày bắt đầu và hạn hoàn thành; người dùng có thể kéo thẻ từ khung này vào một ngày trên lịch để thiết lập thời gian thực hiện.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.12: Giao diện Lịch với khung thẻ chưa lên lịch

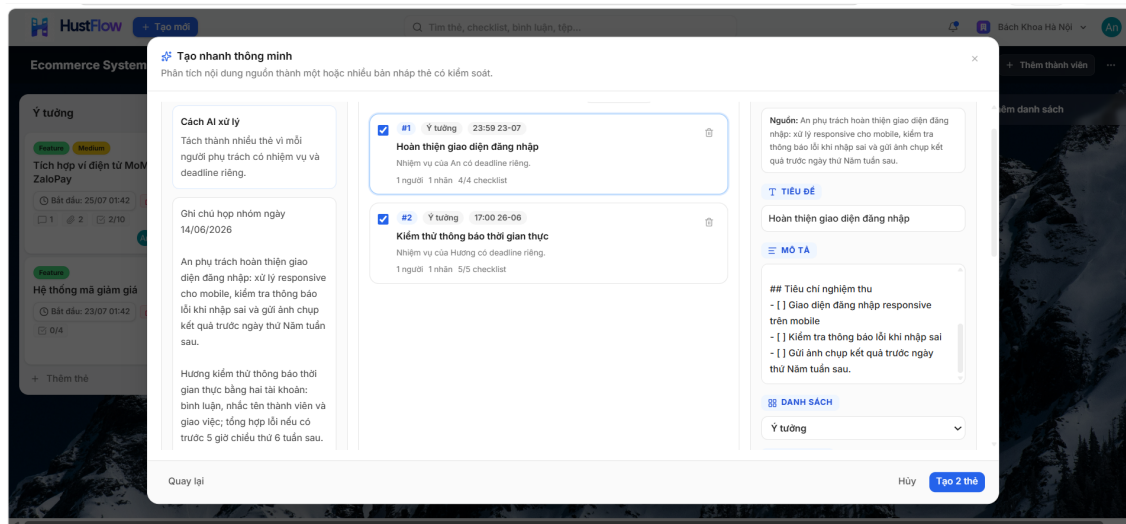
Hình 4.13 trình bày chế độ Dòng thời gian. Các thẻ có ngày bắt đầu và hạn hoàn thành được biểu diễn thành các thanh theo khoảng thời gian tương ứng; các đường nối thể hiện quan hệ phụ thuộc giữa những công việc liên quan. Giao diện này hỗ trợ quan sát thời lượng, thứ tự thực hiện và ảnh hưởng của một công việc tới tiến độ chung.



Hình 4.13: Giao diện lập kế hoạch theo dòng thời gian

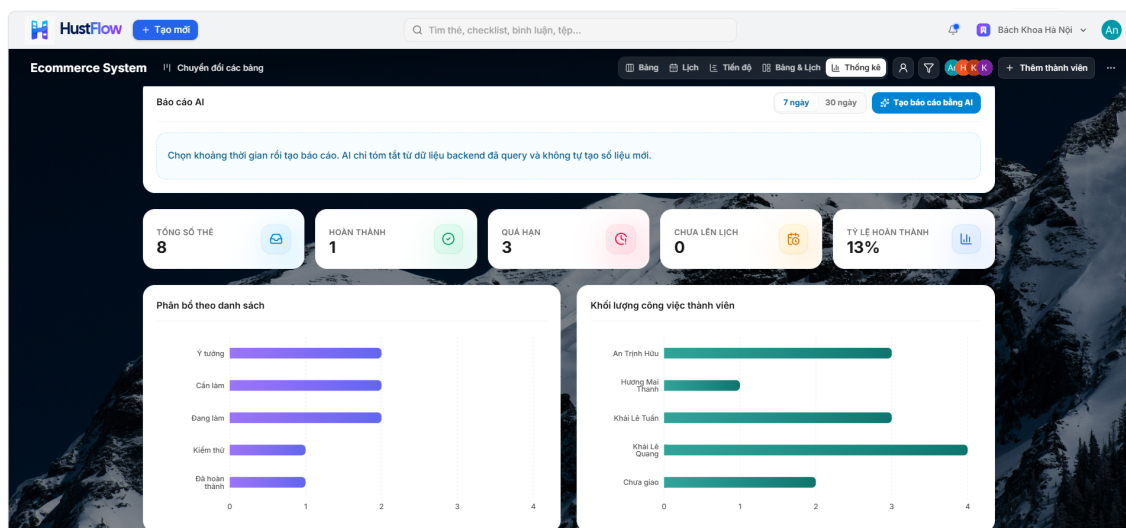
Hình 4.14 trình bày bản nháp đầu ra của Smart Capture sau khi phân tích nội dung bằng ngôn ngữ tự nhiên. Bản nháp gồm tiêu đề, mô tả, checklist, danh sách đích, hạn chót, người thực hiện và nhãn công việc. Người dùng có thể kiểm tra, chỉnh sửa hoặc loại bỏ các đề xuất trước khi xác nhận tạo thẻ, do đó AI chỉ hỗ trợ nhập liệu và không tự động ghi dữ liệu vào bảng.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.14: Bản nháp thẻ do Smart Capture tạo ra

Hình 4.15 trình bày trang Thống kê và báo cáo AI của bảng. Trang này tổng hợp các chỉ số về số lượng và tiến độ thẻ, công việc chưa lên lịch hoặc quá hạn, phân bổ công việc theo danh sách và khối lượng công việc của thành viên. Trước khi gọi mô hình, máy chủ chuẩn bị ngữ cảnh gồm các chỉ số và hoạt động đã truy vấn từ hệ thống; prompt yêu cầu mô hình chỉ sử dụng ngữ cảnh này để tóm tắt kết quả, nhận diện rủi ro và đề xuất hành động. Phản hồi được kiểm tra cấu trúc và giới hạn dữ liệu bằng Zod, nhưng cài đặt hiện tại chưa đối chiếu tất định từng số liệu hoặc nhận định trong nội dung do mô hình tạo ra.



Hình 4.15: Giao diện thống kê và báo cáo của bảng

4.4 Kiểm thử

4.4.1 Phương pháp và môi trường kiểm thử

Các chức năng được đánh giá bằng kiểm thử hộp đen theo kịch bản, dựa trên dữ liệu đầu vào, thao tác của người dùng và kết quả quan sát được trên giao diện. Bốn kỹ thuật được sử dụng gồm: (i) kiểm thử theo use case cho luồng nghiệp vụ chính;

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

(ii) phân lớp tương đương để đại diện cho nhóm dữ liệu hợp lệ và không hợp lệ; (iii) phân tích giá trị biên đối với độ dài văn bản; và (iv) kiểm thử chuyển trạng thái đối với quan hệ phụ thuộc và dữ liệu được chia sẻ giữa nhiều phiên. Cách tiếp cận này phù hợp với mục tiêu đánh giá hành vi chức năng; kiểm thử hộp trắng không được sử dụng do phạm vi này không đo độ phủ câu lệnh, nhánh hoặc đường đi trong mã nguồn.

Kiểm thử được thực hiện trên máy phát triển chạy Windows, Node.js 20.18.0 và npm 10.8.2. Ứng dụng Next.js hoạt động tại localhost:3000 và sử dụng cơ sở dữ liệu MySQL thử nghiệm. Dữ liệu chuẩn bị gồm một bảng có hai danh sách, ba thẻ, hai tài khoản có quyền chỉnh sửa, một tài khoản **VIEWER**, cùng cấu hình cần thiết cho Smart Capture. Dữ liệu được đưa về trạng thái ban đầu trước mỗi nhóm kiểm thử.

4.4.2 Thiết kế ca kiểm thử chức năng

Bảng 4.8 trình bày chín ca kiểm thử cho ba chức năng quan trọng: quản lý thẻ và đồng bộ, thiết lập quan hệ phụ thuộc, và tạo thẻ bằng Smart Capture. Mỗi nhóm bao gồm luồng hợp lệ và trường hợp ngoại lệ hoặc giá trị biên.

Bảng 4.8: Các ca kiểm thử chức năng trọng tâm

Mã	Kỹ thuật	Dữ liệu và bước thực hiện	Kết quả mong đợi	Kết quả thực tế	Đánh giá
FT01	Theo use case	Tài khoản MEMBER kéo một thẻ sang danh sách khác; tài khoản thứ hai đang mở cùng bảng.	Thẻ xuất hiện đúng vị trí mới ở cả hai phiên mà không cần tải lại trang.	Hai phiên hiển thị cùng vị trí mới của thẻ.	Đạt
FT02	Phân lớp tương đương	Tài khoản VIEWER thử sửa tiêu đề và kéo thẻ sang danh sách khác.	Hệ thống không cho phép thay đổi; dữ liệu thẻ giữ nguyên.	Các thao tác bị từ chối và dữ liệu không thay đổi.	Đạt
FT03	Chuyển trạng thái	Hai phiên mở cùng một thẻ. Phiên thứ nhất lưu mô tả mới; phiên thứ hai lưu nội dung từ phiên bản cũ.	Phiên thứ hai nhận cảnh báo xung đột; mô tả mới nhất không bị ghi đè.	Cảnh báo được hiển thị và nội dung của phiên thứ nhất được giữ lại.	Đạt
FT04	Theo use case	Với hai thẻ A và B cùng bảng, thiết lập A là thẻ chặn của B.	Quan hệ được tạo và hiển thị đúng trên hai thẻ liên quan.	Quan hệ A-B được hiển thị đúng sau khi xác nhận.	Đạt

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mã	Kỹ thuật	Dữ liệu và bước thực hiện	Kết quả mong đợi	Kết quả thực tế	Đánh giá
FT05	Phân lớp tương đương	Lần lượt chọn một thẻ phụ thuộc chính nó và tạo lại quan hệ A-B đã tồn tại.	Cả hai thao tác bị từ chối với thông báo phù hợp; không phát sinh quan hệ mới.	Hệ thống báo lỗi tương ứng và số quan hệ không đổi.	Đạt
FT06	Chuyển trạng thái	Chuẩn bị $A \rightarrow B$ và $B \rightarrow C$, sau đó thêm $C \rightarrow A$.	Hệ thống cảnh báo quan hệ mới tạo vòng lặp và không lưu thay đổi.	Cảnh báo vòng lặp được hiển thị; quan hệ C-A không được tạo.	Đạt
FT07	Theo use case	Nhập văn bản tiếng Việt gồm công việc, hạn hoàn thành và người phụ trách; xem bản nháp rồi xác nhận tạo thẻ.	Bản nháp chứa thông tin đã trích xuất; thẻ chỉ được tạo sau khi người dùng xác nhận.	Bản nháp hiển thị đúng và thẻ được tạo sau khi xác nhận.	Đạt
FT08	Giá trị biên	Lần lượt gửi nội dung rỗng, nội dung dài 4.000 ký tự và 4.001 ký tự tới Smart Capture.	Nội dung rỗng và 4.001 ký tự bị từ chối; nội dung 4.000 ký tự được chấp nhận để phân tích.	Hệ thống xử lý đúng cả ba giá trị biên.	Đạt
FT09	Chuyển trạng thái	Phân tích một nội dung hợp lệ để tạo bản nháp, sau đó đóng hộp thoại mà không xác nhận.	Bản nháp bị hủy và không có thẻ mới trong danh sách.	Không phát sinh thẻ sau khi hủy bản nháp.	Đạt

4.4.3 Tổng hợp kết quả

Kết quả cho thấy cả chín ca kiểm thử chức năng đều đạt, không có ca không đạt, tương ứng tỷ lệ đạt 100%. Các ca phủ định được xem là đạt khi hệ thống từ chối đúng thao tác không hợp lệ và giữ nguyên dữ liệu. Do không phát sinh ca thất bại nên phần này không có nguyên nhân lỗi cần phân tích.

4.5 Triển khai

4.5.1 Môi trường chạy thử nghiệm

HustFlow hiện chỉ được chạy thử nghiệm cục bộ trên máy phát triển, chưa được công khai trên Internet và chưa có máy chủ production. Trình duyệt, tiến trình Next.js và cơ sở dữ liệu MySQL cùng hoạt động trên một thiết bị. Tiến trình Next.js được khởi động bằng lệnh `npm run dev` và phục vụ ứng dụng tại địa chỉ `http://localhost:3000`; Prisma kết nối tới MySQL tại `localhost:3306`.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Cấu hình thiết bị và phần mềm dùng để chạy thử nghiệm được tổng hợp trong Bảng 4.9. Cấu hình này đáp ứng việc phát triển và kiểm tra các chức năng trong phạm vi đề án, nhưng không đại diện cho cấu hình của một máy chủ vận hành thực tế.

Bảng 4.9: Cấu hình môi trường chạy thử nghiệm HustFlow

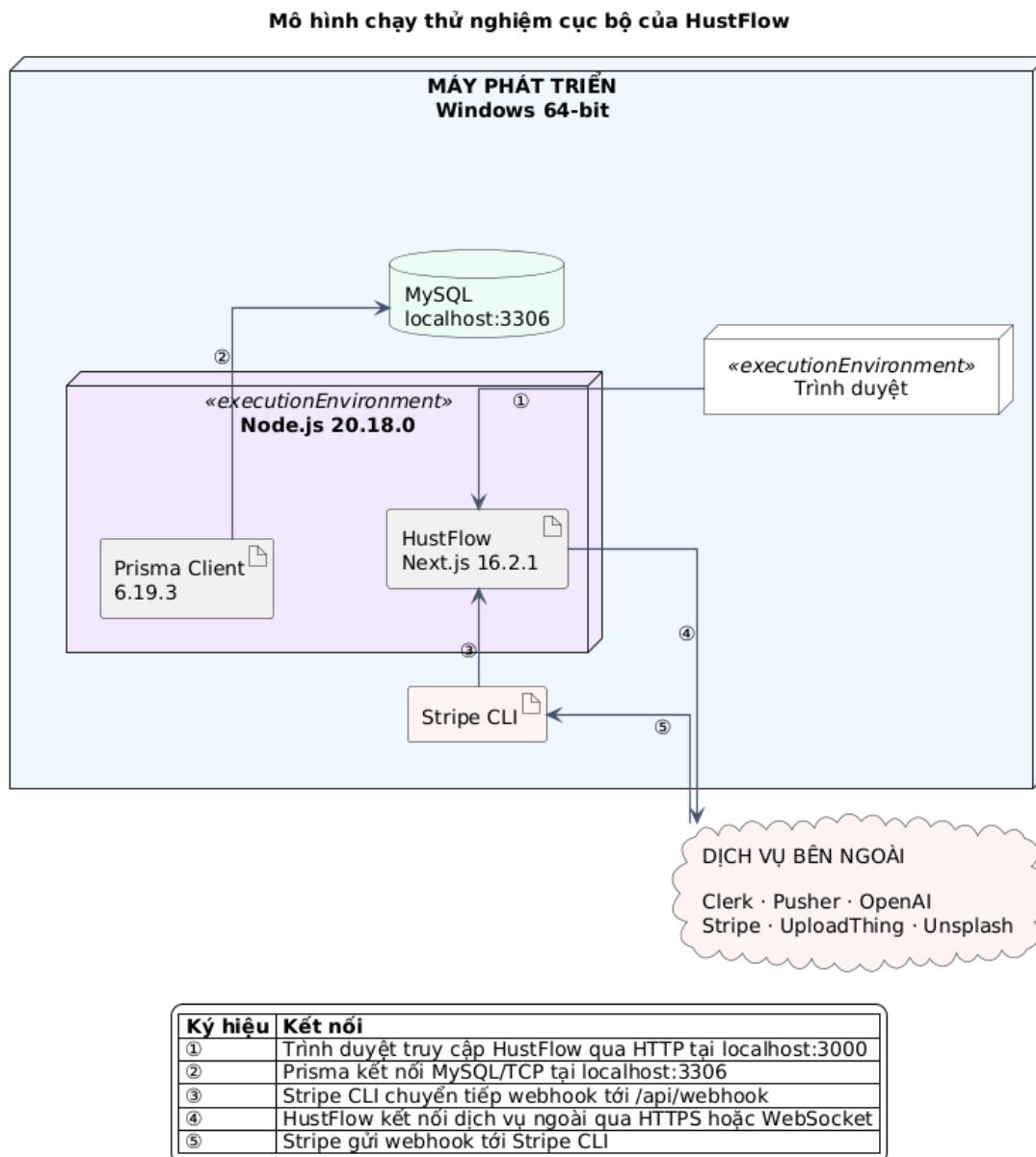
Thành phần	Cấu hình
Hệ điều hành	Windows 64-bit
Bộ xử lý	AMD Ryzen 5 5500U
Bộ nhớ	16 GB RAM
Môi trường thực thi	Node.js 20.18.0 và npm 10.8.2.
Ứng dụng web	Next.js 16.2.1, chạy bằng npm run dev tại localhost:3000.
Dữ liệu	MySQL chạy tại localhost:3306; Prisma 6.19.3 thực hiện truy cập dữ liệu.

Quy trình chạy thử nghiệm gồm: (i) khởi động MySQL trên máy phát triển; (ii) cấu hình chuỗi kết nối DATABASE_URL và khóa truy cập các dịch vụ trong tệp .env; (iii) sinh Prisma Client khi lược đồ dữ liệu thay đổi; (iv) khởi động ứng dụng bằng lệnh npm run dev; (v) truy cập hệ thống tại http://localhost:3000; và (vi) khi kiểm tra chức năng thanh toán, sử dụng Stripe CLI để chuyển tiếp webhook tới localhost:3000/api/webhook. Các giá trị bí mật của biến môi trường không được công bố trong báo cáo.

4.5.2 Mô hình chạy thử nghiệm cục bộ

Hình 4.16 trình bày mô hình chạy thử nghiệm cục bộ của HustFlow. Trình duyệt gửi yêu cầu HTTP tới tiến trình Next.js qua cổng 3000. Các Server Component, Server Action và Route Handler sử dụng Prisma để truy cập MySQL qua cổng 3306. Toàn bộ các thành phần này cùng nằm trên máy phát triển.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.16: Mô hình chạy thử nghiệm cục bộ của HustFlow, không phải môi trường production

Trong quá trình chạy, ứng dụng vẫn truy cập Clerk, Pusher, OpenAI, Stripe, UploadThing và Unsplash qua Internet. Stripe CLI đóng vai trò trung gian nhận webhook từ Stripe và chuyển tiếp yêu cầu tới Route Handler trên máy local. Do hệ thống chưa được triển khai cho người dùng thực tế, báo cáo không đưa ra kết luận về số lượng người dùng, số lượt truy cập, thời gian phản hồi hoặc khả năng chịu tải.

Chương này đã chuyển các nền tảng công nghệ ở Chương 3 thành thiết kế cụ thể cho HustFlow. Hệ thống được tổ chức theo kiến trúc nguyên khối mô-đun với ranh giới giữa giao diện, xử lý ứng dụng, nghiệp vụ và dữ liệu. Các luồng cập nhật thẻ, kiểm soát phụ thuộc, Smart Capture và đồng bộ thời gian thực đều thực hiện kiểm

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

tra phía máy chủ trước khi thay đổi dữ liệu. Lược đồ MySQL gồm 17 model hỗ trợ quản lý công việc, cộng tác, phân quyền, thông báo và thuê bao.

Kết quả xây dựng bao gồm các chế độ Kanban, Lịch, Dòng thời gian, Chia đôi và Thống kê cùng các chức năng chi tiết của thẻ, AI và thanh toán. Chín ca kiểm thử hộp đen cho ba chức năng trọng tâm đều đạt; build production hoàn tất và lint không có lỗi, còn ba cảnh báo. Hệ thống đã được chạy và kiểm tra trong môi trường cục bộ trên máy phát triển, chưa được triển khai cho người dùng thực tế. Từ nền tảng thiết kế và kết quả này, Chương 5 sẽ phân tích sâu các giải pháp và đóng góp kỹ thuật nổi bật của đề án.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày kiến trúc, thiết kế dữ liệu, các luồng xử lý và kết quả xây dựng hệ thống HustFlow. Trên cơ sở đó, chương này phân tích ba đóng góp kỹ thuật nổi bật: (i) đồng bộ dữ liệu thời gian thực có kiểm soát; (ii) kiểm soát quan hệ phụ thuộc giữa các thẻ bằng đồ thị có hướng; và (iii) tích hợp trí tuệ nhân tạo theo nguyên tắc sinh bản nháp có ràng buộc.

Các đóng góp được lựa chọn dựa trên mức độ khó của bài toán, ảnh hưởng tới tính đúng đắn của dữ liệu và khả năng tái sử dụng trong các chức năng khác. Nội dung chương tập trung vào bài toán, quyết định thiết kế, kết quả và giới hạn; các công nghệ nền tảng và kiến trúc tổng thể đã được trình bày tại Chương 3 và Chương 4.

5.1 Đồng bộ dữ liệu thời gian thực có kiểm soát

5.1.1 Bài toán đặt ra

HustFlow cung cấp nhiều cách quan sát và thay đổi cùng một tập dữ liệu, bao gồm Kanban, Lịch, Dòng thời gian, Chia đôi và hộp thoại chi tiết thẻ. Khi một thành viên di chuyển thẻ trên Kanban, thay đổi ngày trong Lịch hoặc cập nhật checklist, các giao diện khác đang mở cần phản ánh trạng thái mới. Nếu mỗi giao diện chỉ tải dữ liệu khi được mở, trạng thái hiển thị có thể trở nên cũ. Người dùng khi đó có thể thao tác dựa trên thứ tự thẻ, người phụ trách hoặc thời hạn không còn đúng với dữ liệu ở máy chủ.

Một sự kiện thời gian thực có thể được nhận nhiều lần, đến sau một cập nhật mới hơn hoặc không chứa đủ dữ liệu để vá trạng thái cục bộ. Thành phần phát sinh thao tác cũng có thể đã áp dụng cập nhật lạc quan. Nếu tiếp tục xử lý sự kiện do chính thao tác đó tạo ra, giao diện có nguy cơ áp dụng thay đổi hai lần. Khi kết nối bị gián đoạn, việc đăng ký kênh một lần cũng không bảo đảm thành phần giao diện luôn sẵn sàng nhận sự kiện.

Quyền truy cập cũng là một phần của bài toán. Tên bảng hoặc tên thẻ không phải là bí mật và có thể xuất hiện trong URL. Nếu chỉ cần biết tên kênh để đăng ký, người dùng đã đăng nhập nhưng không thuộc bảng vẫn có thể nhận sự kiện về hoạt động của bảng đó. Do vậy, giải pháp đồng bộ cần đồng thời giải quyết ba yêu cầu: dữ liệu bền vững phải được ghi trước khi thông báo được phát; máy khách phải có chiến lược cập nhật và phục hồi khi dữ liệu sự kiện không đầy đủ; và quyền đăng ký kênh phải được kiểm tra theo đúng phạm vi dữ liệu.

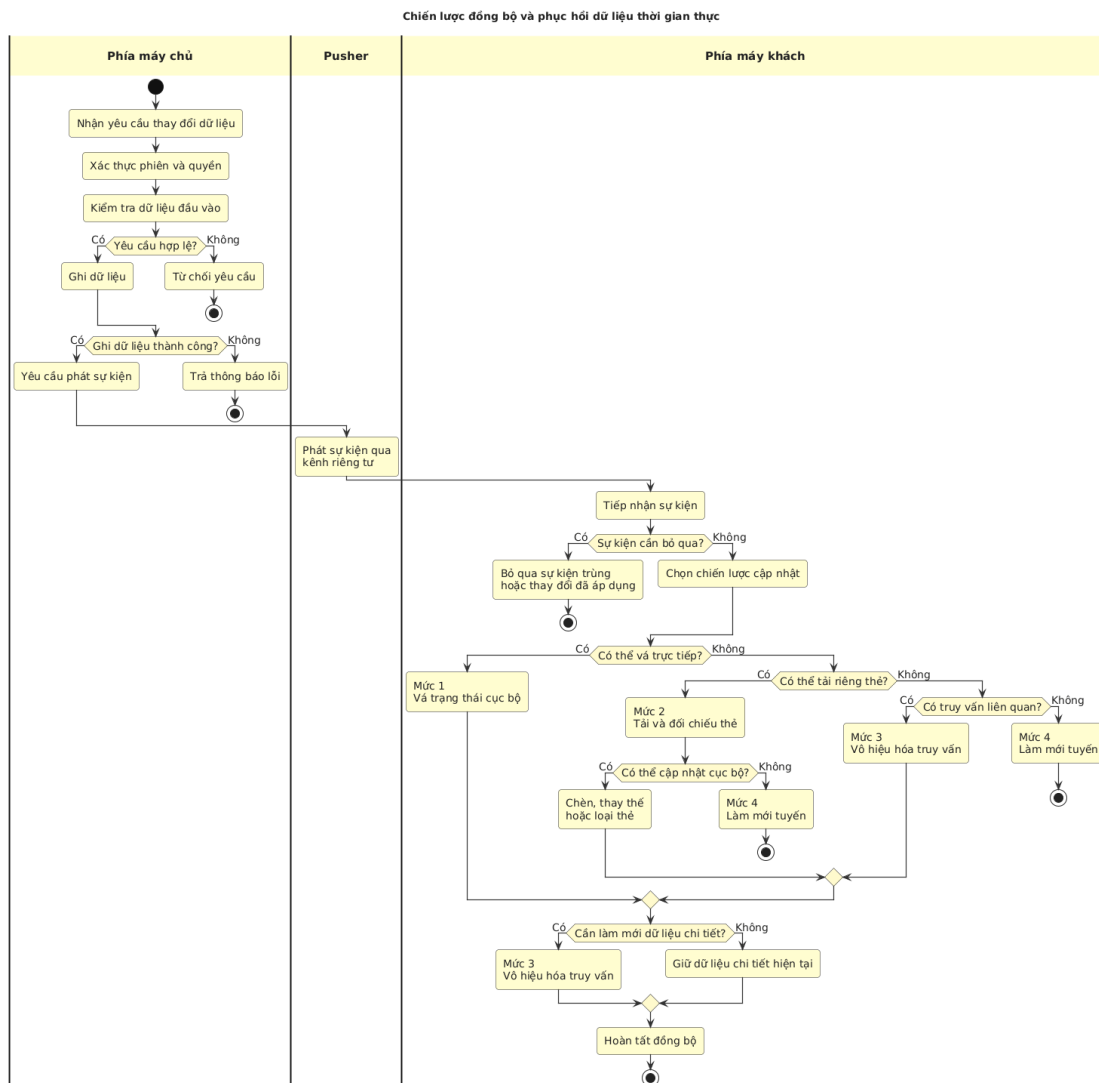
5.1.2 Giải pháp đề xuất

Giải pháp của HustFlow sử dụng máy chủ và cơ sở dữ liệu làm nguồn dữ liệu chuẩn. Một thao tác ghi trước hết đi qua Server Action, nơi dữ liệu đầu vào được kiểm tra, phiên người dùng được xác thực và quy tắc nghiệp vụ được áp dụng. Chỉ sau khi thay đổi được ghi vào MySQL, hệ thống mới tạo sự kiện Pusher [11]. Trình tự này tránh trường hợp giao diện khác nhận được tín hiệu về một trạng thái chưa tồn tại hoặc cuối cùng bị từ chối ở cơ sở dữ liệu. Với các thao tác cập nhật nhiều bản ghi, sự kiện được phát sau khi giao dịch Prisma hoàn tất để máy khách không quan sát trạng thái trung gian.

Hệ thống phân chia kênh riêng tư theo bốn phạm vi gồm tổ chức, bảng, thẻ và người dùng. Tên kênh lần lượt có dạng `private-org-*`, `private-board-*`, `private-card-*` và `private-user-*`. Điểm cuối xác thực Pusher phân tích tên kênh trước khi cấp chữ ký đăng ký. Kênh người dùng chỉ được cấp cho chính người đang đăng nhập; kênh tổ chức phải trùng tổ chức hoạt động trong phiên; kênh bảng yêu cầu tồn tại bản ghi BoardMember; còn kênh thẻ yêu cầu thẻ thuộc một bảng mà người dùng là thành viên. Vì vậy, việc đoán được định danh không đủ để nhận luồng sự kiện.

Các sự kiện được mô tả bằng kiểu TypeScript theo từng tên sự kiện. Phần lớn sự kiện phục vụ đồng bộ bảng và thẻ chứa `eventId`, `actorUserId`, phạm vi dữ liệu và thời điểm phát sinh. Những sự kiện tác động tới dữ liệu do React Query quản lý còn mang danh sách khóa truy vấn cần vô hiệu hóa. Thiết kế này tạo hợp đồng rõ giữa mã phát và mã tiếp nhận, đồng thời hạn chế việc xử lý sai cấu trúc dữ liệu.

Ở phía máy khách, HustFlow áp dụng chiến lược cập nhật bốn mức được minh họa trong Hình 5.1. Mức thứ nhất vá trực tiếp trạng thái khi dữ liệu sự kiện đầy đủ và trạng thái cục bộ phù hợp. Ví dụ, sự kiện cập nhật thẻ có thể thay thế tiêu đề, mô tả, ngày, trạng thái hoàn thành và nhắc việc trong danh sách đang hiển thị. Sự kiện sắp xếp mang theo các định danh theo thứ tự mới; máy khách chỉ áp dụng khi số lượng và tập định danh khớp với dữ liệu hiện có.



Hình 5.1: Chiến lược đồng bộ và phục hồi dữ liệu thời gian thực

Mức thứ hai được sử dụng khi sự kiện xác định được thẻ bị tác động nhưng không mang đủ dữ liệu để vá. Máy khách tải riêng thẻ qua API rồi đối chiếu với trạng thái cục bộ. Thẻ không còn tồn tại bị loại; nếu danh sách đích đang hiển thị, hệ thống giữ bản cục bộ mới hơn hoặc chèn bản ghi tải về. Khi không thể cập nhật cục bộ, hệ thống chuyển sang làm mới tuyến.

Mức thứ ba vô hiệu hóa các truy vấn React Query liên quan tới hộp thoại chi tiết, nhật ký hoặc bình luận. Mức này có thể được áp dụng độc lập hoặc bổ sung sau khi trạng thái danh sách đã được cập nhật. Khi dữ liệu không thể phục hồi cục bộ, mức thứ tư gọi `router.refresh()` để đồng bộ lại dữ liệu của tuyến từ máy chủ.

Để xử lý sự kiện lặp, các hook đồng bộ duy trì tập `eventId` đã tiếp nhận trong vòng đời của thành phần giao diện. Sự kiện đã xuất hiện sẽ bị bỏ qua. Với các thao tác gán thành viên hoặc gán nhãn đã được cập nhật lạc quan, máy khách dựa vào `actorUserId` để không áp dụng lại sự kiện của chính người dùng hiện tại.

Phần quản lý kết nối sử dụng một đối tượng Pusher dùng chung ở phía máy khách và bộ đếm tham chiếu theo tên kênh. Kênh chỉ bị hủy khi số tham chiếu trở về không. Khi đăng ký lỗi hoặc chưa sẵn sàng sau bốn giây, hook yêu cầu đăng ký lại. Khoảng giới hạn giữa các lần phục hồi ngăn nhiều hook đồng thời tạo chuỗi kết nối lại liên tục.

5.1.3 Kết quả đạt được

Giải pháp đã hình thành cơ chế đồng bộ dùng chung cho dữ liệu bảng, danh sách, thẻ và các quan hệ chi tiết. Kanban có thể cập nhật gia tăng đối với các thao tác phổ biến, trong khi Lịch, Dòng thời gian và hộp thoại chi tiết làm mới dữ liệu theo phạm vi phù hợp. Kiểm tra quyền kênh ngăn người không thuộc phạm vi dữ liệu đăng ký luồng sự kiện.

Đóng góp chính là sự kết hợp giữa sự kiện có kiểu, kiểm tra quyền, cập nhật gia tăng và bốn mức phục hồi. Cấu trúc này giảm nhu cầu tải lại toàn bộ trang nhưng vẫn duy trì đường quay về cơ sở dữ liệu là nguồn chuẩn.

Giải pháp chưa có cơ chế phiên bản hóa tổng quát cho nhánh vá trực tiếp. Vì vậy, một sự kiện cũ đến muộn vẫn có thể ghi đè trường dữ liệu mới hơn trong một số luồng. Đồ án cũng chưa đo độ trễ, tỷ lệ mất sự kiện, mức bảo đảm giao nhận hoặc tải nhiều người dùng, nên chưa đưa ra kết luận về hiệu năng và khả năng chịu tải.

5.2 Kiểm soát quan hệ phụ thuộc bằng đồ thị có hướng

5.2.1 Bài toán đặt ra

Quan hệ phụ thuộc biểu diễn thứ tự xử lý giữa các thẻ. Với cạnh $u \rightarrow v$, thẻ u là thẻ chặn và thẻ v là thẻ bị chặn. Nếu quan hệ được tạo mà không kiểm soát, dữ liệu có thể xuất hiện bốn nhóm sai lệch: một thẻ phụ thuộc vào chính nó, cùng một cạnh được lưu nhiều lần, một trong hai thẻ không hợp lệ trong phạm vi bảng hiện tại, hoặc cạnh mới khép kín một chu trình. Nhóm thứ ba bao gồm thẻ thuộc bảng hay tổ chức khác, thẻ đã bị lưu trữ và thẻ thuộc danh sách đã bị lưu trữ. Chu trình làm mất ý nghĩa thứ tự trước-sau vì mỗi thẻ trong chu trình đều gián tiếp chờ chính nó.

Danh sách ứng viên trên trình duyệt chỉ hỗ trợ thao tác và có thể trở nên cũ trước khi người dùng xác nhận. Yêu cầu cũng có thể được gửi trực tiếp tới Server Action mà không đi qua giao diện. Vì vậy, quyết định chấp nhận quan hệ phải được thực hiện tại máy chủ, dựa trên dữ liệu hiện có và trước khi cạnh được ghi. Cơ chế này bảo vệ cấu trúc đồ thị; việc một thẻ còn bị chặn có được đánh dấu hoàn thành hay không là một quy tắc riêng.

5.2.2 Giải pháp đề xuất

Nền tảng DAG và phép kiểm tra tính khả đạt đã được trình bày tại phần 3.1.1. Trong cài đặt HustFlow, mỗi cạnh được ánh xạ thành một bản ghi CardDependency. Trường blockerCardId là đỉnh nguồn u , còn blockedCardId là đỉnh đích v . Hai khóa ngoại cùng trỏ tới Card nhưng sử dụng hai tên quan hệ Prisma khác nhau. Ở mức lưu trữ, khóa ngoại bảo vệ sự tồn tại của thẻ và ràng buộc duy nhất trên cặp blockerCardId, blockedCardId ngăn cùng một cạnh được lưu hai lần. Các ràng buộc này không tự bảo đảm hai thẻ thuộc cùng bảng và cũng không phát hiện chu trình.

Các quy tắc còn lại được thực hiện trong Server Action createCardDependency. Action xác thực phiên, yêu cầu quyền chỉnh sửa bảng, loại tự phụ thuộc và truy vấn đồng thời hai thẻ với điều kiện thuộc đúng bảng–tổ chức, chưa bị lưu trữ và nằm trong danh sách chưa bị lưu trữ. Sau đó, hệ thống kiểm tra cạnh trùng trước khi duyệt đồ thị. Việc kiểm tra ở máy chủ tạo lớp bảo vệ độc lập với danh sách ứng viên trên giao diện.

Khi chuẩn bị thêm cạnh $u \rightarrow v$, hệ thống cần xác định trong đồ thị hiện tại có đường đi từ v tới u hay không. Nếu đường đi đó tồn tại, cạnh mới sẽ nối u trở lại v và tạo chu trình. Hàm kiểm tra tải các cạnh có đỉnh nguồn thuộc bảng và tổ chức hiện tại, xây danh sách kề theo hướng từ thẻ chặn tới các thẻ bị chặn, sau đó duyệt BFS từ v . Mã giả của bước kiểm tra được trình bày trong Thuật toán 1.

Algorithm 1: Kiểm tra chu trình trước khi thêm quan hệ phụ thuộc

Input: Tập cạnh hiện có E , cạnh dự kiến $u \rightarrow v$

Output: true nếu cạnh mới tạo chu trình, ngược lại false

Tạo danh sách kề Adj từ các cạnh trong E

Khởi tạo hàng đợi $Q \leftarrow [v]$, chỉ số đầu $h \leftarrow 1$ và tập đã thăm $Visited \leftarrow \{v\}$

while $h \leq |Q|$ **do**

$x \leftarrow Q[h]$, sau đó $h \leftarrow h + 1$

if $x = u$ **then**

return true

foreach $y \in Adj[x]$ **do**

if $y \notin Visited$ **then**

Thêm y vào $Visited$ và cuối hàng đợi Q

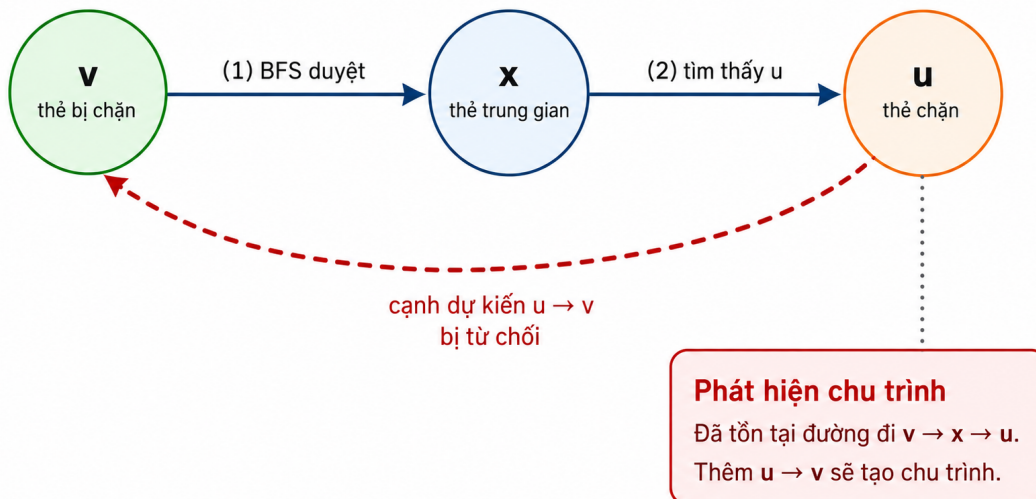
return false

Tập Visited bảo đảm mỗi đỉnh chỉ được đưa vào hàng đợi một lần. Trong mã giả, chỉ số h tăng dần nên thao tác lấy phần tử kế tiếp có chi phí $O(1)$. Khi đó, bước

xây dựng danh sách kề và duyệt BFS có độ phức tạp thời gian $O(V + E)$, còn độ phức tạp không gian là $O(V + E)$ khi tính cả danh sách kề, hàng đợi và tập đỉnh đã thăm. BFS phù hợp vì bài toán chỉ cần kiểm tra tính khả đạt khi thêm một cạnh, không cần tính lại thứ tự tô pô của toàn bộ bảng.

Bản cài đặt TypeScript hiện sử dụng `Array.shift()` để lấy phần tử đầu hàng đợi. Thao tác này có thể phải dịch chuyển các phần tử còn lại, vì vậy không thể dùng trực tiếp cận $O(V + E)$ của hàng đợi trừu tượng để khẳng định độ phức tạp chặt cho mã hiện tại. Việc thay bằng chỉ số đầu như trong mã giả sẽ làm cách cài đặt phù hợp với phân tích tuyến tính mà không thay đổi kết quả duyệt.

Hình 5.2 minh họa trường hợp đã có đường $v \rightarrow x \rightarrow u$. Khi người dùng thêm $u \rightarrow v$, BFS bắt đầu từ v , lần lượt tới x và u , từ đó từ chối cạnh mới. Nếu không tới được u , cạnh được xem là không tạo chu trình tại thời điểm kiểm tra.



Hình 5.2: Kiểm tra đường đi ngược trước khi thêm cạnh phụ thuộc

Chỉ khi toàn bộ điều kiện đạt, quan hệ mới được tạo. Hệ thống tiếp tục ghi nhật ký dưới ngữ cảnh thẻ bị chặn và phát sự kiện cập nhật cho cả thẻ chặn lẫn thẻ bị chặn. Quan hệ được hiển thị trong hộp thoại chi tiết thẻ và bằng đường nối trên Dòng thời gian.

Khi trạng thái `isCompleted` của một thẻ thay đổi, dịch vụ cập nhật truy vấn các quan hệ mà thẻ tham gia và phát sự kiện cho các thẻ liên quan. Các máy khách nhận sự kiện sẽ làm mới dữ liệu cần thiết; nhờ đó, số thẻ chặn chưa hoàn thành, trạng thái phụ thuộc và đường nối trên Dòng thời gian được tính lại từ dữ liệu mới.

5.2.3 Kết quả đạt được

Trong luồng xử lý tuần tự thông thường, giải pháp từ chối tự phụ thuộc, cạnh trùng, thẻ không hợp lệ trong phạm vi bảng và cạnh tạo chu trình. Ràng buộc khóa ngoại cùng ràng buộc duy nhất của Prisma bổ sung lớp bảo vệ ở mức lưu trữ, nhưng ràng buộc duy nhất chỉ ngăn cạnh trùng và không bảo vệ tính phi chu trình. Sau khi quan hệ hoặc trạng thái hoàn thành thay đổi, các thẻ liên quan được yêu cầu làm mới để giao diện phản ánh dữ liệu mới.

Kiểm tra chu trình và thao tác ghi cạnh hiện chưa nằm trong cùng một giao dịch có cơ chế cô lập hoặc khóa phù hợp. Nếu hai yêu cầu thêm cạnh chạy đồng thời, cả hai phép kiểm tra có thể đọc cùng một trạng thái cũ trước khi ghi. Vì vậy, khả năng ngăn chu trình đã được xác nhận từ luồng xử lý và mã nguồn hiện tại, nhưng chưa phải cam kết dưới tranh chấp đồng thời.

Hệ thống cũng chưa cưỡng chế thứ tự hoàn thành. Khi người dùng đánh dấu hoàn thành một thẻ còn bị chặn, giao diện liệt kê các thẻ chặn chưa hoàn thành và yêu cầu xác nhận. Người dùng vẫn có thể tiếp tục, còn dịch vụ phía máy chủ không từ chối cập nhật isCompleted dựa trên các quan hệ này. Cơ chế hiện tại vì thế phát hiện và cảnh báo điều kiện chặn, thay vì bắt buộc mọi thẻ tiền nhiệm phải hoàn thành.

Đóng góp của giải pháp là kết hợp mô hình đồ thị có hướng với kiểm tra nghiệp vụ phía máy chủ, ràng buộc toàn vẹn ở tầng lưu trữ và cơ chế làm mới dữ liệu liên quan trên giao diện. Sự phân lớp này xử lý được các sai lệch phổ biến trong luồng tuần tự, đồng thời chỉ ra rõ những trường hợp cần bổ sung cơ chế giao dịch nếu hệ thống được mở rộng cho tải đồng thời cao hơn.

5.3 Tích hợp trí tuệ nhân tạo theo nguyên tắc sinh bản nháp có ràng buộc

5.3.1 Bài toán đặt ra

Thông tin công việc ban đầu thường tồn tại dưới dạng thư điện tử, tin nhắn hoặc biên bản họp. Nội dung này có thể chứa nhiều câu, ngày tháng tương đối, tên viết tắt và các đầu việc chưa được phân tách. Việc chuyển nội dung đó thành tiêu đề, mô tả, checklist, hạn hoàn thành, nhãn và người phụ trách đòi hỏi nhiều thao tác thủ công.

Trong thiết kế HustFlow, mọi phản hồi từ dịch vụ AI được xem là dữ liệu đầu vào bên ngoài. Đối với các luồng có thể dẫn tới thay đổi dữ liệu, hệ thống không ghi trực tiếp phản hồi của mô hình mà thực hiện kiểm tra và yêu cầu người dùng xác nhận. Các trường hợp được kiểm soát bằng quy tắc tắt định gồm phản hồi sai cấu trúc JSON, định danh không thuộc bảng, đề xuất gán việc cho thành viên không

thuộc tập có thể nhận việc, chẳng hạn người có vai trò VIEWER, và ngày không hợp lệ. Đối với báo cáo AI, hệ thống giới hạn ngữ cảnh đầu vào, yêu cầu mô hình không tạo thông tin ngoài ngữ cảnh và kiểm tra cấu trúc phản hồi; tuy nhiên, cài đặt hiện tại chưa xác minh ngữ nghĩa từng nhận định hoặc số liệu trong báo cáo. Vì vậy, cơ sở dữ liệu và các quy tắc nghiệp vụ tại máy chủ vẫn là nguồn dữ liệu chuẩn, còn đầu ra AI cần được người dùng rà soát trước khi sử dụng.

5.3.2 Giải pháp đề xuất

Tất cả lời gọi OpenAI API [14] được thực hiện ở phía máy chủ nên khóa API không được chuyển tới trình duyệt. Mô-đun dùng chung gọi điểm cuối Chat Completions với thời gian chờ tối đa 25 giây và chuyển lỗi kỹ thuật thành thông báo phù hợp cho người dùng. Tên mô hình được đọc từ biến `OPENAI_MODEL`; giá trị mặc định của dự án là `gpt-4o-mini`. Cách cấu hình này cho phép thay đổi mô hình mà không làm thay đổi hợp đồng của các Server Action.

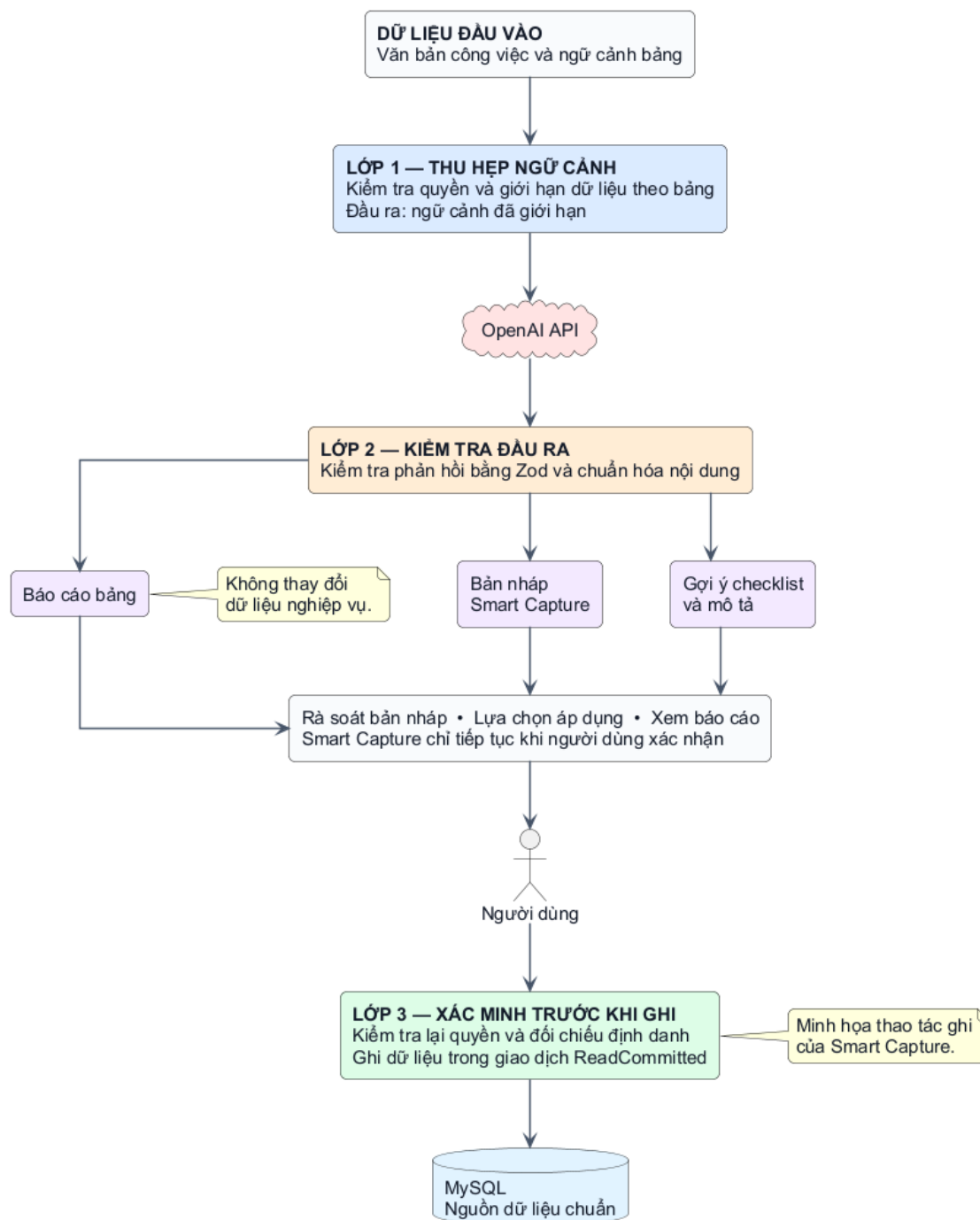
Giải pháp kiểm soát đầu ra AI qua ba lớp. Lớp thứ nhất thu hẹp ngữ cảnh trước khi gọi mô hình. Trong Smart Capture, máy chủ kiểm tra quyền chỉnh sửa và chỉ cung cấp văn bản đầu vào, thông tin thời gian cùng các danh sách, nhãn và thành viên thuộc đúng bảng. Thành viên có vai trò VIEWER không nằm trong tập có thể nhận việc. Ngữ cảnh chỉ cung cấp các định danh hợp lệ và prompt yêu cầu mô hình không tạo định danh mới.

Lớp thứ hai kiểm tra và chuẩn hóa phản hồi. Prompt yêu cầu mô hình trả JSON theo cấu trúc xác định trước; phản hồi sau đó được phân tích và kiểm tra bằng Zod. Các quy tắc bắt định tiếp tục giới hạn tiêu đề ở 200 ký tự, mô tả ở 3.000 ký tự, đồng thời chuẩn hóa, loại trùng và giới hạn checklist. Mô tả thiếu các đề mục bắt buộc, chứa khối mã hoặc vượt giới hạn được thay bằng nội dung dự phòng từ văn bản gốc.

Ngày do mô hình đề xuất được kiểm tra trước khi chuyển sang ISO 8601. Khi phản hồi chưa có độ lệch múi giờ, hệ thống có thể bổ sung giá trị như UTC+07:00; giá trị không hợp lệ bị loại bỏ. Đề xuất người phụ trách phải kèm bằng chứng từ văn bản đầu vào. Trường hợp bằng chứng khớp nhiều thành viên được trả về dưới dạng cảnh báo, còn định danh danh sách và nhãn ngoài phạm vi bị thay thế hoặc loại bỏ theo quy tắc của hệ thống.

Sau bước chuẩn hóa, Smart Capture chỉ trả về một bản nháp để người dùng rà soát và chỉnh sửa. Khi người dùng xác nhận, lớp thứ ba kiểm tra lại quyền chỉnh sửa, danh sách, thành viên và nhãn tại máy chủ. Thẻ, người phụ trách, nhãn và checklist được tạo trong giao dịch Prisma ở mức cô lập ReadCommitted; nhật ký và sự kiện `card.created` chỉ được phát sau khi giao dịch thành công. Hình 5.3 khái

quát ba lớp kiểm soát này.



Hình 5.3: Quy trình kiểm soát đầu ra của trợ lý AI

Ba lớp kiểm soát được áp dụng đầy đủ cho luồng Smart Capture vì luồng này có thể dẫn tới thao tác ghi. Với gợi ý checklist và hỗ trợ mô tả, giai đoạn AI chỉ tạo nội dung để người dùng lựa chọn hoặc chỉnh sửa; việc lưu nội dung được thực hiện bằng thao tác nghiệp vụ riêng và tiếp tục được kiểm tra tại máy chủ. Đối với báo cáo bằng, máy chủ tổng hợp trước các chỉ số, phân bổ công việc, thẻ quá hạn, tiến độ checklist và hoạt động gần đây. Prompt yêu cầu mô hình diễn giải trong phạm vi ngữ cảnh này và phản hồi được Zod kiểm tra về cấu trúc, nhưng hệ thống chưa có

bước đối chiếu tất định từng số liệu hoặc kết luận được tạo ra. Báo cáo AI không làm thay đổi dữ liệu nghiệp vụ.

5.3.3 Kết quả đạt được

HustFlow đã triển khai bốn nhóm chức năng gồm Smart Capture, gợi ý checklist, hỗ trợ tạo và cải thiện mô tả thể, cùng báo cáo bảng. Mỗi nhóm sử dụng prompt, lược đồ kiểm tra và giới hạn đầu vào, đầu ra phù hợp với nhiệm vụ. Nội dung do mô hình tạo ra không được tự động coi là dữ liệu hợp lệ: Smart Capture cần người dùng xác nhận, các gợi ý nội dung cần được lựa chọn hoặc chỉnh sửa, còn báo cáo được tạo từ ngữ cảnh do máy chủ chuẩn bị và vẫn cần người dùng rà soát.

Đóng góp của giải pháp không nằm ở việc gọi mô hình ngôn ngữ, mà ở cách chuyển đầu ra có tính xác suất thành dữ liệu được kiểm soát. Ba lớp thu hẹp ngữ cảnh, kiểm tra–chuẩn hóa phản hồi và xác minh trước khi ghi giữ quyết định cuối cùng ở người dùng và các quy tắc nghiệp vụ phía máy chủ.

Đồ án chưa thực hiện nghiên cứu người dùng để đo thời gian nhập liệu, tỷ lệ chấp nhận đề xuất hoặc chất lượng nội dung. Kết quả ngôn ngữ vẫn phụ thuộc vào mô hình, prompt và dữ liệu đầu vào. Riêng với báo cáo AI, kiểm tra Zod chỉ xác nhận cấu trúc và giới hạn của phản hồi, không chứng minh tính đúng đắn của từng nhận định hoặc số liệu; mô hình vẫn có thể diễn giải sai hoặc đưa ra kết luận chưa được dữ liệu chứng minh đầy đủ. Vì vậy, người dùng cần rà soát nội dung trước khi lưu hoặc sử dụng trong báo cáo.

Chương này đã phân tích ba đóng góp kỹ thuật của HustFlow. Mỗi giải pháp xử lý một dạng rủi ro khác nhau: trạng thái máy khách không đồng nhất, quan hệ phụ thuộc không hợp lệ và đầu ra AI không xác định. Các giải pháp đều giới hạn dữ liệu theo ngữ cảnh, kiểm tra quy tắc tại máy chủ và duy trì cơ sở dữ liệu làm nguồn dữ liệu chuẩn.

Kết quả cho thấy hệ thống đã hình thành các cơ chế bảo vệ có thể tái sử dụng giữa nhiều chức năng, đồng thời vẫn duy trì đường phục hồi về nguồn dữ liệu chuẩn. Những giới hạn về xử lý đồng thời, bảo đảm giao nhận sự kiện và đánh giá chất lượng AI cũng đã được xác định rõ. Trên cơ sở đó, Chương 6 sẽ tổng kết kết quả toàn đồ án và đề xuất các hướng hoàn thiện tiếp theo.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương 5 đã phân tích các giải pháp và đóng góp kỹ thuật nổi bật của HustFlow. Chương này tổng kết, đối chiếu và tự đánh giá kết quả của đề án, sau đó đề xuất các công việc hoàn thiện chức năng hiện có trước khi mở rộng hệ thống.

6.1 Kết luận

Đề án đặt ra mục tiêu xây dựng một nền tảng quản lý công việc cho các nhóm quy mô vừa và nhỏ, kết hợp quản lý đa góc nhìn, cộng tác thời gian thực, quan hệ phụ thuộc, hỗ trợ trí tuệ nhân tạo, phân quyền và giới hạn dịch vụ. Ứng dụng web HustFlow đã cài đặt các nhóm chức năng nêu trong Chương 1 và tổ chức dữ liệu theo phạm vi tổ chức, bảng, danh sách, thẻ.

So với Trello, HustFlow tương đồng ở mô hình Kanban và các góc nhìn Lịch, Dòng thời gian, Thống kê. Trello có hệ sinh thái tích hợp, luật tự động hóa, ứng dụng trên nhiều nền tảng và các chế độ xem theo từng gói dịch vụ [1]. HustFlow vì vậy không phải sản phẩm thay thế toàn diện, mà là nguyên mẫu kết hợp các góc nhìn tiến độ với quan hệ phụ thuộc, vai trò thành viên bảng và quy trình kiểm soát nội dung do AI tạo ra.

Jira cung cấp khả năng cấu hình quy trình, báo cáo, tự động hóa, phân quyền và lập kế hoạch liên nhóm; các gói cao hơn còn hỗ trợ quản trị quy mô lớn và cam kết mức dịch vụ [2]. Asana hỗ trợ Bảng, Lịch, Dòng thời gian, Gantt, quan hệ phụ thuộc và các chức năng AI [3]. HustFlow có phạm vi hẹp hơn và chưa có bằng chứng vận hành tương đương. Điểm khác biệt được nghiên cứu là sự kết hợp giữa cơ chế bảo vệ dữ liệu phía máy chủ và luồng AI sinh bản nháp có kiểm soát.

Về kết quả xây dựng, HustFlow đã triển khai năm chế độ xem gồm Kanban, Lịch, Dòng thời gian, Chia đôi và Thống kê trên cùng dữ liệu công việc. Hệ thống hỗ trợ quản lý vòng đời thẻ, cộng tác giữa thành viên, tìm kiếm, lọc, thông báo, lưu trữ và xuất dữ liệu. Các thao tác thay đổi dữ liệu được xử lý qua *Server Action* hoặc *Route Handler*; giao diện không truy cập trực tiếp cơ sở dữ liệu.

Ba đóng góp nổi bật tương ứng với các giải pháp ở Chương 5: (i) đồng bộ thời gian thực bằng kênh riêng tư, dữ liệu sự kiện có kiểu, kiểm tra quyền đăng ký, loại sự kiện trùng và các mức phục hồi về nguồn dữ liệu chuẩn; (ii) bảo vệ quan hệ phụ thuộc bằng kiểm tra phạm vi thẻ, ngăn tự phụ thuộc và cạnh trùng, kết hợp ràng buộc dữ liệu với tìm kiếm theo chiều rộng (BFS) để phát hiện chu trình; và (iii) kiểm soát đầu ra AI thông qua thu hẹp ngữ cảnh, kiểm tra–chuẩn hóa phản hồi và xác nhận của người dùng trước khi ghi dữ liệu. Cả ba giải pháp đều đặt việc kiểm

tra quyền và quy tắc nghiệp vụ tại máy chủ, đồng thời coi cơ sở dữ liệu là nguồn dữ liệu chuẩn.

Hệ thống vẫn còn một số giới hạn. Mã nguồn chưa có bộ kiểm thử tự động; chín ca kiểm thử hộp đen ở Chương 4 chỉ bao phủ ba chức năng trọng tâm. Đồ án chưa đo thời gian phản hồi, độ trễ đồng bộ, tải đồng thời, tỷ lệ lỗi hoặc hiệu quả sử dụng, và ứng dụng mới được chạy thử nghiệm trên máy phát triển cục bộ. Quan hệ phụ thuộc mới cảnh báo, chưa cưỡng chế thứ tự hoàn thành tại máy chủ. Stripe chỉ xử lý `checkout.session.completed` và `invoice.payment_succeeded`, còn chức năng AI chưa được đánh giá trên một bộ dữ liệu cố định.

Quá trình thực hiện đồ án cho thấy điều khiển trên giao diện không thể thay thế phân quyền và bất biến nghiệp vụ tại máy chủ. Đồng bộ thời gian thực cần vừa cập nhật nhanh, vừa có đường phục hồi về dữ liệu đã lưu. Nội dung do mô hình sinh ra phải được xem như dữ liệu bên ngoài, cần kiểm tra, giới hạn và xác nhận trước khi sử dụng. Báo cáo kỹ thuật cũng phải phân biệt chức năng đã cài đặt, phạm vi đã kiểm thử và kết quả đã đo lường để phản ánh đúng trạng thái sản phẩm.

Như vậy, HustFlow đã hiện thực hóa các nhóm chức năng được xác định trong phạm vi đề tài và hình thành ba cơ chế bảo vệ dữ liệu có thể tái sử dụng. Kết quả hiện có xác nhận nguyên mẫu hoạt động trong môi trường cục bộ đối với phạm vi đã đánh giá; các giới hạn về kiểm thử, đo lường và triển khai là cơ sở cho hướng phát triển tiếp theo.

6.2 Hướng phát triển

Ưu tiên trước mắt là hoàn thiện khả năng kiểm chứng các chức năng hiện có. Dự án cần bổ sung kiểm thử đơn vị, tích hợp và hồi quy cho phân quyền, cập nhật thẻ, quan hệ phụ thuộc, Smart Capture, Stripe webhook và quy tắc ngày. Kiểm thử trình duyệt cần bao phủ các vai trò cấp bảng ADMIN, MEMBER, VIEWER, nhiều phiên đồng thời và luồng phục hồi khi mất kết nối. Dữ liệu đầu vào, kết quả chạy và ảnh chụp trạng thái cần được lưu làm bằng chứng kiểm thử.

Quy trình bảo đảm chất lượng cần được tự động hóa bằng tích hợp liên tục, chạy lint, kiểm tra kiểu, build và test trước khi hợp nhất thay đổi. Các cảnh báo hiện tại cần được xử lý. Hệ thống cũng cần được đo trên bộ dữ liệu cố định ở nhiều mức tải theo thời gian phản hồi, độ trễ đồng bộ, tỷ lệ lỗi và tài nguyên, qua đó tạo cơ sở đánh giá khả năng mở rộng.

Bước tiếp theo là triển khai HustFlow trên môi trường vận hành có HTTPS và cơ sở dữ liệu được quản lý. Quy trình triển khai cần quản lý phiên bản lược đồ, sao lưu, khôi phục và khóa bí mật theo môi trường. Lỗi ứng dụng, truy vấn chậm, kết

nổi thời gian thực, lời gọi AI và webhook cần được thu thập tập trung để hạn chế sai lệch dữ liệu.

Các chức năng hiện có cần tiếp tục được hoàn thiện theo rủi ro nghiệp vụ. Stripe cần xử lý hủy thuê bao, thanh toán thất bại, hoàn tiền, tranh chấp và đối soát webhook bị bỏ lỡ. Quan hệ phụ thuộc có thể hỗ trợ chế độ cảnh báo hoặc cưỡng chế; ở chế độ cưỡng chế, điều kiện hoàn thành thẻ tiền nhiệm phải được kiểm tra tại máy chủ. AI cần được đánh giá bằng bộ dữ liệu tiếng Việt có nhiều dạng ngày, tên mơ hồ, nhãn không tồn tại, nội dung thiếu và phản hồi sai cấu trúc. Các chỉ số gồm tỷ lệ JSON hợp lệ, độ chính xác của ngày, người phụ trách và nhãn, tỷ lệ dùng nội dung dự phòng và tỷ lệ bản nháp được chấp nhận.

Sau khi chức năng hiện tại được kiểm chứng, hệ thống có thể bổ sung trường dữ liệu tùy chỉnh, mẫu bảng, tác vụ lặp, luật tự động hóa, danh mục dự án, góc nhìn nhiều bảng và phụ thuộc liên bảng. Những thay đổi này phải mở rộng mô hình phân quyền và phạm vi kiểm tra chu trình, do người dùng có thể có quyền khác nhau trên thẻ nguồn, thẻ đích và từng bảng liên quan.

HustFlow cũng có thể tích hợp lịch, thư điện tử, công cụ trao đổi nhóm và kho mã nguồn; đồng thời nghiên cứu ứng dụng web tiến bộ, giao diện di động hoặc chế độ ngoại tuyến có kiểm soát. Mỗi tích hợp cần xác minh webhook, chống xử lý lặp và giới hạn dữ liệu theo quyền. Các hướng AI như tìm kiếm ngữ nghĩa, hỏi–đáp và dự báo rủi ro phải cung cấp căn cứ, tuân thủ quyền truy cập và yêu cầu xác nhận trước khi thay đổi dữ liệu.

Lộ trình phát triển cần đi từ kiểm chứng chức năng hiện tại, hoàn thiện khả năng vận hành, đến mở rộng quy trình, tích hợp và hỗ trợ thông minh trên nền tảng các cơ chế kiểm soát đã xây dựng.

TÀI LIỆU THAM KHẢO

- [1] Atlassian, *Trello plans and pricing*. Accessed: Jun. 22, 2026. [Online]. Available: <https://trello.com/pricing>
- [2] Atlassian, *Jira plans and pricing*. Accessed: Jun. 22, 2026. [Online]. Available: <https://www.atlassian.com/software/jira/pricing>
- [3] Asana, Inc., *Asana plans and pricing*. Accessed: Jun. 22, 2026. [Online]. Available: <https://asana.com/pricing>
- [4] TanStack, *TanStack Query documentation*. Accessed: Jun. 22, 2026. [Online]. Available: <https://tanstack.com/query/latest/docs/framework/react/overview>
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [6] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970. DOI: 10.1145/362384.362685
- [7] Vercel Inc., *Next.js documentation*. Accessed: Jun. 22, 2026. [Online]. Available: <https://nextjs.org/docs>
- [8] Meta Platforms Inc., *React documentation*. Accessed: Jun. 22, 2026. [Online]. Available: <https://react.dev>
- [9] P. DuBois, *MySQL*, 5th ed. Addison-Wesley Professional, 2013.
- [10] Prisma Data Inc., *Prisma ORM documentation*. Accessed: Jun. 22, 2026. [Online]. Available: <https://www.prisma.io/docs>
- [11] Pusher, *Pusher Channels documentation*. Accessed: Jun. 22, 2026. [Online]. Available: <https://pusher.com/docs/channels/>
- [12] Clerk Inc., *Clerk documentation*. Accessed: Jun. 22, 2026. [Online]. Available: <https://clerk.com/docs>
- [13] Stripe Inc., *Stripe API reference*. Accessed: Jun. 22, 2026. [Online]. Available: <https://docs.stripe.com/api>
- [14] OpenAI, *OpenAI API documentation*. Accessed: Jun. 22, 2026. [Online]. Available: <https://developers.openai.com/api/docs>
- [15] Hello Pangea, *@hello-pangea/dnd: Drag and drop for React*. Accessed: Jun. 22, 2026. [Online]. Available: <https://github.com/hello-pangea/dnd>
- [16] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. O’Reilly Media, 2020.
- [17] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O’Reilly Media, 2021.